

Java

Java is a **programming language** and a **platform**.

Java is a high level, robust, object-oriented and secure programming language.

Platform: Any hardware or software environment in which a program runs, is known as a platform. Since Java has a runtime environment (JRE) and API, it is called a platform.

Applications

According to Sun, 3 billion devices run Java. There are many devices where Java is currently used. Some of them are as follows:

1. Desktop Applications such as acrobat reader, media player, antivirus, etc.
2. Web Applications such as irctc.co.in, javatpoint.com, etc.
3. Enterprise Applications such as banking applications.
4. Mobile
5. Embedded System
6. Smart Card
7. Robotics
8. Games, etc.

Types of Java Applications

There are mainly 4 types of applications that can be created using Java programming:

1) Standalone Application

Standalone applications are also known as desktop applications or window-based applications. These are traditional software that we need to install on every machine. Examples of standalone application are Media player, antivirus, etc. AWT and Swing are used in Java for creating standalone applications.

2) Web Application

An application that runs on the server side and creates a dynamic page is called a web application. Currently, Servlet, JSP, Struts, Spring, Hibernate, JSF, etc. technologies are used for creating web applications in Java.

3) Enterprise Application

An application that is distributed in nature, such as banking applications, etc. is called enterprise application. It has advantages of the high-level security, load balancing, and clustering. In Java, **EJB** is used for creating enterprise applications.

4) Mobile Application

An application which is created for mobile devices is called a mobile application. Currently, Android and Java ME are used for creating mobile applications.

Java Platforms / Editions

There are 4 platforms or editions of Java:

1) Java SE (Java Standard Edition)

It is a Java programming platform. It includes Java programming APIs such as java.lang, java.io, java.net, java.util, java.sql, java.math etc. It includes core topics like OOPs, String, Regex, Exception, Inner classes, Multithreading, I/O Stream, Networking, AWT, Swing, Reflection, Collection, etc.

2) Java EE (Java Enterprise Edition)

It is an enterprise platform which is mainly used to develop web and enterprise applications. It is built on the top of the Java SE platform. It includes topics like Servlet, JSP, Web Services, EJB, JPA, etc.

3) Java ME (Java Micro Edition)

It is a micro platform which is mainly used to develop mobile applications.

4) JavaFX

It is used to develop rich internet applications. It uses a light-weight user interface API.

Java Version History

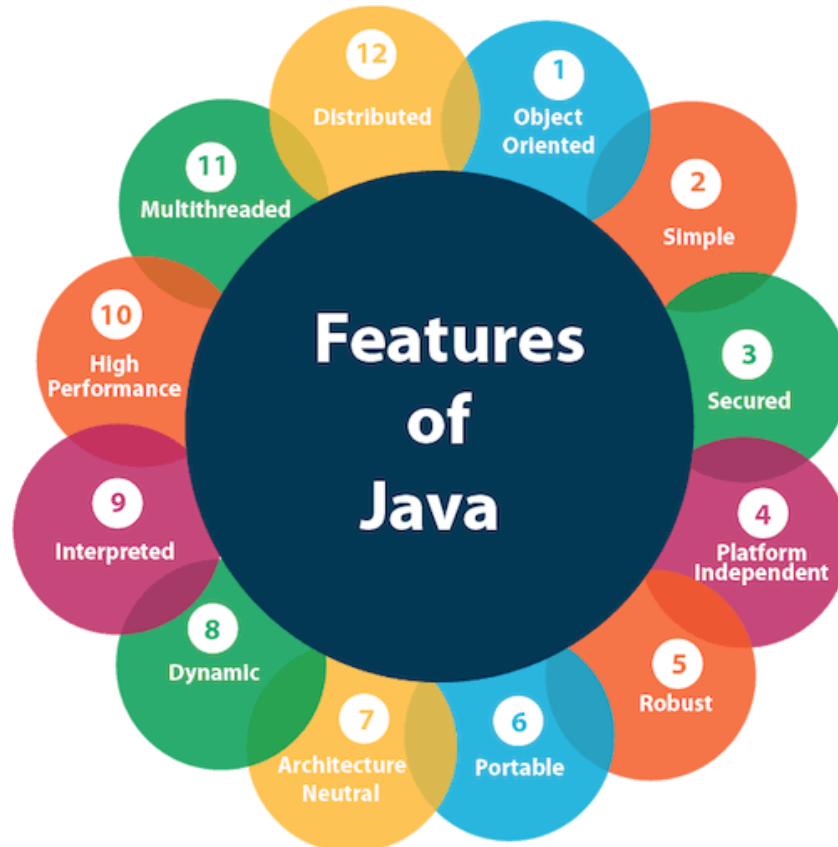
Many java versions have been released till now. The current stable release of Java is Java SE 10.

1. JDK Alpha and Beta (1995)
2. JDK 1.0 (23rd Jan 1996)
3. JDK 1.1 (19th Feb 1997)
4. J2SE 1.2 (8th Dec 1998)
5. J2SE 1.3 (8th May 2000)
6. J2SE 1.4 (6th Feb 2002)
7. J2SE 5.0 (30th Sep 2004)
8. Java SE 6 (11th Dec 2006)
9. Java SE 7 (28th July 2011)
10. Java SE 8 (18th March 2014)
11. Java SE 9 (21st Sep 2017)
12. Java SE 10 (20th March 2018)

Features of Java

The primary objective of Java programming language creation was to make it portable, simple and secure programming language. Apart from this, there are also some excellent features which play an important role in the popularity of this language. The features of Java are also known as *java buzzwords*.

A list of most important features of Java language is given below.



Simple

Java is very easy to learn, and its syntax is simple, clean and easy to understand. According to Sun, Java language is a simple programming language because:

- Java syntax is based on C++ (so easier for programmers to learn it after C++).
- Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.
- There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

Object-oriented

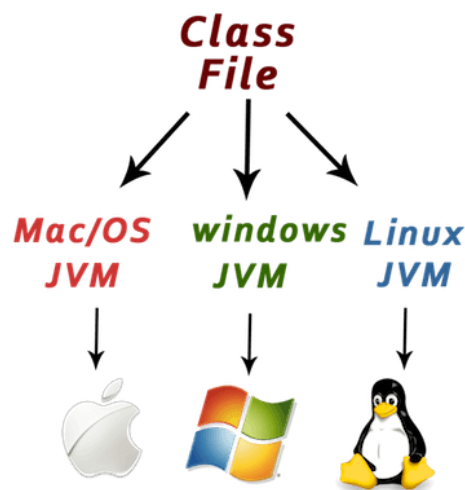
Java is an object-oriented programming language. Everything in Java is an object. Object-oriented means we organize our software as a combination of different types of objects that incorporates both data and behavior.

Object-oriented programming (OOPs) is a methodology that simplifies software development and maintenance by providing some rules.

Basic concepts of OOPs are:

1. Object
2. Class
3. Inheritance
4. Polymorphism
5. Abstraction
6. Encapsulation

Platform Independent



Java is platform independent because it is different from other languages like C, C++, etc. which are compiled into platform specific machines while Java is a write once, run anywhere language. A platform is the hardware or software environment in which a program runs.

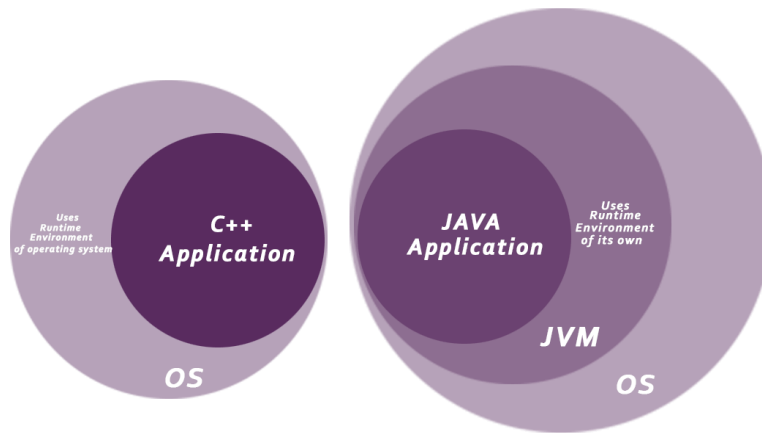
There are two types of platforms software-based and hardware-based. Java provides a software-based platform.

The Java platform differs from most other platforms in the sense that it is a software-based platform that runs on the top of other hardware-based platforms. It has two components:

1. Runtime Environment
2. API(Application Programming Interface)

Java code can be run on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc. Java code is compiled by the compiler and converted into bytecode. This bytecode is a platform-independent code because it can be run on multiple platforms, i.e., Write Once and Run Anywhere(WORA).

Secured



Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:

- **No explicit pointer**
- **Java Programs run inside a virtual machine sandbox**
- **ClassLoader:** Classloader in Java is a part of the Java Runtime Environment(JRE) which is used to load Java classes into the Java Virtual Machine dynamically. It adds security by separating the package for the classes of the local file system from those that are imported from network sources.
- **Bytecode Verifier:** It checks the code fragments for illegal code that can violate access right to objects.
- **Security Manager:** It determines what resources a class can access such as reading and writing to the local disk.

Java language provides these securities by default. Some security can also be provided by an application developer explicitly through SSL, JAAS, Cryptography, etc.

Robust

Robust simply means strong. Java is robust because:

- It uses strong memory management.
- There is a lack of pointers that avoids security problems.
- There is automatic garbage collection in java which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
- There are exception handling and the type checking mechanism in Java. All these points make Java robust.

Architecture-neutral

Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed.

In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture. However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

Portable

Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.

High-performance

Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code. It is still a little bit slower than a compiled language (e.g., C++). Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.

Distributed

Java is distributed because it facilitates users to create distributed applications in Java. RMI and EJB are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.

Multi-threaded

A thread is like a separate program, executing concurrently. We can write Java programs that deal with many tasks at once by defining multiple threads. The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area. Threads are important for multi-media, Web applications, etc.

Dynamic

Java is a dynamic language. It supports dynamic loading of classes. It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++.

Java supports dynamic compilation and automatic memory management (garbage collection).

C++ vs Java

There are many differences and similarities between the [C++ programming](#) language and [Java](#). A list of top differences between C++ and Java are given below:

Comparison Index	C++	Java
Platform-independent	C++ is platform-dependent.	Java is platform-independent.
Mainly used for	C++ is mainly used for system programming.	Java is mainly used for application programming. It is widely used in window, web-based, enterprise and mobile applications.

Design Goal	C++ was designed for systems and applications programming. It was an extension of C programming language .	Java was designed and created as an interpreter for printing systems but later extended as a support network computing. It was designed with a goal of being easy to use and accessible to a broader audience.
Goto	C++ supports the goto statement.	Java doesn't support the goto statement.
Multiple inheritance	C++ supports multiple inheritance.	Java doesn't support multiple inheritance through class. It can be achieved by interfaces in java .
Operator Overloading	C++ supports operator overloading .	Java doesn't support operator overloading.
Pointers	C++ supports pointers . You can write pointer program in C++.	Java supports pointer internally. However, you can't write the pointer program in java. It means java has restricted pointer support in java.
Compiler and Interpreter	C++ uses compiler only. C++ is compiled and run using the compiler which converts source code into machine code so, C++ is platform dependent.	Java uses compiler and interpreter both. Java source code is converted into bytecode at compilation time. The interpreter executes this bytecode at runtime and produces output. Java is interpreted that is why it is platform independent.
Call by Value and Call by reference	C++ supports both call by value and call by reference.	Java supports call by value only. There is no call by reference in java.
Structure and Union	C++ supports structures and unions.	Java doesn't support structures and unions.
Thread Support	C++ doesn't have built-in support for threads. It relies on third-party libraries for thread support.	Java has built-in thread support.
Documentation comment	C++ doesn't support documentation comment.	Java supports documentation comment (<code>/** ... */</code>) to create documentation for java source code.
Virtual Keyword	C++ supports virtual keyword so that we can decide whether or not override a function.	Java has no virtual keyword. We can override all non-static methods by default. In other words, non-static methods are virtual by default.

unsigned right shift >>>	C++ doesn't support >>> operator.	Java supports unsigned right shift >>> operator that fills zero at the top for the negative numbers. For positive numbers, it works same like >> operator.
Inheritance Tree	C++ creates a new inheritance tree always.	Java uses a single inheritance tree always because all classes are the child of Object class in java. The object class is the root of the inheritance tree in java.
Hardware	C++ is nearer to hardware.	Java is not so interactive with hardware.
Object-oriented	C++ is an object-oriented language. However, in C language, single root hierarchy is not possible.	Java is also an object-oriented language. However, everything (except fundamental types) is an object in Java. It is a single root hierarchy as everything gets derived from java.lang.Object.

Creating Hello World Example

Let's create the hello java program:

```
class Simple
{
    public static void main(String args[])
    {
        System.out.println("Hello Java");
    }
}
```

save this file as Simple.java

To compile: javac Simple.java

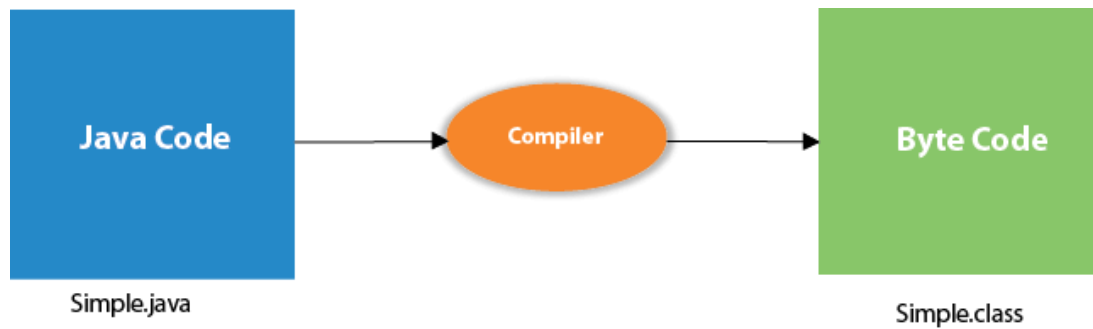
To execute: java Simple

Output:

Hello Java

Compilation Flow:

When we compile Java program using javac tool, java compiler converts the source code into byte code.



Parameters used in First Java Program

Let's see what is the meaning of class, public, static, void, main, String[], System.out.println().

- **class** keyword is used to declare a class in java.
- **public** keyword is an access modifier which represents visibility. It means it is visible to all.
- **static** is a keyword. If we declare any method as static, it is known as the static method. The core advantage of the static method is that there is no need to create an object to invoke the static method. The main method is executed by the JVM, so it doesn't require to create an object to invoke the main method. So it saves memory.
- **void** is the return type of the method. It means it doesn't return any value.
- **main** represents the starting point of the program.
- **String[] args** is used for command line argument. We will learn it later.
- **System.out.println()** is used to print statement. Here, System is a class, out is the object of PrintStream class, println() is the method of PrintStream class. We will learn about the internal working of System.out.println statement later.

How many ways can we write a Java program

There are many ways to write a Java program. The modifications that can be done in a Java program are given below:

1) By changing the sequence of the modifiers, method prototype is not changed in Java.

Let's see the simple code of the main method.

```
static public void main(String args[])
```

2) The subscript notation in Java array can be used after type, before the variable or after the variable.

Let's see the different codes to write the main method.

```
public static void main(String[] args)
```

```
public static void main(String []args)
```

```
public static void main(String args[])
```

3) You can provide var-args support to the main method by passing 3 ellipses (dots)

Let's see the simple code of using var-args in the main method. We will learn about var-args later in Java New Features chapter.

```
public static void main(String... args)
```

4) Having a semicolon at the end of class is optional in Java.

Let's see the simple code.

```
class A{  
  
static public void main(String... args){  
  
System.out.println("hello java4");  
  
}  
  
};
```

Valid java main method signature

```
public static void main(String[] args)  
public static void main(String []args)  
public static void main(String args[])  
public static void main(String... args)  
static public void main(String[] args)  
public static final void main(String[] args)  
final public static void main(String[] args)  
final strictfp public static void main(String[] args)
```

Invalid java main method signature

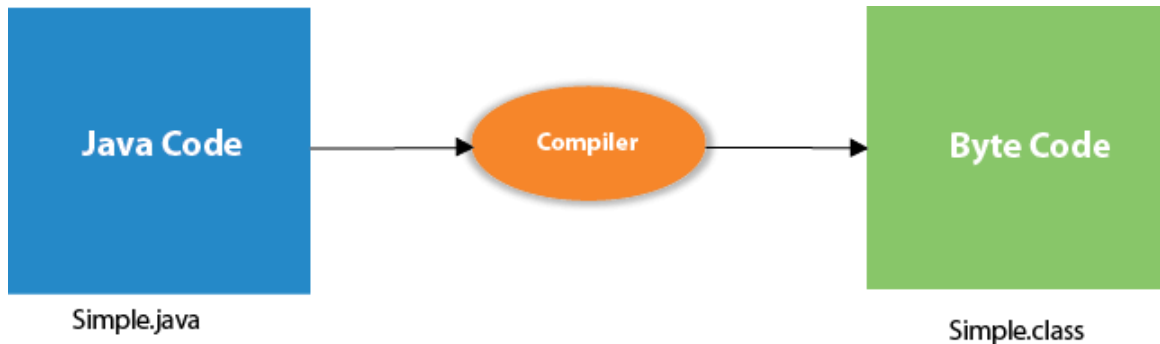
```
public void main(String[] args)  
static void main(String[] args)  
public void static main(String[] args)  
abstract public static void main(String[] args)
```

Internal Details of Hello Java Program

In the previous page, we have learnt about the first program, how to compile and run the first java program. Here, we are going to learn, what happens while compiling and running the java program. Moreover, we will see some question based on the first program.

What happens at compile time?

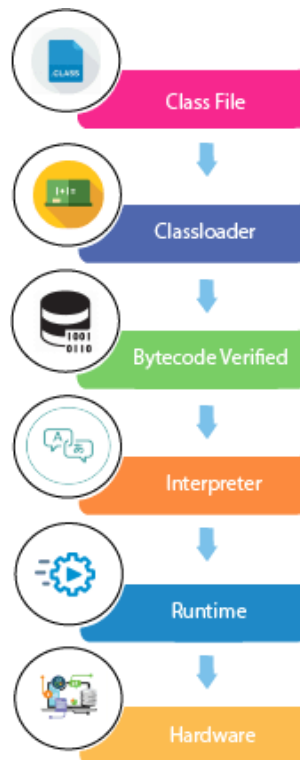
At



compile time, java file is compiled by Java Compiler (It does not interact with OS) and converts the java code into bytecode.

What happens at runtime?

At runtime, following steps are performed:



Classloader: is the subsystem of JVM that is used to load class files.

Bytecode Verifier: checks the code fragments for illegal code that can violate access right to objects.

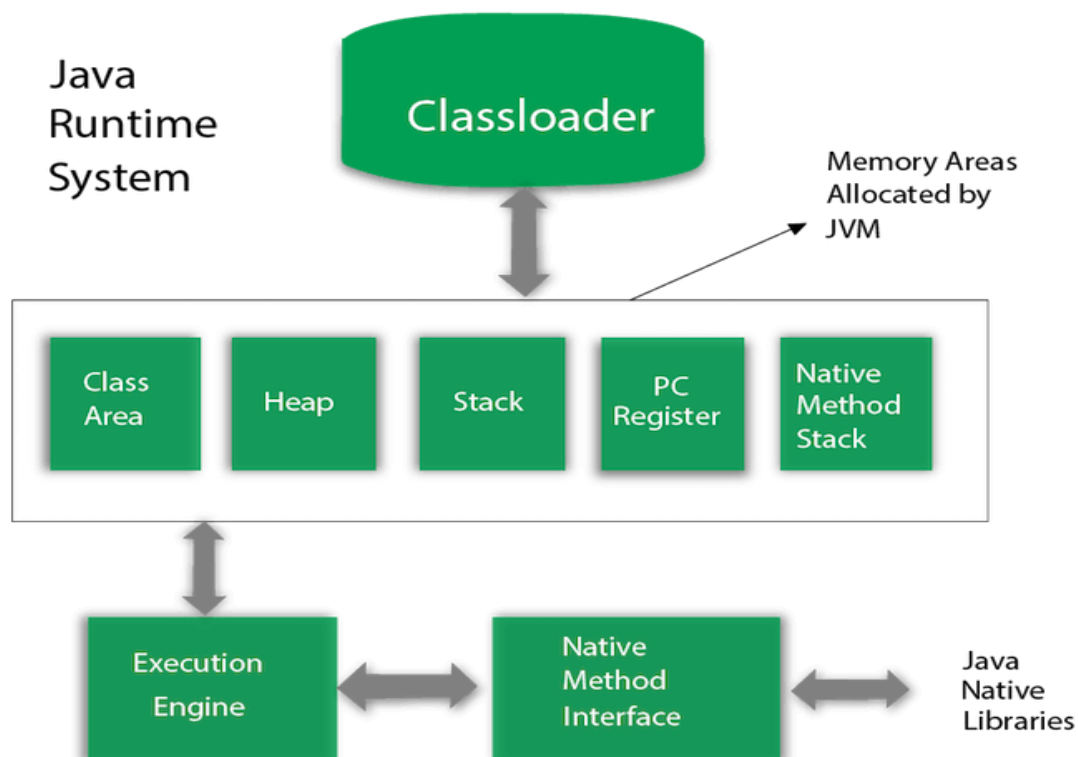
Interpreter: read bytecode stream then execute the instructions.

Difference between JDK, JRE, and JVM

JVM

JVM (Java Virtual Machine) is an abstract machine. It is called a virtual machine because it doesn't physically exist. It is a specification that provides a runtime environment in which Java bytecode can be executed. It can also run those programs which are written in other languages and compiled to Java bytecode.

JVMs are available for many hardware and software platforms. JVM, JRE, and JDK are platform dependent because the configuration of each OS is different from each other. However, Java is platform independent. There are three notions of the JVM: *specification*, *implementation*, and *instance*.



The JVM performs the following main tasks:

- Loads code
- Verifies code
- Executes code
- Provides runtime environment

JVM provides definitions for the:

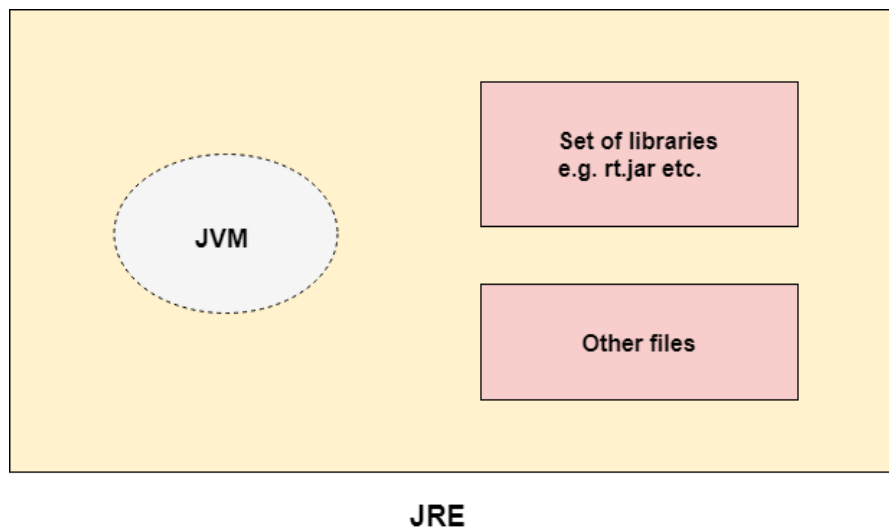
- Memory area

- Class file format
- Register set
- Garbage-collected heap
- Fatal error reporting etc.

JRE

JRE is an acronym for Java Runtime Environment. It is also written as Java RTE. The Java Runtime Environment is a set of software tools which are used for developing Java applications. It is used to provide the runtime environment. It is the implementation of JVM. It physically exists. It contains a set of libraries + other files that JVM uses at runtime.

The implementation of JVM is also actively released by other companies besides Sun Micro Systems.



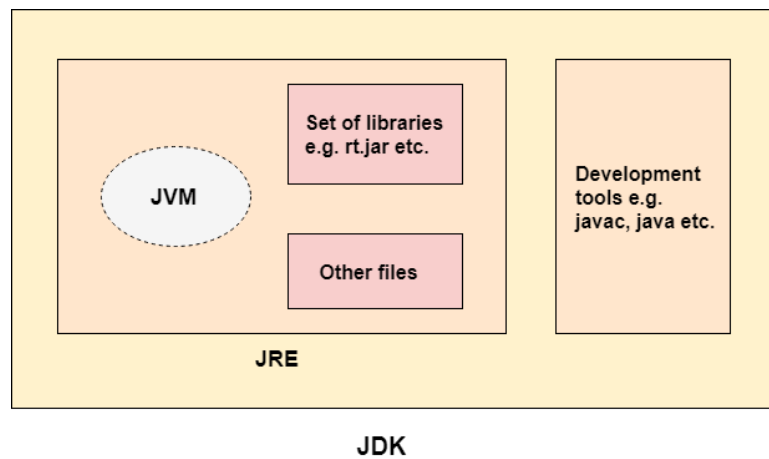
JDK

JDK is an acronym for Java Development Kit. The Java Development Kit (JDK) is a software development environment which is used to develop Java applications and [applets](#). It physically exists. It contains JRE + development tools.

JDK is an implementation of any one of the below given Java Platforms released by Oracle Corporation:

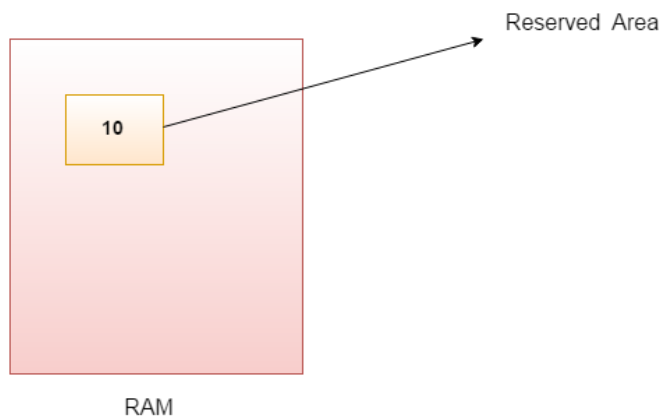
- Standard Edition Java Platform
- Enterprise Edition Java Platform
- Micro Edition Java Platform

The JDK contains a private Java Virtual Machine (JVM) and a few other resources such as an interpreter/loader (java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc), etc. to complete the development of a Java Application.



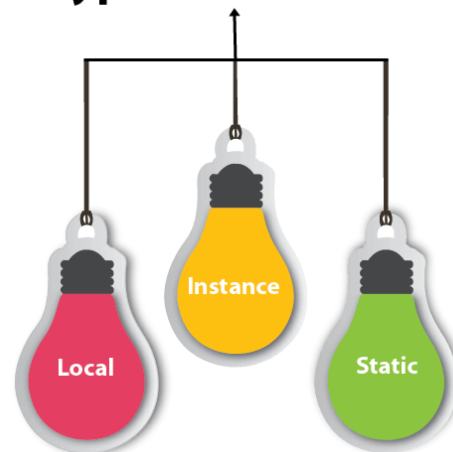
Java Variables

Variable is name of *reserved area allocated in memory*. In other words, it is a *name of memory location*. It is a combination of "vary + able" that means its value can be changed.



```
int data=10; //Here data is variable
```

Types of Variables



1) Local Variable

A variable declared inside the body of the method is called local variable. You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

A local variable cannot be defined with "static" keyword.

2) Instance Variable

A variable declared inside the class but outside the body of the method, is called instance variable. It is not declared as static.

It is called instance variable because its value is instance specific and is not shared among instances.

3) Static variable

A variable which is declared as static is called static variable. It cannot be local. You can create a single copy of static variable and share among all the instances of the class. Memory allocation for static variable happens only once when the class is loaded in the memory.

```
class A
{
    int data = 50; //instance variable
    static int m = 100; //static variable
    void method()
    {
        int n = 90; //local variable
    }
} //end of class
```

Java Variable Example: Widening

```
class Simple
{
    public static void main(String[] args)
    {
        int a = 10;
        float f = a;
        System.out.println(a);
        System.out.println(f);
    }
}
```

Output:

10

10.0

Java Variable Example: Narrowing (Typecasting)

```
class Simple
{
    public static void main(String[] args)
    {
        float f = 10.5f;
        //int a = f;//Compile time error
        int a = (int)f;
        System.out.println(f);
        System.out.println(a);
    }
}
```

Output:

10.5

10

Java Variable Example: Overflow

```
class Simple
{
    public static void main(String[] args)
    {
        //Overflow
        int a = 130;
        byte b = (byte)a;
        System.out.println(a);
        System.out.println(b);
    }
}
```

Output:

130

-126

Java Variable Example: Adding Lower Type

```
class Simple
{
    public static void main(String[] args)
    {
        byte a = 10;
        byte b = 10;
        //byte c = a + b;//Compile Time Error: because a+b=20 will be int
        byte c = (byte) (a + b);
        System.out.println(c);
    }
}
```

Output:

20

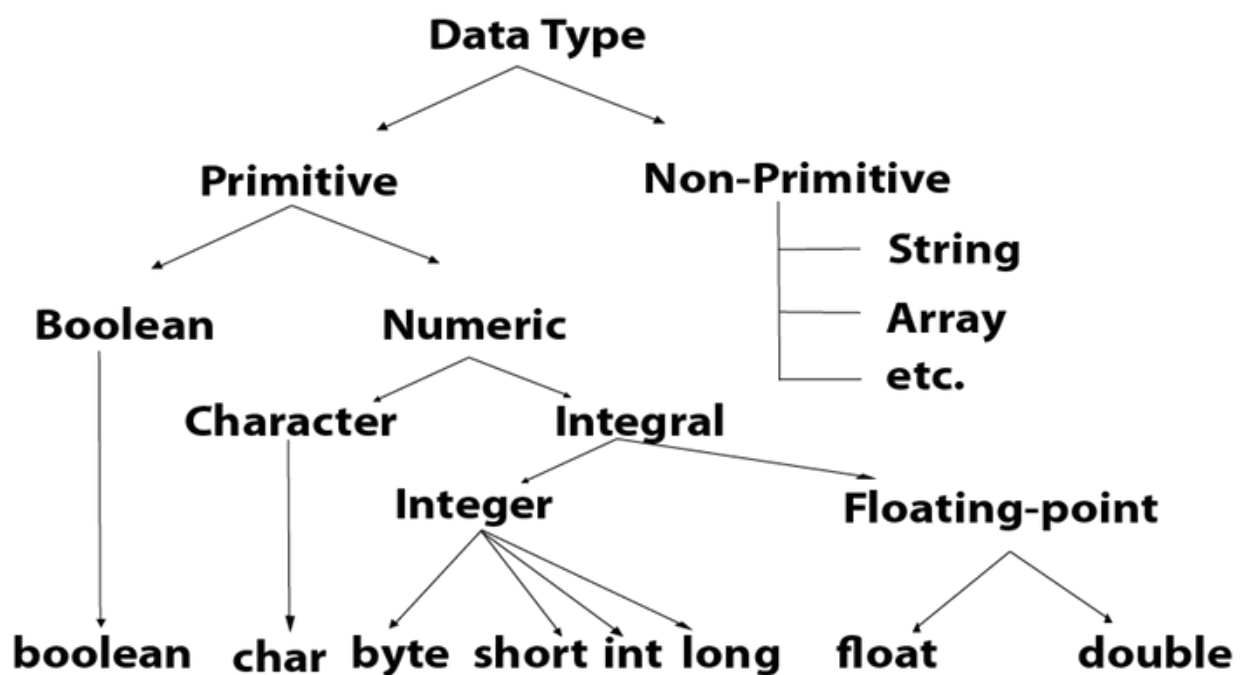
Data Types in Java

Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java:

1. **Primitive data types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
2. **Non-primitive data types:** The non-primitive data types include Classes, Interfaces, and Arrays.

Java Primitive Data Types

In Java language, primitive data types are the building blocks of data manipulation. These are the most basic data types available in Java language.



Data Type	Default Value	Default size
boolean	false	1 bit
char	'\u0000'	2 byte
byte	0	1 byte
short	0	2 byte
int	0	4 byte
long	0L	8 byte
float	0.0f	4 byte

double	0.0d	8 byte
--------	------	--------

Boolean

The Boolean data type is used to store only two possible values: true and false. This data type is used for simple flags that track true/false conditions.

The Boolean data type specifies one bit of information, but its "size" can't be defined precisely.

Example: Boolean one = false

Byte

The byte data type is an example of primitive data type. It is an 8-bit signed two's complement integer. Its value-range lies between -128 to 127 (inclusive). Its minimum value is -128 and maximum value is 127. Its default value is 0.

The byte data type is used to save memory in large arrays where the memory savings is most required. It saves space because a byte is 4 times smaller than an integer. It can also be used in place of "int" data type.

Example: byte a = 10, byte b = -20

Short

The short data type is a 16-bit signed two's complement integer. Its value-range lies between -32,768 to 32,767 (inclusive). Its minimum value is -32,768 and maximum value is 32,767. Its default value is 0.

The short data type can also be used to save memory just like byte data type. A short data type is 2 times smaller than an integer.

Example: short s = 10000, short r = -5000

Int

The int data type is a 32-bit signed two's complement integer. Its value-range lies between -2,147,483,648 (-2^{31}) to 2,147,483,647 ($2^{31} - 1$) (inclusive). Its minimum value is -2,147,483,648 and maximum value is 2,147,483,647. Its default value is 0.

The int data type is generally used as a default data type for integral values unless if there is no problem about memory.

Example: int a = 100000, int b = -200000

Long

The long data type is a 64-bit two's complement integer. Its value-range lies between $-9,223,372,036,854,775,808$ (-2^{63}) to $9,223,372,036,854,775,807$ ($2^{63} - 1$) (inclusive). Its minimum value is $-9,223,372,036,854,775,808$ and maximum value is $9,223,372,036,854,775,807$. Its default value is 0. The long data type is used when you need a range of values more than those provided by int.

Example: `long a = 100000L, long b = -200000L`

Float

The float data type is a single-precision 32-bit IEEE 754 floating point. Its value range is unlimited. It is recommended to use a float (instead of double) if you need to save memory in large arrays of floating point numbers. The float data type should never be used for precise values, such as currency. Its default value is 0.0F.

Example: `float f1 = 234.5f`

Double

The double data type is a double-precision 64-bit IEEE 754 floating point. Its value range is unlimited. The double data type is generally used for decimal values just like float. The double data type also should never be used for precise values, such as currency. Its default value is 0.0d.

Example: `double d1 = 12.3`

Char

The char data type is a single 16-bit Unicode character. Its value-range lies between '\u0000' (or 0) to '\uffff' (or 65,535 inclusive). It is because java uses Unicode system not ASCII code system. The \u0000 is the lowest range of Unicode system. The char data type is used to store characters.

Example: `char letterA = 'A'`

Operators in java

Operator in java is a symbol that is used to perform operations. For example: +, -, *, / etc.

There are many types of operators in java which are given below:

- Unary Operator,
- Arithmetic Operator,
- Shift Operator,
- Relational Operator,
- Bitwise Operator,

- Logical Operator,
- Ternary Operator and
- Assignment Operator.

Operator Type	Category	Precedence
Unary	postfix	<code>expr++ expr--</code>
	prefix	<code>++expr --expr +expr -expr ~ !</code>
Arithmetic	multiplicative	<code>* / %</code>
	additive	<code>+ -</code>
Shift	shift	<code><< >> >>></code>
Relational	comparison	<code>< > <= >= instanceof</code>
	equality	<code>== !=</code>
Bitwise	bitwise AND	<code>&</code>
	bitwise exclusive OR	<code>^</code>
	bitwise inclusive OR	<code> </code>
Logical	logical AND	<code>&&</code>
	logical OR	<code> </code>
Ternary	ternary	<code>? :</code>
Assignment	assignment	<code>= += -= *= /= %= &= ^= = <<= >>= >>>=</code>

Control Statements in Java

if Statement

The Java if statement tests the condition. It executes the *if block* if condition is true.

Syntax:

```
if(condition)
{
    //code to be executed
}
```

if-else Statement

The Java if-else statement also tests the condition. It executes the *if block* if condition is true otherwise *else block* is executed.

Syntax:

```
if(condition)
{
    //code if condition is true
}
Else
{
    //code if condition is false
}
```

if-else-if ladder

The if-else-if ladder statement executes one condition from multiple statements.

Syntax:

```
if(condition1)
{
    //code to be executed if condition1 is true
}
else if(condition2)
{
    //code to be executed if condition2 is true
}
else if(condition3)
{
    //code to be executed if condition3 is true
}
...
else
{
    //code to be executed if all the conditions are false
}
```

Nested if

The nested if statement represents the *if block within another if block*. Here, the inner if block condition executes only when outer if block condition is true.

Syntax:

```
if(condition)
{
    //code to be executed
    if(condition)
    {
        //code to be executed
    }
}
```

Switch Statement

The Java *switch statement* executes one statement from multiple conditions. It is like if-else-if ladder statement. The switch statement works with byte, short, int, long, enum types, String and some wrapper types like Byte, Short, Int, and Long. Since Java 7, you can use strings in the switch statement.

In other words, the switch statement tests the equality of a variable against multiple values.

Points to Remember

- There can be *one or N number of case values* for a switch expression.
- The case value must be of switch expression type only. The case value must be *literal or constant*. It doesn't allow variables.
- The case values must be *unique*. In case of duplicate value, it renders compile-time error.
- The Java switch expression must be of *byte, short, int, long (with its Wrapper type), enums and string*.
- Each case statement can have a *break statement* which is optional. When control reaches to the break statement, it jumps the control after the switch expression. If a break statement is not found, it executes the next case.
- The case value can have a *default label* which is optional.

Syntax:

```
switch(expression)
{
    case value1:
        //code to be executed;
        break; //optional
    case value2:
        //code to be executed;
        break; //optional
}
```

```
.....

    default:
        code to be executed if all cases are not matched;
}
```

Switch Statement with String

Java allows us to use strings in switch expression since Java SE 7. The case statement should be string literal.

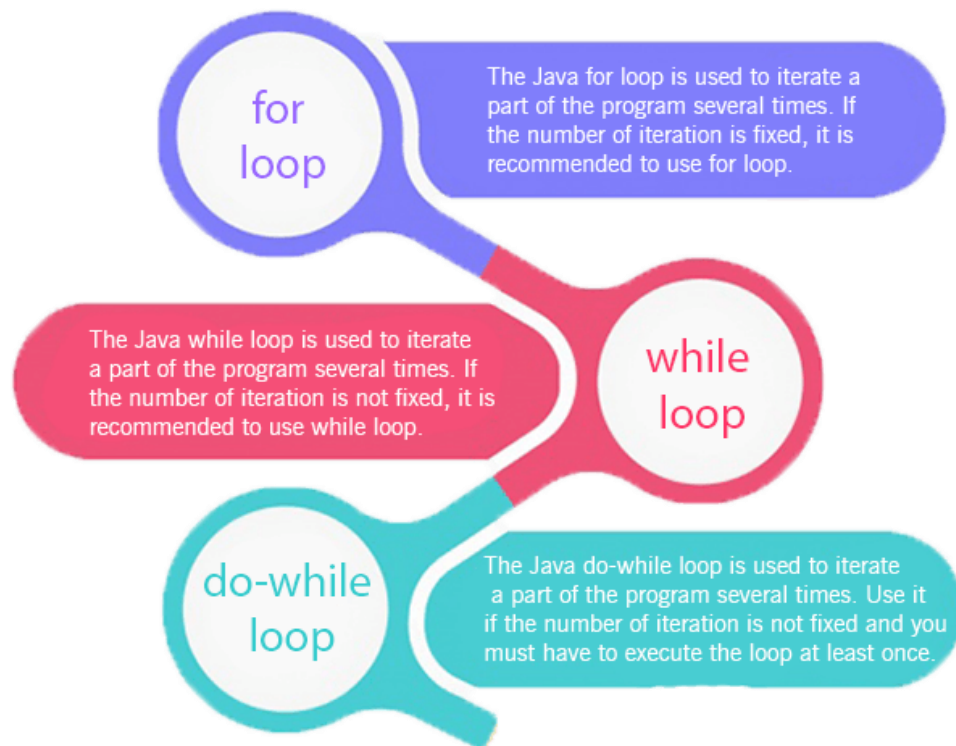
Example:

```
//Java Program to demonstrate the use of Java Switch
//statement with String
public class SwitchStringExample
{
    public static void main(String[] args)
    {
        //Declaring String variable
        String levelString = "Expert";
        int level = 0;
        //Using String in Switch expression
        switch(levelString)
        {
            //Using String Literal in Switch case
            case "Beginner":
                level = 1;
                break;
            case "Intermediate":
                level = 2;
                break;
            case "Expert":
                level = 3;
                break;
            default:
                level = 0;
                break;
        }
        System.out.println("Your Level is: "+level);
    }
}
```

Loops in Java

In programming languages, loops are used to execute a set of instructions/functions repeatedly when some conditions become true. There are three types of loops in java.

- for loop
- while loop
- do-while loop



For Loop vs While Loop vs Do While Loop

Comparison	for loop	while loop	do while loop
Introduction	The Java for loop is a control flow statement that iterates a part of the programs multiple times.	The Java while loop is a control flow statement that executes a part of the programs repeatedly on the basis of given boolean condition.	The Java do while loop is a control flow statement that executes a part of the programs at least once and the further execution depends upon the given boolean condition.
When to use	If the number of iteration is fixed, it is recommended to use for loop.	If the number of iteration is not fixed, it is recommended to use while loop.	If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use the do-while loop.

Syntax	<pre>for(init;condition;incr/decr){ // code to be executed }</pre>	<pre>while(condition){ //code to be executed }</pre>	<pre>do{ //code to be executed }while(condition);</pre>
Example	<pre>//for loop for(int i=1;i<=10;i++){ System.out.println(i); }</pre>	<pre>//while loop int i=1; while(i<=10){ System.out.println(i); i++; }</pre>	<pre>//do-while loop int i=1; do{ System.out.println(i); i++; }while(i<=10);</pre>
Syntax for infinitive loop	<pre>for(;;){ //code to be executed }</pre>	<pre>while(true){ //code to be executed }</pre>	<pre>do{ //code to be executed }while(true);</pre>

for-each Loop

The for-each loop is used to traverse array or collection in java. It is easier to use than simple for loop because we don't need to increment value and use subscript notation.

It works on elements basis not index. It returns element one by one in the defined variable.

Syntax:

```
for(Type var:array)
{
    //code to be executed
}
```

Example:

```
//Java For-each loop example which prints the
//elements of the array
public class ForEachExample
{
    public static void main(String[] args)
    {
        //Declaring an array
        int arr[] = {12, 23, 44, 56, 78};
        //Printing array using for-each loop
        for(int i : arr)
        {
            System.out.println(i);
        }
    }
}
```

Labeled for Loop

We can have a name of each Java for loop. To do so, we use label before the for loop. It is useful if we have nested for loop so that we can break/continue specific for loop.

Usually, break and continue keywords breaks/continues the innermost for loop only.

Syntax:

```
labelname:
for(initialization; condition; incr/decr)
{
    //code to be executed
}
```

Example:

```
//A Java program to demonstrate the use of labeled for loop
public class LabeledForExample
{
    public static void main(String[] args)
    {
        //Using Label for outer and for loop
        aa:
        for(int i=1; i<=3; i++)
        {
            bb:
            for(int j=1; j<=3; j++)
            {
                if(i==2 && j==2)
                {
                    break aa;
                }
                System.out.println(i+ " "+j);
            }
        }
    }
}
```

Output:

```
1 1
1 2
1 3
2 1
```

Java Comments

The java comments are statements that are not executed by the compiler and interpreter. The comments can be used to provide information or explanation about the variable, method, class or any statement. It can also be used to hide program code for specific time.

Types

There are 3 types of comments in java.

1. Single Line Comment (//)
2. Multi Line Comment (/* .. */)
3. Documentation Comment (/** .. */)

Documentation Comment

The documentation comment is used to create documentation API. To create documentation API, you need to use **javadoc tool**.

Syntax:

```
/**
This
is
documentation
comment
*/
```

Example:

```
/** The Calculator class provides methods to get addition and subtraction of given 2 numbers.*/
```

```
public class Calculator
{
    /** The add() method returns addition of given numbers.*/
    public static int add(int a, int b)
    {
        return a + b;
    }
    /** The sub() method returns subtraction of given numbers.*/
    public static int sub(int a, int b)
    {
        return a - b;
    }
}
```

Compile it by javac tool:

```
javac Calculator.java
```

Create Documentation API by javadoc tool:

```
javadoc Calculator.java
```

Now, there will be HTML files created for your Calculator class in the current directory. Open the HTML files and see the explanation of Calculator class provided through documentation comment.

OOPs Concepts

Object-Oriented Programming is a paradigm that provides many concepts such as **inheritance**, **data binding**, **polymorphism**, etc.

Simula is considered the first object-oriented programming language. The programming paradigm where everything is represented as an object is known as a truly object-oriented programming language.

Smalltalk is considered the first truly object-oriented programming language.

The popular object-oriented languages are [Java](#), [C#](#), [PHP](#), [Python](#), [C++](#), etc.

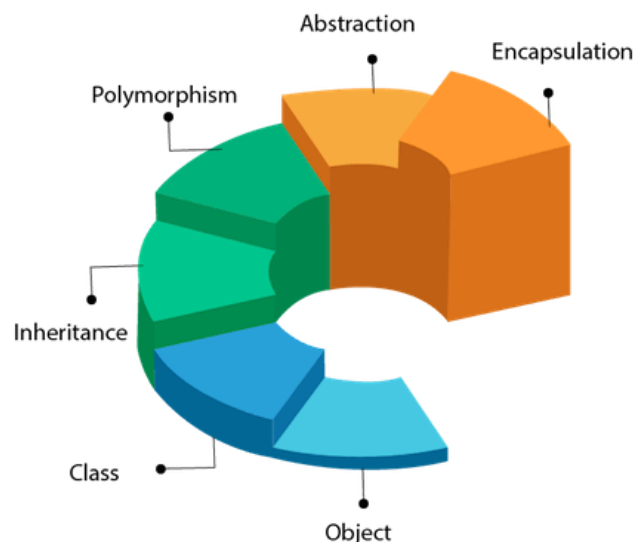
The main aim of object-oriented programming is to implement real-world entities for example object, classes, abstraction, inheritance, polymorphism, etc.

OOPs (Object-Oriented Programming System)

Object means a real-world entity such as a pen, chair, table, computer, watch, etc. **Object-Oriented Programming** is a methodology or paradigm to design a program using classes and objects. It simplifies the software development and maintenance by providing some concepts:

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

OOPs (Object-Oriented Programming System)



Object

Any entity that has state and behavior is known as an object. For example a chair, pen, table, keyboard, bike, etc. It can be physical or logical.

An Object can be defined as an instance of a class. An object contains an address and takes up some space in memory. Objects can communicate without knowing the details of each other's data or code. The only necessary thing is the type of message accepted and the type of response returned by the objects.



Example: A dog is an object because it has states like color, name, breed, etc. as well as behaviors like wagging the tail, barking, eating, etc.

Class

Collection of objects is called class. It is a logical entity.

A class can also be defined as a blueprint from which you can create an individual object. Class doesn't consume any space.

Inheritance

When one object acquires all the properties and behaviors of a parent object, it is known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism.

Polymorphism

If *one task is performed by different ways*, it is known as polymorphism. For example: to convince the customer differently, to draw something, for example, shape, triangle, rectangle, etc.

In Java, we use method overloading and method overriding to achieve polymorphism.

Another example can be to speak something; for example, a cat speaks meow, dog barks woof, etc.



Abstraction

Hiding internal details and showing functionality is known as abstraction. For example phone call, we don't know the internal processing.

In Java, we use abstract class and interface to achieve abstraction.

Encapsulation

Binding (or wrapping) code and data together into a single unit are known as encapsulation. For example capsule, it is wrapped with different medicines.

A java class is the example of encapsulation. Java bean is the fully encapsulated class because all the data members are private here.

Advantage of OOPs over Procedure-oriented programming language

- 1) OOPs makes development and maintenance easier whereas in a procedure-oriented programming language it is not easy to manage if code grows as project size increases.
- 2) OOPs provides data hiding whereas in a procedure-oriented programming language a global data can be accessed from anywhere.

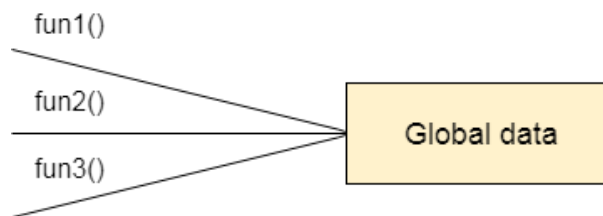


Figure: Data Representation in Procedure-Oriented Programming

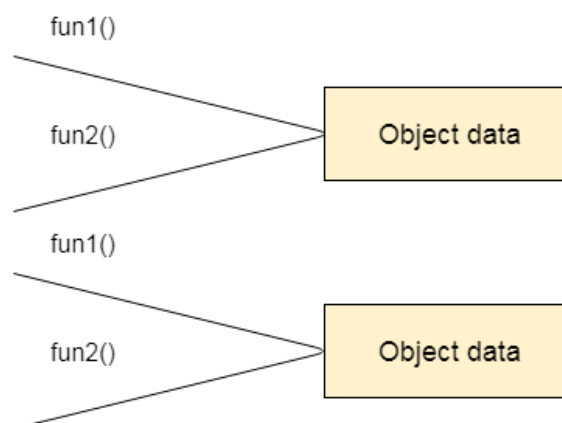


Figure: Data Representation in Object-Oriented Programming

3) OOPs provides the ability to simulate real-world event much more effectively. We can provide the solution of real word problem if we are using the Object-Oriented Programming language.

object-oriented programming language vs object-based programming language

Object-based programming language follows all the features of OOPs except Inheritance. JavaScript and VBScript are examples of object-based programming languages.

Java Naming conventions

Java **naming convention** is a rule to follow as you decide what to name your identifiers such as class, package, variable, constant, method etc.

But, it is not forced to follow. So, it is known as convention not rule.

All the classes, interfaces, packages, methods and fields of java programming language are given according to java naming convention.

Advantage of naming conventions in java

By using standard Java naming conventions, you make your code easier to read for yourself and for other programmers. Readability of Java program is very important. It indicates that **less time** is spent to figure out what the code does.

Name	Convention
class name	should start with uppercase letter and be a noun e.g. String, Color, Button, System, Thread etc.
interface name	should start with uppercase letter and be an adjective e.g. Runnable, Remote, ActionListener etc.
method name	should start with lowercase letter and be a verb e.g. actionPerformed(), main(), print(), println() etc.
variable name	should start with lowercase letter e.g. firstName, orderNumber etc.
package name	should be in lowercase letter e.g. java, lang, sql, util etc.
constants name	should be in uppercase letter. e.g. RED, YELLOW, MAX_PRIORITY etc.

CamelCase in java naming conventions

Java follows camelcase syntax for naming the class, interface, method and variable.

If name is combined with two words, second word will start with uppercase letter always e.g. actionPerformed(), firstName, ActionEvent, ActionListener etc.

Objects and Classes

An object in Java is the physical as well as logical entity whereas a class in Java is a logical entity only.

What is an object

An entity that has state and behavior is known as an object e.g. chair, bike, marker, pen, table, car etc. It can be physical or logical (tangible and intangible). The example of an intangible object is the banking system.

An object has three characteristics:

- **State:** represents the data (value) of an object.
- **Behavior:** represents the behavior (functionality) of an object such as deposit, withdraw, etc.
- **Identity:** An object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. However, it is used internally by the JVM to identify each object uniquely.

For Example, Pen is an object. Its name is Reynolds; color is white, known as its state. It is used to write, so writing is its behavior.

An object is an instance of a class. A class is a template or blueprint from which objects are created. So, an object is the instance(result) of a class.

Object Definitions:

- An object is *a real-world entity*.
- An object is *a runtime entity*.
- The object is *an entity which has state and behavior*.
- The object is *an instance of a class*.

What is a class

A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.

A class in Java can contain:

- **Fields**
- **Methods**
- **Constructors**
- **Blocks**
- **Nested class and interface**

Syntax to declare a class:

```
class <class_name>
{
    field;
    method;
}
```

Instance variable

A variable which is created inside the class but outside the method is known as an instance variable. Instance variable doesn't get memory at compile time. It gets memory at runtime when an object or instance is created. That is why it is known as an instance variable.

Method

In Java, a method is like a function which is used to expose the behavior of an object.

Advantage of Method

- Code Reusability
- Code Optimization

new keyword

The new keyword is used to allocate memory at runtime. All objects get memory in Heap memory area.

Object and Class Example: main within the class

In this example, we have created a Student class which has two data members id and name. We are creating the object of the Student class by new keyword and printing the object's value.

Here, we are creating a main() method inside the class.

```
//Defining a Student class.
class Student
{
    //defining fields
    int id;//field or data member or instance variable
    String name;
    //creating main method inside the Student class
    public static void main(String args[])
    {
        //Creating an object or instance
        Student s1 = new Student();//creating an object of Student
        //Printing values of the object
        System.out.println(s1.id);//accessing member through reference variable
        System.out.println(s1.name);
    }
}
```

Output:

```
0
null
```

Object and Class Example: main outside the class

In real time development, we create classes and use it from another class. It is a better approach than previous one. Let's see a simple example, where we are having main() method in another class.

We can have multiple classes in different java files or single java file. If you define multiple classes in a single java source file, it is a good idea to save the file name with the class name which has main() method.

```
//another class
//Creating Student class.
class Student
{
    int id;
    String name;
}
//Creating another class TestStudent1 which contains the main method
class TestStudent1
{
    public static void main(String args[])
    {
        Student s1 = new Student();
        System.out.println(s1.id);
        System.out.println(s1.name);
    }
}
```

Output:

```
0
null
```

3 Ways to initialize object

There are 3 ways to initialize object in java.

1. By reference variable
2. By method
3. By constructor

Initialization through reference

Initializing an object means storing data into the object. Let's see a simple example where we are going to initialize the object through a reference variable.

```
class Student
```

```
{
    int id;
    String name;
}
class TestStudent2
{
    public static void main(String args[])
    {
        Student s1 = new Student();
        s1.id = 101;
        s1.name = "Sonoo";
        System.out.println(s1.id+" "+s1.name);
    }
}
```

Output:

```
101 Sonoo
```

Initialization through method

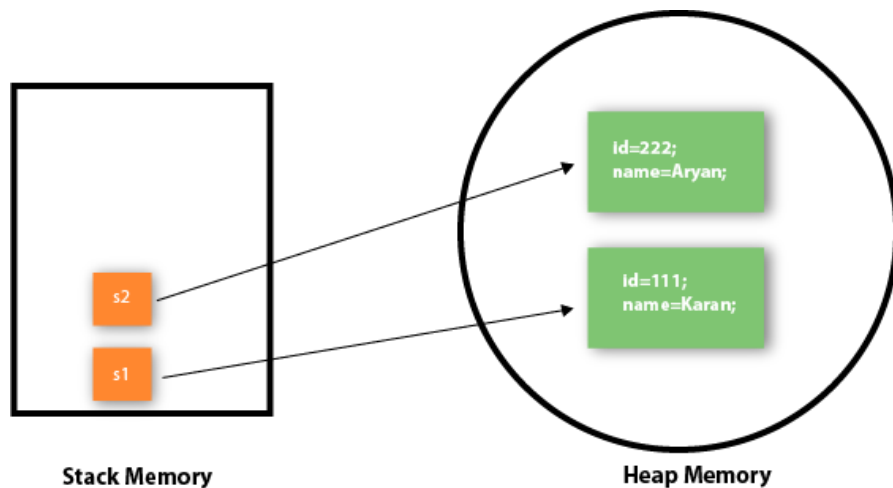
In this example, we are creating the two objects of Student class and initializing the value to these objects by invoking the insertRecord method. Here, we are displaying the state (data) of the objects by invoking the displayInformation() method.

File: TestStudent4.java

```
class Student
{
    int rollno;
    String name;
    void insertRecord(int r, String n)
    {
        rollno = r;
        name = n;
    }
    void displayInformation()
    {
        System.out.println(rollno+" "+name);
    }
}
class TestStudent4
{
    public static void main(String args[])
    {
        Student s1 = new Student();
        Student s2 = new Student();
        s1.insertRecord(111, "Karan");
        s2.insertRecord(222, "Aryan");
        s1.displayInformation();
        s2.displayInformation();
    }
}
```

Output:

```
111 Karan
222 Aryan
```



As you can see in the above figure, object gets the memory in heap memory area. The reference variable refers to the object allocated in the heap memory area. Here, s1 and s2 both are reference variables that refer to the objects allocated in memory.

Anonymous object

Anonymous simply means nameless. An object which has no reference is known as an anonymous object. It can be used at the time of object creation only.

If you have to use an object only once, an anonymous object is a good approach. For example:

```
new Calculation();//anonymous object
```

Calling method through a reference:

```
Calculation c = new Calculation();  
c.fact(5);
```

Calling method through an anonymous object

```
new Calculation().fact(5);
```

Let's see the full example of an anonymous object in Java.

```
class Calculation  
{  
    void fact(int n)  
    {  
        int fact = 1;  
        for(int i=1; i<=n; i++)  
        {  
            fact = fact*i;  
        }  
        System.out.println("factorial is "+fact);  
    }  
}
```

```
    }  
    public static void main(String args[])  
    {  
        new Calculation().fact(5); //calling method with anonymous object  
    }  
}
```

Output:

```
Factorial is 120
```

Creating multiple objects by one type only

We can create multiple objects by one type only as we do in case of primitives.

Initialization of primitive variables:

```
int a=10, b=20;
```

Initialization of reference variables:

```
Rectangle r1=new Rectangle(), r2=new Rectangle(); //creating two objects
```

Constructors

A constructor is a block of codes similar to the method. It is called when an instance of the object is created, and memory is allocated for the object.

It is a special type of method which is used to initialize the object.

When is a constructor called

Every time an object is created using new() keyword, at least one constructor is called. It calls a default constructor.

Note: It is called constructor because it constructs the values at the time of object creation. It is not necessary to write a constructor for a class. It is because java compiler creates a default constructor if your class doesn't have any.

Rules for creating Java constructor

There are two rules defined for the constructor.

1. Constructor name must be the same as its class name
2. A Constructor must have no explicit return type
3. A Java constructor cannot be abstract, static, final, and synchronized

Note: We can use access modifiers while declaring a constructor. It controls the object creation. In other words, we can have private, protected, public or default constructor in Java.

Types of Java constructors

There are two types of constructors in Java:

1. Default constructor (no-arg constructor)
2. Parameterized constructor

Java Default Constructor

A constructor is called "Default Constructor" when it doesn't have any parameter.

Syntax of default constructor:

```
<class_name>()  
{  
    .....  
}
```

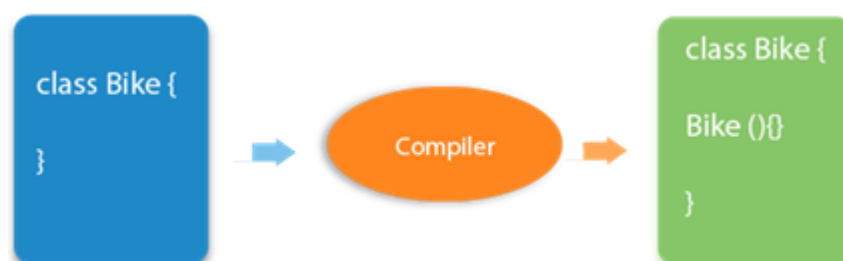
Example of default constructor

In this example, we are creating the no-arg constructor in the Bike class. It will be invoked at the time of object creation.

```
//Java Program to create and call a default constructor  
class Bike1  
{  
    //creating a default constructor  
    Bike1()  
    {  
        System.out.println("Bike is created");  
    }  
    //main method  
    public static void main(String args[])  
    {  
        //calling a default constructor  
        Bike1 b=new Bike1();  
    }  
}
```

Output:

Bike is created



What is the purpose of a default constructor?

The default constructor is used to provide the default values to the object like 0, null, etc., depending on the type.

Example of default constructor that displays the default values

```
//Let us see another example of default constructor
//which displays the default values
class Student3
{
    int id;
    String name;
    //method to display the value of id and name
    void display()
    {
        System.out.println(id+ " "+name);
    }
    public static void main(String args[])
    {
        //creating objects
        Student3 s1 = new Student3();
        Student3 s2 = new Student3();
        //displaying values of the object
        s1.display();
        s2.display();
    }
}
```

Output:

```
0 null
0 null
```

Explanation: In the above class, you are not creating any constructor so compiler provides you a default constructor. Here 0 and null values are provided by default constructor.

Parameterized Constructor

A constructor which has a specific number of parameters is called a parameterized constructor.

Why use the parameterized constructor?

The parameterized constructor is used to provide different values to the distinct objects. However, you can provide the same values also.

Example

In this example, we have created the constructor of Student class that have two parameters. We can have any number of parameters in the constructor.

```
class Student4
{
    int id;
```

```
String name;
//creating a parameterized constructor
Student4(int i,String n)
{
    id = i;
    name = n;
}
//method to display the values
void display()
{
    System.out.println(id+" "+name);
}
public static void main(String args[])
{
    //creating objects and passing values
    Student4 s1 = new Student4(111,"Karan");
    Student4 s2 = new Student4(222,"Aryan");
    //calling method to display the values of object
    s1.display();
    s2.display();
}
}
```

Output:

```
111 Karan
222 Aryan
```

Constructor Overloading

A constructor is just like a method but without return type. It can also be overloaded like Java methods.

Constructor overloading in Java is a technique of having more than one constructor with different parameter lists. They are arranged in a way that each constructor performs a different task. They are differentiated by the compiler by the number of parameters in the list and their types.

```
class Student5
{
    int id;
    String name;
    int age;
    //creating two arg constructor
    Student5(int i,String n)
    {
        id = i;
        name = n;
    }
    //creating three arg constructor
    Student5(int i,String n,int a)
    {
        id = i;
        name = n;
        age = a;
    }
    void display()
```

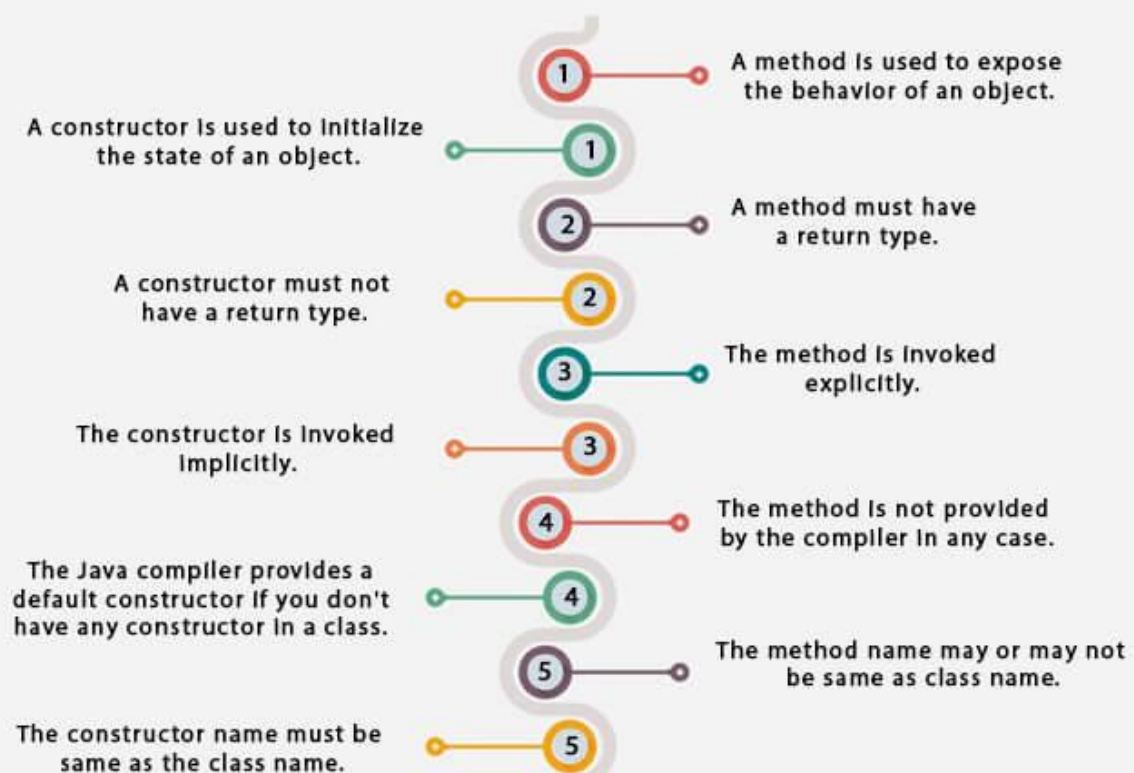


```
{  
    System.out.println(id+ " "+name+" "+age);  
}  
  
public static void main(String args[])  
{  
    Student5 s1 = new Student5(111,"Karan");  
    Student5 s2 = new Student5(222,"Aryan",25);  
    s1.display();  
    s2.display();  
}}
```

Output:

```
111 Karan 0  
222 Aryan 25
```

Difference between constructor and method in Java



Copy Constructor

There is no copy constructor in java. However, we can copy the values from one object to another like copy constructor in C++.

There are many ways to copy the values of one object into another in java. They are:

- By constructor
- By assigning the values of one object into another
- By clone() method of Object class

In this example, we are going to copy the values of one object into another using java constructor.

```
class Student
{
    int id;
    String name;
    //constructor to initialize integer and string
    Student (int i,String n)
    {
        id = i;
        name = n;
    }
    //constructor to initialize another object
    Student (Student s)
    {
        id = s.id;
        name = s.name;
    }
    void display()
    {
        System.out.println(id+ " "+name);
    }
    public static void main(String args[])
    {
        Student s1 = new Student (111, "Karan");
        Student s2 = new Student (s1);
        s1.display();
        s2.display();
    }
}
```

Output:

```
111 Karan
111 Karan
```

Does constructor return any value?

Yes, it is the current class instance (You cannot use return type yet it returns a value).

Can constructor perform other tasks instead of initialization?

Yes, like object creation, starting a thread, calling a method, etc. You can perform any operation in the constructor as you perform in the method.

static keyword

The **static keyword** in Java is used for memory management mainly. We can apply java static keyword with variables, methods, blocks and nested class. The static keyword belongs to the class than an instance of the class.

The static can be:

1. Variable (also known as a class variable)
2. Method (also known as a class method)
3. Block
4. Nested class

static variable

If you declare any variable as static, it is known as a static variable.

- The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc.
- The static variable gets memory only once in the class area at the time of class loading.

Advantages of static variable

It makes your program **memory efficient** (i.e., it saves memory).

Understanding the problem without static variable

```
class Student
{
    int rollno;
    String name;
    String college = "ITS";
}
```

Suppose there are 500 students in my college, now all instance data members will get memory each time when the object is created. All students have its unique rollno and name, so instance data member is good in such case. Here, "college" refers to the common property of all objects. If we make it static, this field will get the memory only once.

Note: Java static property is shared to all objects.

Example

```
class Student
{
    int rollno;//instance variable
    String name;
    static String college ="ITS";//static variable
    //constructor
    Student(int r, String n)
    {
```

```

        rollno = r;
        name = n;
    }
    //method to display the values
    void display ()
    {
        System.out.println(rollno+" "+name+" "+college);
    }
}
//Test class to show the values of objects
public class TestStaticVariable1
{
    public static void main(String args[])
    {
        Student s1 = new Student(111,"Karan");
        Student s2 = new Student(222,"Aryan");
        //we can change the college of all objects by the single line of code
        //Student.college="BBDIT";
        s1.display();
        s2.display();
    }
}

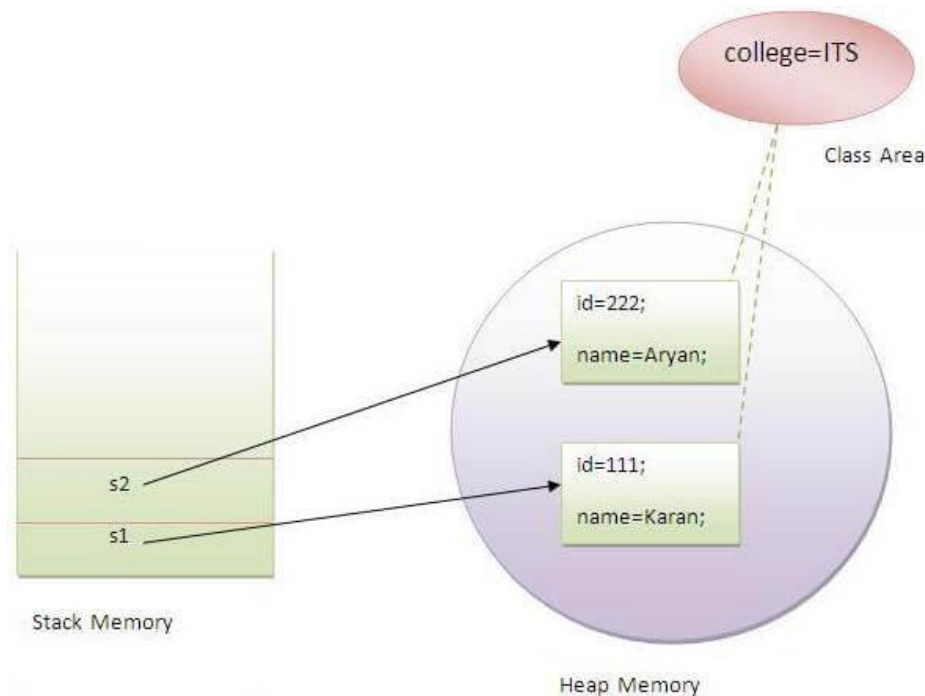
```

Output:

```

111 Karan ITS
222 Aryan ITS

```

**static method**

If you apply static keyword with any method, it is known as static method.

- A static method belongs to the class rather than the object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- A static method can access static data member and can change the value of it.

```
class Student
{
    int rollno;
    String name;
    static String college = "ITS";
    //static method to change the value of static variable
    static void change()
    {
        college = "BBDIT";
    }
    //constructor to initialize the variable
    Student(int r, String n)
    {
        rollno = r;
        name = n;
    }
    //method to display values
    void display()
    {
        System.out.println(rollno+" "+name+" "+college);
    }
}
//Test class to create and display the values of object
public class TestStaticMethod
{
    public static void main(String args[])
    {
        Student.change();//calling change method
        //creating objects
        Student s1 = new Student(111,"Karan");
        Student s2 = new Student(222,"Aryan");
        Student s3 = new Student(333,"Sonoo");
        //calling display method
        s1.display();
        s2.display();
        s3.display();
    }
}
```

Output:

```
111 Karan BBDIT
222 Aryan BBDIT
333 Sonoo BBDIT
```

static block

- Is used to initialize the static data member.
- It is executed before the main method at the time of classloading.

```
class A2
{
    static
    {
        System.out.println("static block is invoked");
    }
    public static void main(String args[])
    {
        System.out.println("Hello main");
    }
}
```

```
}
```

Output:

```
static block is invoked  
Hello main
```

Can we execute a program without main() method?

No, one of the ways was the static block, but it was possible till JDK 1.6. Since JDK 1.7, it is not possible to execute a java class without the main method.

```
class A3  
{  
    static  
    {  
        System.out.println("static block is invoked");  
        System.exit(0);  
    }  
}
```

Output:

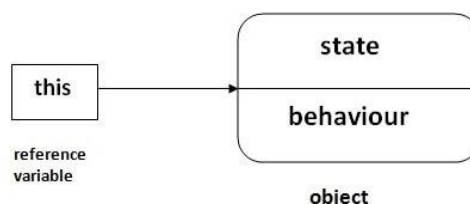
```
static block is invoked
```

Since JDK 1.7 and above, output would be:

```
Error: Main method not found in class A3, please define the main method as:  
    public static void main(String[] args)  
or a JavaFX application class must extend javafx.application.Application
```

this keyword

There can be a lot of usage of **java this keyword**. In java, this is a **reference variable** that refers to the current object.



Usage of java this

keyword

Here is given the 6 usage of java this keyword.

1. this can be used to refer current class instance variable.
2. this can be used to invoke current class method (implicitly)
3. this() can be used to invoke current class constructor.
4. this can be passed as an argument in the method call.
5. this can be passed as argument in the constructor call.
6. this can be used to return the current class instance from the method.

The this keyword can be used to refer current class instance variable. If there is ambiguity between the instance variables and parameters, this keyword resolves the problem of ambiguity.

Let's understand the problem if we don't use this keyword by the example given below:

```
class Student
{
    int rollNo;
    String name;
    float fee;
    Student(int rollNo,String name,float fee)
    {
        rollNo = rollNo;
        name = name;
        fee = fee;
    }
    void display()
    {
        System.out.println(rollNo+" "+name+" "+fee);
    }
}
class TestThis1
{
    public static void main(String args[])
    {
        Student s1 = new Student(111, "ankit", 5000f);
        Student s2 = new Student(112, "sumit", 6000f);
        s1.display();
        s2.display();
    }
}
```

Output:

```
0 null 0.0
0 null 0.0
```

In the above example, parameters (formal arguments) and instance variables are same. So, we are using this keyword to distinguish local variable and instance variable.

Solution of the above problem by this keyword

```
class Student
{
    int rollNo;
    String name;
    float fee;
    Student(int rollNo, String name, float fee)
    {
        this.rollNo = rollNo;
        this.name = name;
        this.fee = fee;
    }
    void display()
    {
        System.out.println(rollNo+" "+name+" "+fee);
    }
}
class TestThis2
{
}
```

```
public static void main(String args[])
{
    Student s1 = new Student(111, "ankit", 5000f);
    Student s2 = new Student(112, "sumit", 6000f);
    s1.display();
    s2.display();
}
```

Output:

```
111 ankit 5000
112 sumit 6000
```

this: to invoke current class method

You may invoke the method of the current class by using the this keyword. If you don't use the this keyword, compiler automatically adds this keyword while invoking the method. Let's see the example

```
class A
{
    void m()
    {
        System.out.println("hello m");
    }
    void n()
    {
        System.out.println("hello n");
        //m();//same as this.m()
        this.m();
    }
}
class TestThis4
{
    public static void main(String args[])
    {
        A a = new A();
        a.n();
    }
}
```

Output:

```
hello n
hello m
```

this() : to invoke current class constructor

The this() constructor call can be used to invoke the current class constructor. It is used to reuse the constructor. In other words, it is used for constructor chaining.

Calling default constructor from parameterized constructor:

```
class A
```



```
{
    A()
    {
        System.out.println("hello a");
    }
    A(int x)
    {
        this();
        System.out.println(x);
    }
}
class TestThis5
{
    public static void main(String args[])
    {
        A a = new A(10);
    }
}
```

Output:

```
hello a
10
```

Java Garbage Collection

In java, garbage means unreferenced objects.

Garbage Collection is process of reclaiming the runtime unused memory automatically. In other words, it is a way to destroy the unused objects.

To do so, we were using free() function in C language and delete() in C++. But, in java it is performed automatically. So, java provides better memory management.

Advantage of Garbage Collection

- It makes java **memory efficient** because garbage collector removes the unreferenced objects from heap memory.
- It is **automatically done** by the garbage collector(a part of JVM) so we don't need to make extra efforts.

How can an object be unreferenced?

There are many ways:

- By nulling the reference
- By assigning a reference to another
- By anonymous object etc.

By nulling a reference:

```
Employee e = new Employee();  
e = null;
```

By assigning a reference to another:

```
Employee e1 = new Employee();  
Employee e2 = new Employee();  
e1 = e2; //now the first object referred by e1 is available for garbage collection
```

By anonymous object:

```
new Employee();
```

finalize() method

The finalize() method is invoked each time before the object is garbage collected. This method can be used to perform cleanup processing. This method is defined in Object class as:

```
protected void finalize(){}
```

gc() method

The gc() method is used to invoke the garbage collector to perform cleanup processing. The gc() is found in System and Runtime classes.

```
public static void gc(){}  
  
public class TestGarbage1  
{  
    public void finalize()  
    {  
        System.out.println("object is garbage collected");  
    }  
    public static void main(String args[])  
    {  
        TestGarbage1 s1 = new TestGarbage1();  
        TestGarbage1 s2 = new TestGarbage1();  
        s1 = null;  
        s2 = null;  
        System.gc();  
    }  
}
```

Output:

```
object is garbage collected  
object is garbage collected
```

Inheritance

Inheritance is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important part of OOPs (Object Oriented programming system).

The idea behind inheritance in Java is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also.

Inheritance represents the **IS-A relationship** which is also known as a *parent-child* relationship.

Why use inheritance

- For Method Overriding (so runtime polymorphism can be achieved).
- For Code Reusability.

Terms used in Inheritance

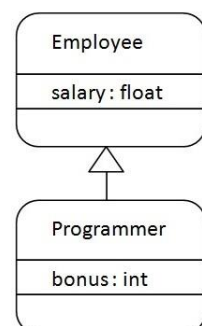
- **Class:** A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.
- **Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- **Super Class/Parent Class:** Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.
- **Reusability:** As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in the previous class.

Syntax:

```
class Subclass-name extends Superclass-name
{
    //methods and fields
}
```

The **extends keyword** indicates that you are making a new class that derives from an existing class. The meaning of "extends" is to increase the functionality.

In the terminology of Java, a class which is inherited is called a parent or superclass, and the new class is called child or subclass.



As displayed in the above figure, Programmer is the subclass and Employee is the superclass. The relationship between the two classes is **Programmer IS-A Employee**. It means that Programmer is a type of Employee.

```
class Employee
{
    float salary = 40000;
```

```
}  
class Programmer extends Employee  
{  
    int bonus = 10000;  
    public static void main(String args[])  
    {  
        Programmer p = new Programmer();  
        System.out.println("Programmer salary is:"+p.salary);  
        System.out.println("Bonus of Programmer is:"+p.bonus);  
    }  
}
```

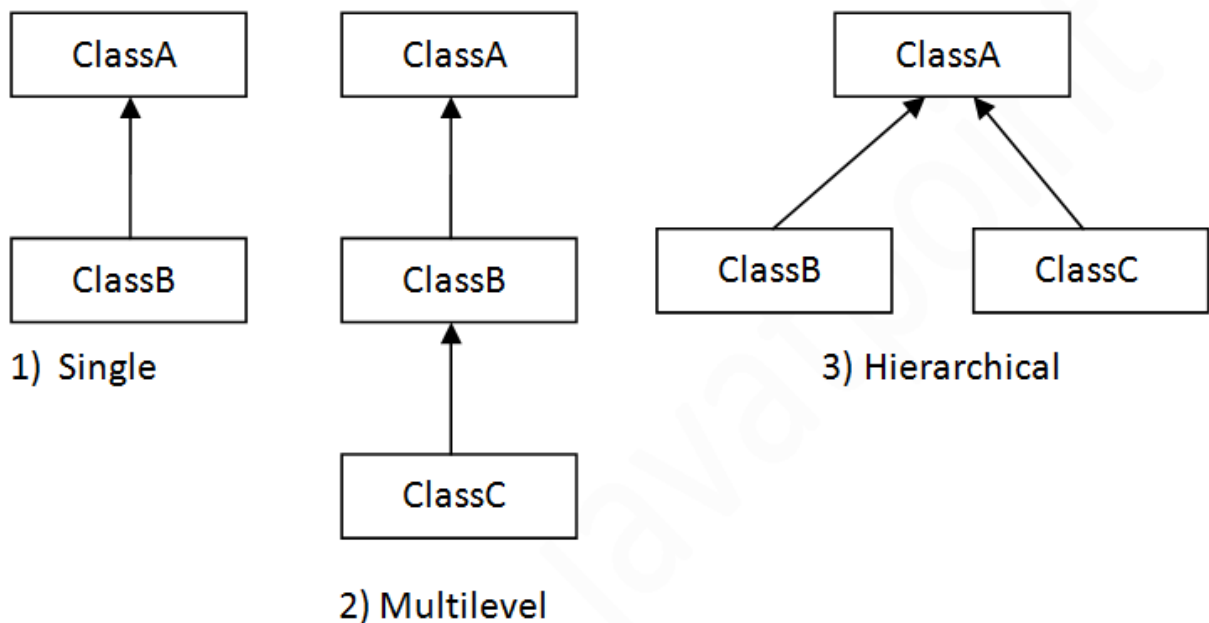
Output:

```
Programmer salary is:40000.0  
Bonus of programmer is:10000
```

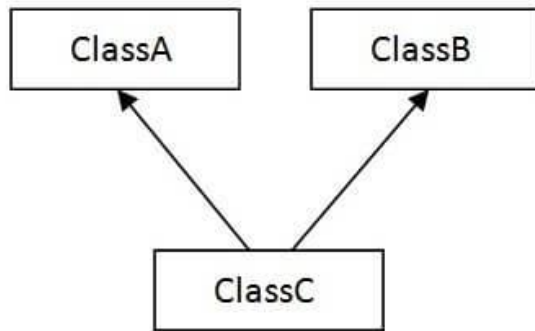
Types of inheritance in java

On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.

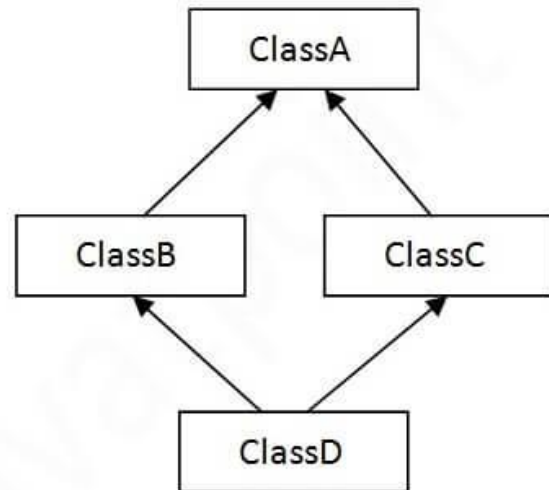
In java programming, multiple and hybrid inheritance is supported through interface only.



When one class inherits multiple classes, it is known as multiple inheritance. For Example:



4) Multiple



5) Hybrid

Single Inheritance Example

```
class Animal
{
    void eat()
    {
        System.out.println("eating...");
    }
}
class Dog extends Animal
{
    void bark()
    {
        System.out.println("barking...");
    }
}
class TestInheritance
{
    public static void main(String args[])
    {
        Dog d = new Dog();
        d.bark();
        d.eat();
    }
}
```

Output:

```
barking...
eating...
```

Multilevel Inheritance Example

```
class Animal
{
    void eat()
    {
        System.out.println("eating...");
    }
}
class Dog extends Animal
{
    void bark()
    {
        System.out.println("barking...");
    }
}
class BabyDog extends Dog
{
    void weep()
    {
        System.out.println("weeping...");
    }
}
class TestInheritance2
{
    public static void main(String args[])
    {
        BabyDog d = new BabyDog();
        d.weep();
        d.bark();
        d.eat();
    }
}
```

Output:

```
weeping...
barking...
eating...
```

Hierarchical Inheritance Example

```
class Animal
{
    void eat()
    {
        System.out.println("eating...");
    }
}
class Dog extends Animal
{
    void bark()
    {
        System.out.println("barking...");
    }
}
class Cat extends Animal
{
    void meow()
    {
        System.out.println("meowing...");
    }
}
```

```
        void meow()
        {
            System.out.println("meowing...");
        }
    }
    class TestInheritance3
    {
        public static void main(String args[])
        {
            Cat c = new Cat();
            c.meow();
            c.eat();
            //c.bark();//C.T.Error
        }
    }
```

Output:

```
meowing...
eating...
```

Why multiple inheritance is not supported in java?

To reduce the complexity and simplify the language, multiple inheritance is not supported in java. Consider a scenario where A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and you call it from child class object, there will be ambiguity to call the method of A or B class.

Since compile-time errors are better than runtime errors, Java renders compile-time error if you inherit 2 classes. So whether you have same method or different, there will be compile time error.

```
class A
{
    void msg()
    {
        System.out.println("Hello");
    }
}
class B
{
    void msg()
    {
        System.out.println("Welcome");
    }
}
class C extends A,B
{
    //suppose if it were
    public static void main(String args[])
    {
        C obj = new C();
        obj.msg(); //Now which msg() method would be invoked?
    }
}
```

Output:

```
Compile Time Error
```

Aggregation

If a class have an entity reference, it is known as Aggregation. Aggregation represents HAS-A relationship.

Consider a situation, Employee object contains many informations such as id, name, emailId etc. It contains one more object named address, which contains its own informations such as city, state, country, zipcode etc. as given below.

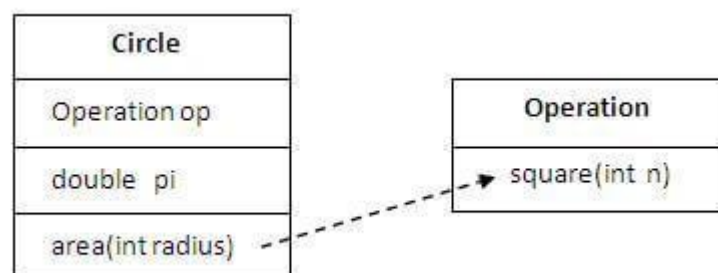
```
class Employee
{
    int id;
    String name;
    Address address; //Address is a class
    ...
}
```

In such case, Employee has an entity reference address, so relationship is Employee HAS-A address.

Why use Aggregation?

- For Code Reusability.

Simple Example of Aggregation



In this example, we have created the reference of Operation class in the Circle class.

```
class Operation
{
    int square(int n)
    {
        return n * n;
    }
}
class Circle
{
    Operation op; //aggregation
    double pi = 3.14;
    double area(int radius)
```



```
{
    op = new Operation();
    //code reusability (i.e. delegates the method call).
    int rsquare = op.square(radius);
    return pi * rsquare;
}
public static void main(String args[])
{
    Circle c = new Circle();
    double result = c.area(5);
    System.out.println(result);
}
}
```

Output: 78.5

Method Overloading

If a class has multiple methods having same name but different in parameters, it is known as **Method Overloading**.

If we have to perform only one operation, having same name of the methods increases the readability of the program.

Suppose you have to perform addition of the given numbers but there can be any number of arguments, if you write the method such as a(int,int) for two parameters, and b(int,int,int) for three parameters then it may be difficult for you as well as other programmers to understand the behavior of the method because its name differs.

So, we perform method overloading to figure out the program quickly.

Advantage of method overloading

Method overloading *increases the readability of the program*.

Different ways to overload the method

There are two ways to overload the method in java

1. By changing number of arguments
2. By changing the data type

```
class Adder
{
    static int add(int a,int b)
    {
        return a+b;
    }
    static int add(int a,int b,int c)
    {
        return a+b+c;
    }
}
```

```
        static double add(double a, double b)
        {
            return a+b;
        }
    }
    class TestOverloading
    {
        public static void main(String[] args)
        {
            System.out.println(Adder.add(11, 11));
            System.out.println(Adder.add(11, 11, 11));
            System.out.println(Adder.add(12.3, 12.6));
        }
    }
```

Method Overriding

If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in Java**.

In other words, If a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.

Usage of Java Method Overriding

- Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.
- Method overriding is used for runtime polymorphism

Rules for Java Method Overriding

1. The method must have the same name as in the parent class
2. The method must have the same parameter as in the parent class.
3. There must be an IS-A relationship (inheritance).

Example

In this example, we have defined the run method in the subclass as defined in the parent class but it has some specific implementation. The name and parameter of the method are the same, and there is IS-A relationship between the classes, so there is method overriding.

```
//Creating a parent class.
class Vehicle
{
    //defining a method
    void run()
    {
        System.out.println("Vehicle is running");
    }
}
//Creating a child class
```

```
class Bike2 extends Vehicle
{
    //defining the same method as in the parent class
    void run()
    {
        System.out.println("Bike is running safely");
    }

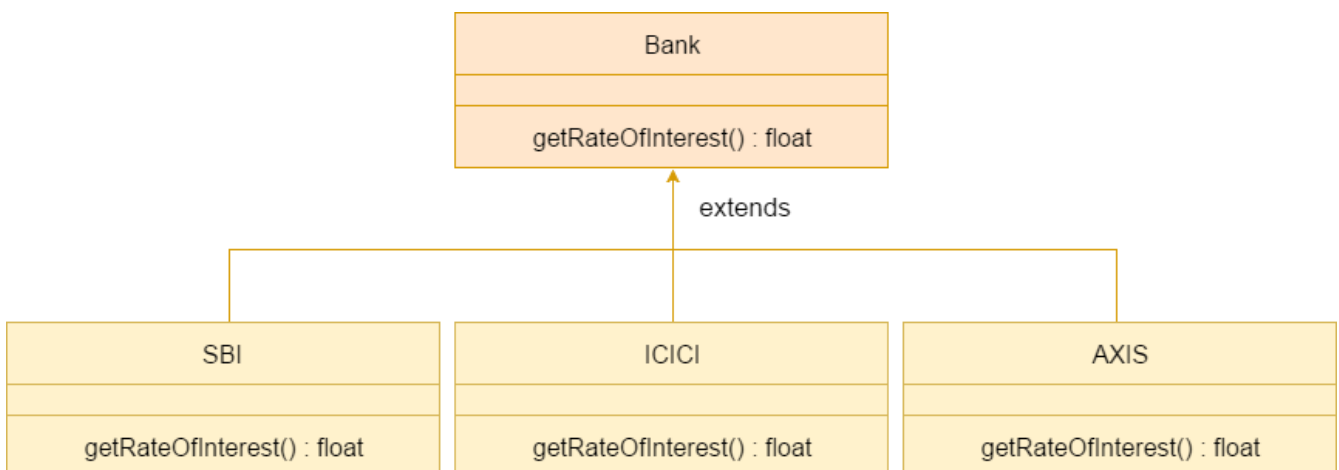
    public static void main(String args[]){
        Bike2 obj = new Bike2();//creating object
        obj.run();//calling method
    }
}
```

Output:

```
Bike is running safely
```

A real example of Java Method Overriding

Consider a scenario where Bank is a class that provides functionality to get the rate of interest. However, the rate of interest varies according to banks. For example, SBI, ICICI and AXIS banks could provide 8%, 7%, and 9% rate of interest.



```
//Creating a parent class.
class Bank
{
    int getRateOfInterest()
    {
        return 0;
    }
}
//Creating child classes.
class SBI extends Bank
{
    int getRateOfInterest()
    {
        return 8;
    }
}

class ICICI extends Bank
```

```
{
    int getRateOfInterest()
    {
        return 7;
    }
}
class AXIS extends Bank
{
    int getRateOfInterest()
    {
        return 9;
    }
}
//Test class to create objects and call the methods
class Test2
{
    public static void main(String args[])
    {
        SBI s = new SBI();
        ICICI i = new ICICI();
        AXIS a = new AXIS();
        System.out.println("SBI Rate of Interest: "+s.getRateOfInterest());
        System.out.println("ICICI Rate of Interest: "+i.getRateOfInterest());
        System.out.println("AXIS Rate of Interest: "+a.getRateOfInterest());
    }
}
```

Output:

```
SBI Rate of Interest: 8
ICICI Rate of Interest: 7
AXIS Rate of Interest: 9
```

A static method cannot be overridden, it is because the static method is bound with class whereas instance method is bound with an object. Static belongs to the class area, and an instance belongs to the heap area.

Covariant Return Type

The covariant return type specifies that the return type may vary in the same direction as the subclass.

Before Java5, it was not possible to override any method by changing the return type. But now, since Java5, it is possible to override method by changing the return type if subclass overrides any method whose return type is Non-Primitive but it changes its return type to subclass type.

```
class A
{
    A get()
    {
        return this;
    }
}

class B1 extends A
{
}
```

```
B1 get()
{
    return this;
}
void message()
{
    System.out.println("welcome to covariant return type");
}
public static void main(String args[])
{
    new B1().get().message();
}
}
```

Output: welcome to covariant return type

super Keyword

The **super** keyword in Java is a reference variable which is used to refer immediate parent class object.

Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable.

Usage of Java super Keyword

1. super can be used to refer immediate parent class instance variable.
2. super can be used to invoke immediate parent class method.
3. super() can be used to invoke immediate parent class constructor.

super example: real use

Let's see the real use of super keyword. Here, Emp class inherits Person class so all the properties of Person will be inherited to Emp by default. To initialize all the property, we are using parent class constructor from child class. In such way, we are reusing the parent class constructor.

```
class Person
{
    int id;
    String name;
    Person(int id, String name)
    {
        this.id = id;
        this.name = name;
    }
}
class Emp extends Person
{
    float salary;
    Emp(int id, String name, float salary)
    {

```

```
        super(id, name); //reusing parent constructor
        this.salary = salary;
    }
    void display(){System.out.println(id+" "+name+" "+salary);}
}
class TestSuper
{
    public static void main(String[] args)
    {
        Emp e1 = new Emp(1, "ankit", 45000f);
        e1.display();
    }
}
```

Output:

```
1 ankit 45000
```

final Keyword

The **final keyword** in java is used to restrict the user. The java final keyword can be used in many context. Final can be:

1. variable
2. method
3. class



final variable

If you make any variable as final, you cannot change the value of final variable(It will be constant).

There is a final variable speedlimit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

```
class Bike9
{
    final int speedlimit = 90; //final variable
    void run()
    {
        speedlimit = 400;
    }
    public static void main(String args[])
    {
        Bike9 obj = new Bike9();
        obj.run();
    }
}
```

```
    }  
} //end of class
```

Output: Compile Time Error

final method

If you make any method as final, you cannot override it.

```
class Bike  
{  
    final void run()  
    {  
        System.out.println("running");  
    }  
}  
class Honda extends Bike  
{  
    void run() //Compile time error  
    {  
        System.out.println("running safely with 100kmph");  
    }  
    public static void main(String args[])  
    {  
        Honda honda = new Honda();  
        honda.run();  
    }  
}
```

Polymorphism

Polymorphism in Java is a concept by which we can perform a *single action in different ways*. Polymorphism is derived from 2 Greek words: poly and morphs. The word "poly" means many and "morphs" means forms. So polymorphism means many forms.

There are two types of polymorphism in Java: compile-time polymorphism and runtime polymorphism. We can perform polymorphism in java by method overloading and method overriding.

If you overload a static method in Java, it is the example of compile time polymorphism. Here, we will focus on runtime polymorphism in java.

Runtime Polymorphism in Java

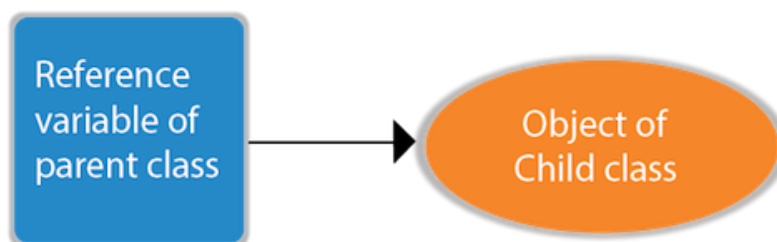
Runtime polymorphism or **Dynamic Method Dispatch** is a process in which a call to an overridden method is resolved at runtime rather than compile-time.

In this process, an overridden method is called through the reference variable of a superclass. The determination of the method to be called is based on the object being referred to by the reference variable.

Let's first understand the upcasting before Runtime Polymorphism.

Upcasting

If the reference variable of Parent class refers to the object of Child class, it is known as upcasting. For example:



```
class A{}  
class B extends A{}  
A a = new B();//upcasting
```

For upcasting, we can use the reference variable of class type or an interface type. For Example:

```
interface I{}  
class A{}  
class B extends A implements I{}
```

Here, the relationship of B class would be:

```
B IS-A A  
B IS-A I  
B IS-A Object
```

Since Object is the root class of all classes in Java, so we can write B IS-A Object.

Example

In this example, we are creating two classes Bike and Splendor. Splendor class extends Bike class and overrides its run() method. We are calling the run method by the reference variable of Parent class. Since it refers to the subclass object and subclass method overrides the Parent class method, the subclass method is invoked at runtime.

Since method invocation is determined by the JVM not compiler, it is known as runtime polymorphism.

```
class Bike
{
    void run()
    {
        System.out.println("running");
    }
}
class Splendor extends Bike
{
    void run()
    {
        System.out.println("running safely with 60km");
    }
    public static void main(String args[])
    {
        Bike b = new Splendor();//upcasting
        b.run();
    }
}
```

Output:

```
running safely with 60km.
```

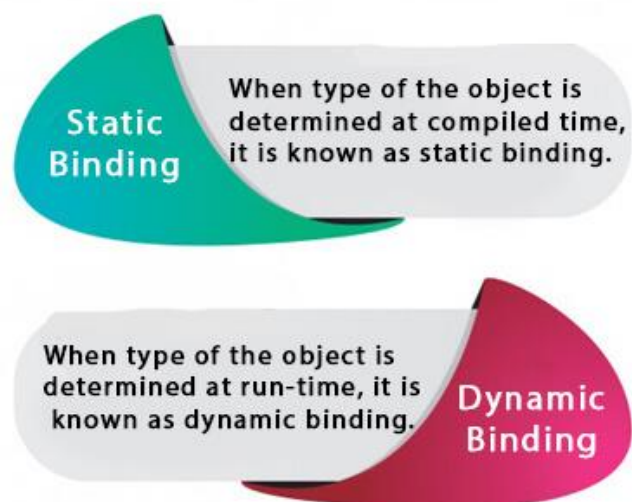
Static Binding and Dynamic Binding

Connecting a method call to the method body is known as binding.

There are two types of binding

1. Static Binding (also known as Early Binding).
2. Dynamic Binding (also known as Late Binding).

Static vs Dynamic Binding



Example of static binding

```
class Dog
{
    private void eat()
    {
        System.out.println("dog is eating...");
    }
    public static void main(String args[])
    {
        Dog d1 = new Dog();
        d1.eat();
    }
}
```

Example of dynamic binding

```
class Animal
{
    void eat()
    {
        System.out.println("animal is eating...");
    }
}
class Dog extends Animal
{
    void eat()
    {
        System.out.println("dog is eating...");
    }
    public static void main(String args[])
    {
        Animal a = new Dog();
        a.eat();
    }
}
```

Output: dog is eating...

In the above example object type cannot be determined by the compiler, because the instance of Dog is also an instance of Animal. So compiler doesn't know its type, only its base type.

instanceof operator

The **java instanceof operator** is used to test whether the object is an instance of the specified type (class or subclass or interface).

The instanceof in java is also known as type *comparison operator* because it compares the instance with type. It returns either true or false. If we apply the instanceof operator with any variable that has null value, it returns false.

Example of java instanceof

Let's see the simple example of instance operator where it tests the current class.

```
class Simple1
{
    public static void main(String args[])
    {
        Simple1 s = new Simple1();
        System.out.println(s instanceof Simple1);//true
    }
}
```

Output: true

Abstraction

Abstraction is a process of hiding the implementation details and showing only functionality to the user.

Another way, it shows only essential things to the user and hides the internal details, for example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.

Abstraction lets you focus on what the object does instead of how it does it.

Ways to achieve Abstraction

There are two ways to achieve abstraction in java

1. Abstract class (0 to 100%)
2. Interface (100%)

Abstract class

A class which is declared as abstract is known as an **abstract class**. It can have abstract and non-abstract methods. It needs to be extended and its method implemented. It cannot be instantiated.

Points to Remember

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructors and static methods also.

- It can have final methods which will force the subclass not to change the body of the method.

```
abstract class A
{
}
```

Abstract Method

A method which is declared as abstract and does not have implementation is known as an abstract method.

Example of abstract method

```
abstract void printStatus();//no method body and abstract
```

Example

In this example, Shape is the abstract class, and its implementation is provided by the Rectangle and Circle classes.

Mostly, we don't know about the implementation class (which is hidden to the end user), and an object of the implementation class is provided by the **factory method**.

A **factory method** is a method that returns the instance of the class. We will learn about the factory method later.

In this example, if you create the instance of Rectangle class, draw() method of Rectangle class will be invoked.

```
abstract class Shape
{
    abstract void draw();
}
//In real scenario, implementation is provided by others i.e. unknown by end user
class Rectangle extends Shape
{
    void draw()
    {
        System.out.println("drawing rectangle");
    }
}
class Circle1 extends Shape
{
    void draw()
    {
        System.out.println("drawing circle");
    }
}
//In real scenario, method is called by programmer or user
```

```
class TestAbstraction1
{
    public static void main(String args[])
    {
        Shape s = new Circle1();
        //In a real scenario, object is provided through method, e.g., getShape() method
        s.draw();
    }
}
```

Output: drawing circle

Interface

An **interface in java** is a blueprint of a class. It has static constants and abstract methods.

The interface in Java is *a mechanism to achieve abstraction*. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritance in Java.

In other words, you can say that interfaces can have abstract methods and variables. It cannot have a method body.

Java Interface also **represents the IS-A relationship**.

It cannot be instantiated just like the abstract class.

Since Java 8, we can have **default and static methods** in an interface.

Since Java 9, we can have **private methods** in an interface.

Why use Java interface?

There are mainly three reasons to use interface. They are given below.

- It is used to achieve abstraction.
- By interface, we can support the functionality of multiple inheritance.
- It can be used to achieve loose coupling.

Declaring an interface:

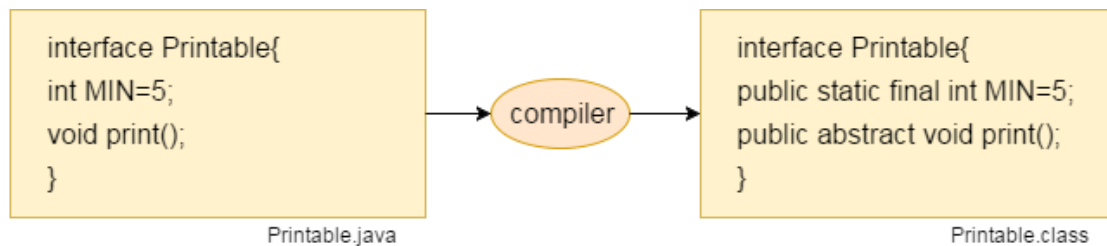
An interface is declared by using the interface keyword. It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default. A class that implements an interface must implement all the methods declared in the interface.

Syntax:

```
interface <interface_name>
{
    // declare constant fields
    // declare methods that abstract
    // by default.
}
```

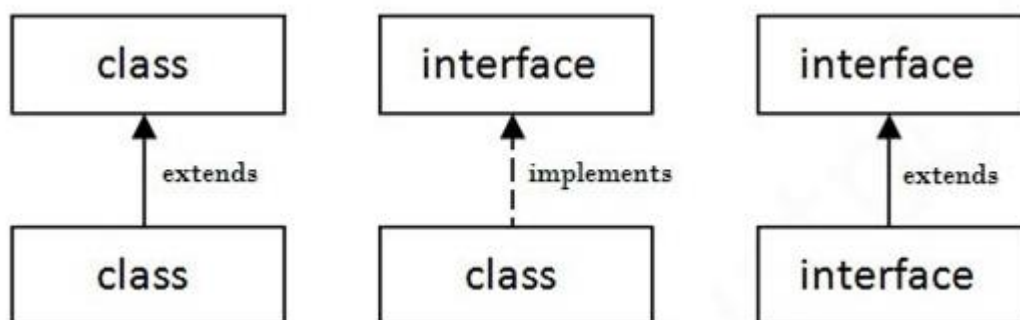
Since Java 8, interface can have default and static methods which is discussed later.

Interface fields are public, static and final by default, and the methods are public and abstract.



The relationship between classes and interfaces

As shown in the figure given below, a class extends another class, an interface extends another interface, but a **class implements an interface**.



Interface Example

In this example, the Drawable interface has only one method. Its implementation is provided by Rectangle and Circle classes. In a real scenario, an interface is defined by someone else, but its implementation is provided by different implementation providers. Moreover, it is used by someone else. The implementation part is hidden by the user who uses the interface.

```
//Interface declaration: by first user
interface Drawable
{
    void draw();
}
```

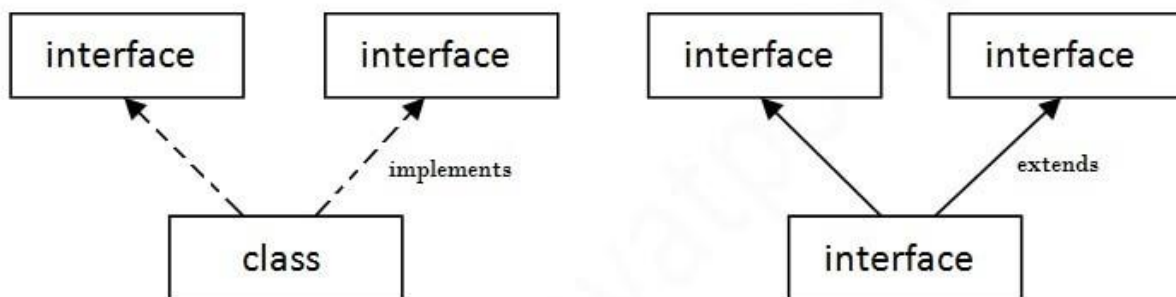
```
//Implementation: by second user
class Rectangle implements Drawable
{
    public void draw()
    {
        System.out.println("drawing rectangle");
    }
}
class Circle implements Drawable
{
    public void draw()
    {
        System.out.println("drawing circle");
    }
}
//Using interface: by third user
class TestInterface
{
    public static void main(String args[])
    {
        Drawable d = new Circle();
        //In real scenario, object is provided by method e.g. getDrawable()
        d.draw();
    }
}
```

Output:

```
drawing circle
```

Multiple inheritance in Java by interface

If a class implements multiple interfaces, or an interface extends multiple interfaces, it is known as multiple inheritance.



Multiple Inheritance in Java

```
interface Printable
{
    void print();
}
interface Showable
{
```

```
        void show();
    }
    class A7 implements Printable, Showable
    {
        public void print()
        {
            System.out.println("Hello");
        }
        public void show()
        {
            System.out.println("Welcome");
        }
        public static void main(String args[])
        {
            A7 obj = new A7();
            obj.print();
            obj.show();
        }
    }
```

Output:

```
    Hello
    Welcome
```

Difference between abstract class and interface

Abstract class and interface both are used to achieve abstraction where we can declare the abstract methods. Abstract class and interface both can't be instantiated.

But there are many differences between abstract class and interface that are given below.

Abstract class	Interface
1) Abstract class can have abstract and non-abstract methods.	Interface can have only abstract methods. Since Java 8, it can have default and static methods also.
2) Abstract class doesn't support multiple inheritance .	Interface supports multiple inheritance .
3) Abstract class can have final, non-final, static and non-static variables .	Interface has only static and final variables .
4) Abstract class can provide the implementation of interface .	Interface can't provide the implementation of abstract class .
5) The abstract keyword is used to declare abstract class.	The interface keyword is used to declare interface.
6) An abstract class can extend another Java class and implement multiple Java interfaces.	An interface can extend another Java interface only.

7) An abstract class can be extended using keyword "extends".	An interface class can be implemented using keyword "implements".
8) A Java abstract class can have class members like private, protected, etc.	Members of a Java interface are public by default.
9) Example: <pre>public abstract class Shape{ public abstract void draw(); }</pre>	Example: <pre>public interface Drawable{ void draw(); }</pre>

Simply, abstract class achieves partial abstraction (0 to 100%) whereas interface achieves fully abstraction (100%).

Let's see a simple example where we are using interface and abstract class both.

//Creating interface that has 4 methods

```
interface A
{
    void a();//bydefault, public and abstract
    void b();
    void c();
    void d();
}
```

//Creating abstract class that provides the implementation of one method of A interface

```
abstract class B implements A
{
    public void a()
    {
        System.out.println("I am a");
    }
    public void c()
    {
        System.out.println("I am C");
    }
}
```

//Creating subclass of abstract class, now we need to provide the implementation of rest of the methods

```
class M extends B
{
    public void b()
    {
        System.out.println("I am b");
    }
    public void d()
    {
        System.out.println("I am d");
    }
}
```

//Creating a test class that calls the methods of A interface

```
class Test5
{
    public static void main(String args[])
    {
        A a = new M();
    }
}
```

```
        a.a();  
        a.b();  
        a.c();  
        a.d();  
    }  
}
```

Output:

```
I am a  
I am b  
I am c  
I am d
```

Inner Classes

Java inner class or nested class is a class which is declared inside the class or interface.

We use inner classes to logically group classes and interfaces in one place so that it can be more readable and maintainable.

Additionally, it can access all the members of outer class including private data members and methods.

Syntax of Inner class

```
class Java_Outer_class  
{  
    //code  
    class Java_Inner_class  
    {  
        //code  
    }  
}
```

Advantage of java inner classes

There are basically three advantages of inner classes in java. They are as follows:

- 1) Nested classes represent a special type of relationship that is **it can access all the members (data members and methods) of outer class** including private.
- 2) Nested classes are used **to develop more readable and maintainable code** because it logically group classes and interfaces in one place only.
- 3) **Code Optimization**: It requires less code to write.

Types of Nested classes

There are two types of nested classes non-static and static nested classes. The non-static nested classes are also known as inner classes.

- Non-static nested class (inner class)
 1. Member inner class
 2. Anonymous inner class
 3. Local inner class
- Static nested class

Type	Description
Member Inner Class	A class created within class and outside method.
Anonymous Inner Class	A class created for implementing interface or extending class. Its name is decided by the java compiler.
Local Inner Class	A class created within method.
Static Nested Class	A static class created within class.
Nested Interface	An interface created within class or interface.

Member inner class example

In this example, we are creating msg() method in member inner class that is accessing the private data member of outer class.

```
class TestMemberOuter
{
    private int data = 30;
    class Inner
    {
        void msg()
        {
            System.out.println("data is "+data);
        }
    }
    public static void main(String args[])
    {
        TestMemberOuter obj = new TestMemberOuter ();
        TestMemberOuter.Inner in = obj.new Inner();
        in.msg();
    }
}
```

Output:

```
data is 30
```

Anonymous inner class

A class that have no name is known as anonymous inner class in java. It should be used if you have to override method of class or interface. Java Anonymous inner class can be created by two ways:

1. Class (may be abstract or concrete).
2. Interface

```
abstract class Person
{
    abstract void eat();
}
class TestAnonymousInner
{
    public static void main(String args[])
    {
        Person p = new Person()
        {
            void eat(){System.out.println("nice fruits");}
        };
        p.eat();
    }
}
```

Output:

```
nice fruits
```

Local inner class

A class i.e. created inside a method is called local inner class in java. If you want to invoke the methods of local inner class, you must instantiate this class inside the method.

```
public class localInner1
{
    private int data = 30; //instance variable
    void display()
    {
        class Local
        {
            void msg()
            {
                System.out.println(data);
            }
        }
        Local l = new Local();
        l.msg();
    }
    public static void main(String args[])
    {
        localInner1 obj = new localInner1();
        obj.display();
    }
}
```

static nested class

A static class i.e. created inside a class is called static nested class in java. It cannot access non-static data members and methods. It can be accessed by outer class name.

- It can access static data members of outer class including private.
- Static nested class cannot access non-static (instance) data member or method.

```
class TestOuter1
{
    static int data = 30;
    static class Inner
    {
        void msg()
        {
            System.out.println("data is "+data);
        }
    }
    public static void main(String args[])
    {
        TestOuter1.Inner obj = new TestOuter1.Inner();
        obj.msg();
    }
}
```

Nested Interface

An interface i.e. declared within another interface or class is known as nested interface. The nested interfaces are used to group related interfaces so that they can be easy to maintain. The nested interface must be referred by the outer interface or class. It can't be accessed directly.

Points to remember for nested interfaces

There are given some points that should be remembered by the java programmer.

- Nested interface must be public if it is declared inside the interface but it can have any access modifier if declared within the class.
- Nested interfaces are declared static implicitly.

Syntax of nested interface which is declared within the interface

```
interface interface_name
{
    ...
    interface nested_interface_name
    {
        ...
    }
}
```

Syntax of nested interface which is declared within the class

```
class class_name
{
    ...
    interface nested_interface_name
    {
        ...
    }
}
```

Example

```
interface Showable
{
    void show();
    interface Message
    {
        void msg();
    }
}
class TestNestedInterface1 implements Showable.Message
{
    public void msg()
    {
        System.out.println("Hello nested interface");
    }
    public static void main(String args[])
    {
        Showable.Message message = new TestNestedInterface1();//upcasting here
        message.msg();
    }
}
```

Output:

```
hello nested interface
```

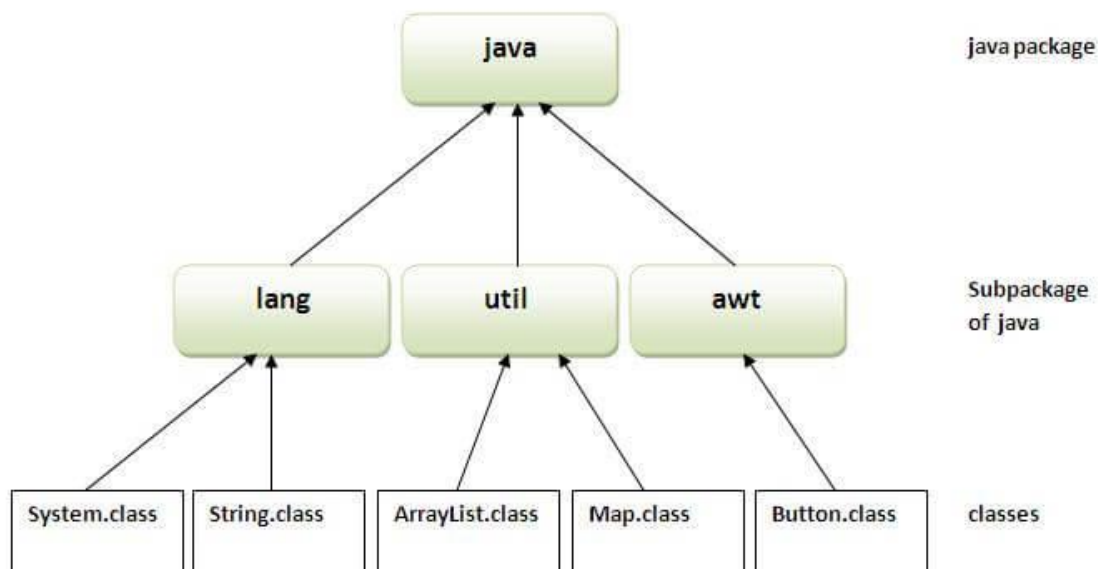
Package

A **java package** is a group of similar types of classes, interfaces and sub-packages. Package in java can be categorized in two form, built-in package and user-defined package. There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Here, we will have the detailed learning of creating and using user-defined packages.

Advantage of Java Package

- 1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- 2) Java package provides access protection.
- 3) Java package removes naming collision.



Simple example of java package

The **package keyword** is used to create a package in java.

```
package mypack;
public class Simple
{
    public static void main(String args[])
    {
        System.out.println("Welcome to package");
    }
}
```

How to compile java package

If you are not using any IDE, you need to follow the **syntax** given below:

```
javac -d directory javafilename
```

For **example**

```
javac -d . Simple.java
```

The -d switch specifies the destination where to put the generated class file. You can use any directory name like /home (in case of Linux), d:/abc (in case of windows) etc. If you want to keep the package within the same directory, you can use . (dot).

How to run java package program

You need to use fully qualified name e.g. mypack.Simple etc to run the class.

To Compile: javac -d . Simple.java

To Run: java mypack.Simple

Output: Welcome to package

How to access package from another package?

There are three ways to access the package from outside the package.

1. import package.*;
2. import package.classname;
3. fully qualified name.

Using packagename.*

If you use package.* then all the classes and interfaces of this package will be accessible but not subpackages.

The import keyword is used to make the classes and interface of another package accessible to the current package.

```
//save by A.java
package pack;
public class A
{
    public void msg()
    {
        System.out.println("Hello");
    }
}
```



```
//save by B.java
package mypack;
import pack.*;
class B
{
    public static void main(String args[])
    {
        A obj = new A();
        obj.msg();
    }
}
```

Output: Hello

Using packagename.classname

If you import package.classname then only declared class of this package will be accessible.

```
//save by A.java
package pack;
public class A
{
    public void msg()
    {
        System.out.println("Hello");
    }
}

//save by B.java
package mypack;
import pack.A;
class B
{
    public static void main(String args[])
    {
        A obj = new A();
        obj.msg();
    }
}
```

Output: Hello

Using fully qualified name

If you use fully qualified name then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.

It is generally used when two packages have same class name e.g. java.util and java.sql packages contain Date class.

```
//save by A.java
package pack;
public class A
{
```

```
public void msg()
{
    System.out.println("Hello");
}

//save by B.java
package mypack;
class B
{
    public static void main(String args[])
    {
        pack.A obj = new pack.A();//using fully qualified name
        obj.msg();
    }
}
```

Output: Hello

Subpackage

Package inside the package is called the **subpackage**. It should be created **to categorize the package further**.

Let's take an example, Sun Microsystems has defined a package named java that contains many classes like System, String, Reader, Writer, Socket etc. These classes represent a particular group e.g. Reader and Writer classes are for Input/Output operation, Socket and ServerSocket classes are for networking etc and so on. So, Sun has subcategorized the java package into subpackages such as lang, net, io etc. and put the Input/Output related classes in io package, Server and ServerSocket classes in net packages and so on.

```
package com.javatpoint.core;
class Simple
{
    public static void main(String args[])
    {
        System.out.println("Hello subpackage");
    }
}
```

To Compile: javac -d . Simple.java

To Run: java com.javatpoint.core.Simple

Output: Hello subpackage

Access Modifiers

There are two types of modifiers in java: **access modifiers** and **non-access modifiers**.

The access modifiers in java specifies accessibility (scope) of a data member, method, constructor or class.

There are 4 types of java access modifiers:

1. private
2. default
3. protected
4. public

There are many non-access modifiers such as static, abstract, synchronized, native, volatile, transient etc. Here, we will learn access modifiers.

1. The private access modifier is accessible only within class.
2. If you don't use any modifier, it is treated as **default** by default. The default modifier is accessible only within package.
3. The **protected access modifier** is accessible within package and outside the package but through inheritance only.

The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

4. The **public access modifier** is accessible everywhere. It has the widest scope among all other modifiers.

Understanding all java access modifiers

Let's understand the access modifiers by a simple table.

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

Object class in Java

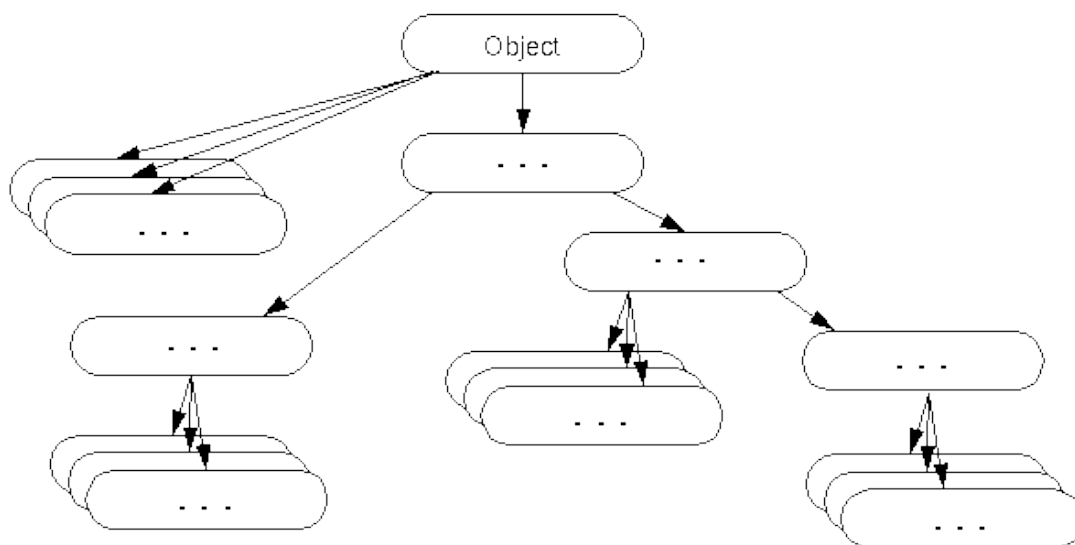
The **Object class** is the parent class of all the classes in java by default. In other words, it is the topmost class of java.

The Object class is beneficial if you want to refer any object whose type you don't know. Notice that parent class reference variable can refer the child class object, known as upcasting.

Let's take an example, there is `getObject()` method that returns an object but it can be of any type like `Employee`, `Student` etc, we can use `Object` class reference to refer that object. For example:

```
Object obj=getObject();//we don't know what object will be returned from this method
```

The Object class provides some common behaviors to all the objects such as object can be compared, object can be cloned, object can be notified etc.



Methods of Object class

The Object class provides many methods. They are as follows:

Method	Description
public final Class getClass()	returns the Class class object of this object. The Class class can further be used to get the metadata of this class.
public int hashCode()	returns the hashCode number for this object.
public boolean equals(Object obj)	compares the given object to this object.

protected Object clone() throws CloneNotSupportedException	creates and returns the exact copy (clone) of this object.
public String toString()	returns the string representation of this object.
public final void notify()	wakes up single thread, waiting on this object's monitor.
public final void notifyAll()	wakes up all the threads, waiting on this object's monitor.
public final void wait(long timeout)throws InterruptedException	causes the current thread to wait for the specified milliseconds, until another thread notifies (invokes notify() or notifyAll() method).
public final void wait(long timeout,int nanos)throws InterruptedException	causes the current thread to wait for the specified milliseconds and nanoseconds, until another thread notifies (invokes notify() or notifyAll() method).
public final void wait()throws InterruptedException	causes the current thread to wait, until another thread notifies (invokes notify() or notifyAll() method).
protected void finalize()throws Throwable	is invoked by the garbage collector before object is being garbage collected.

Math class

The **java.lang.Math** class contains various methods for performing basic numeric operations such as the logarithm, cube root, and trigonometric functions etc. The various java math methods are as follows:

Method	Description
<u>Math.abs()</u>	It will return the Absolute value of the given value.
<u>Math.max()</u>	It returns the Largest of two values.
<u>Math.min()</u>	It is used to return the Smallest of two values.
<u>Math.round()</u>	It is used to round of the decimal numbers to the nearest value.
<u>Math.sqrt()</u>	It is used to return the square root of a number.
<u>Math.cbrt()</u>	It is used to return the cube root of a number.
<u>Math.pow()</u>	It returns the value of first argument raised to the power to second argument.

<u>Math.ceil()</u>	It is used to find the smallest integer value that is greater than or equal to the argument or mathematical integer.
<u>Math.nextDown()</u>	It returns the floating-point value adjacent to d in the direction of negative infinity.
<u>Math.floor()</u>	It is used to find the largest integer value which is less than or equal to the argument and is equal to the mathematical integer of a double value.
<u>Math.random()</u>	It returns a double value with a positive sign, greater than or equal to 0.0 and less than 1.0.

Wrapper class in Java

Wrapper class in java provides the mechanism to *convert primitive into object and object into primitive*.

Since J2SE 5.0, **autoboxing** and **unboxing** feature converts primitive into object and object into primitive automatically. The automatic conversion of primitive into object is known as autoboxing and vice-versa unboxing.

The eight classes of *java.lang* package are known as wrapper classes in java. The list of eight wrapper classes are given below:

Primitive Type	Wrapper class
boolean	<u>Boolean</u>
char	<u>Character</u>
byte	<u>Byte</u>
short	<u>Short</u>
int	<u>Integer</u>
long	<u>Long</u>
float	<u>Float</u>
double	<u>Double</u>

Primitive to Wrapper

```
public class WrapperExample
{
    public static void main(String args[])
    {
        //Converting int into Integer
        int a = 20;
        Integer i = Integer.valueOf(a); //converting int into Integer
        Integer j = a; //autoboxing, now compiler will write Integer.valueOf(a) internally
        System.out.println(a+" "+i+" "+j);
    }
}
```

```
}  
}
```

Output:

```
20 20 20
```

Wrapper to Primitive

```
public class WrapperExample  
{  
    public static void main(String args[])  
    {  
        //Converting Integer to int  
        Integer a = new Integer(3);  
        int i = a.intValue();//converting Integer to int  
        int j = a;//unboxing, now compiler will write a.intValue() internally  
        System.out.println(a+" "+i+" "+j);  
    }  
}
```

Output:

```
3 3 3
```

Java String Methods

Here are the list of the methods available in the Java String class. These methods are explained in the separate tutorials with the help of examples. Links to the tutorials are provided below:

1. **char charAt(int index):**

It returns the character at the specified index. Specified index value should be between 0 to length() -1 both inclusive. It throws IndexOutOfBoundsException if index<0||>= length of String.

2. **boolean equals(Object obj):**

Compares the string with the specified string and returns true if both matches else false.

3. **boolean equalsIgnoreCase(String string):**

It works same as equals method but it doesn't consider the case while comparing strings. It does a case insensitive comparison.

4. **int compareTo(String string):**

This method compares the two strings based on the Unicode value of each character in the strings.

5. **boolean startsWith(String prefix, int offset):**

It checks whether the substring (starting from the specified offset index) is having the specified prefix or not.

6. **boolean startsWith(String prefix):**

It tests whether the string is having specified prefix, if yes then it returns true else false.

7. **boolean endsWith(String suffix):**

Checks whether the string ends with the specified suffix.

8. **int hashCode():**

It returns the hash code of the string.

9. **int indexOf(int ch):**

Returns the index of first occurrence of the specified character ch in the string.

10. **int lastIndexOf(int ch):**

It returns the last occurrence of the character ch in the string.

11. **int indexOf(String str):**

This method returns the index of first occurrence of specified substring str.

12. **String substring(int beginIndex, int endIndex):**

Returns the substring. The substring starts with character at beginIndex and ends with the character at endIndex.

13. **String concat(String str):**

Concatenates the specified string "str" at the end of the string.

14. **String replace(char oldChar, char newChar):**

It returns the new updated string after changing all the occurrences of oldChar with the newChar.

15. **boolean contains(CharSequence s):**

It checks whether the string contains the specified sequence of char values. If yes then it returns true else false. It throws NullPointerException of 's' is null.

16. **int length():**

It returns the length of a String.

Exception Handling

The **Exception Handling in Java** is one of the powerful *mechanism to handle the runtime errors* so that normal flow of the application can be maintained.

What is Exception?

Exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

What is Exception Handling?

Exception Handling is a mechanism to handle runtime errors such as `ClassNotFoundException`, `IOException`, `SQLException`, `RemoteException`, etc.

Advantage of Exception Handling

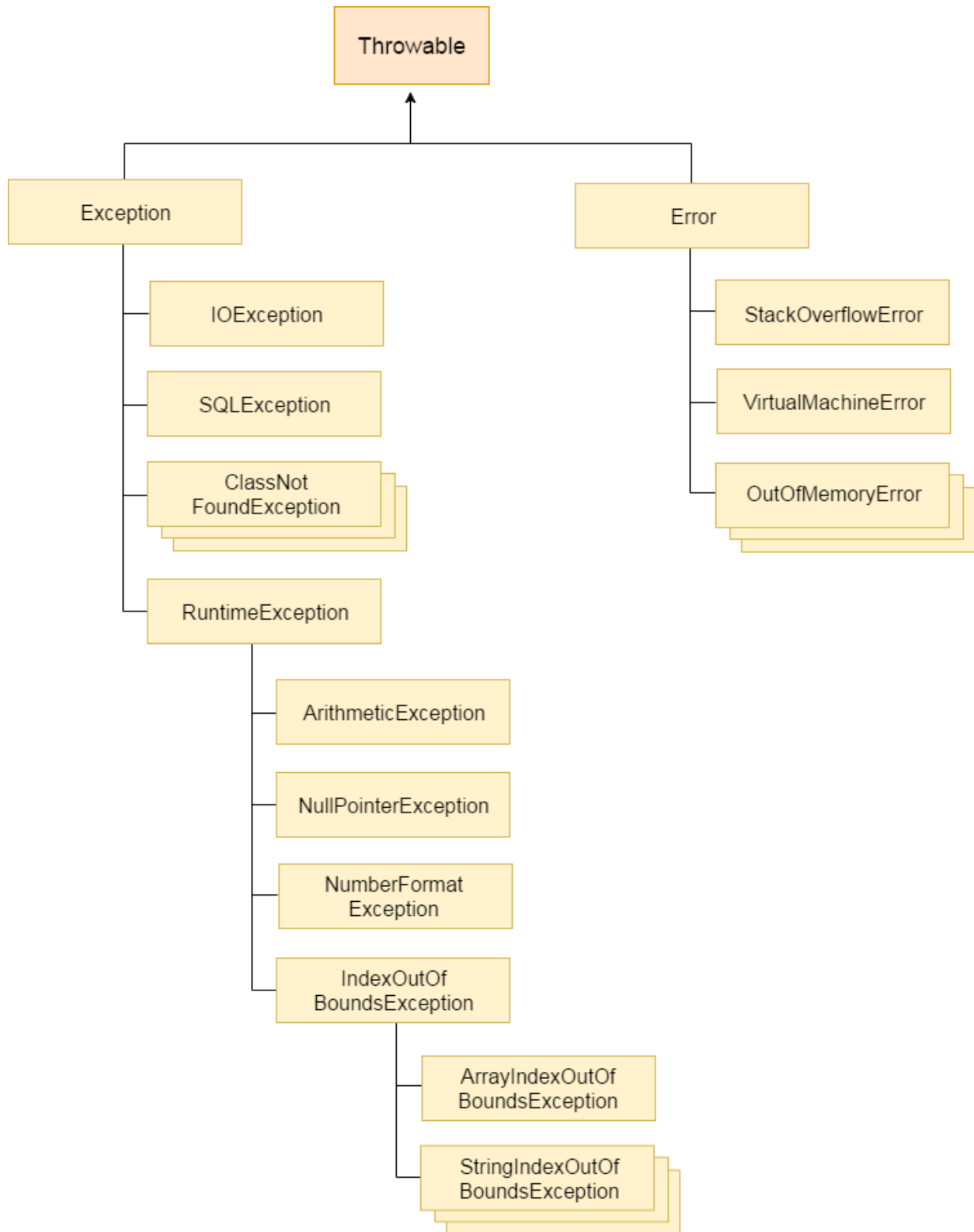
The core advantage of exception handling is **to maintain the normal flow of the application**. An exception normally disrupts the normal flow of the application that is why we use exception handling. Let's take a scenario:

```
statement 1;  
statement 2;  
statement 3;  
statement 4;  
statement 5;//exception occurs  
statement 6;  
statement 7;  
statement 8;  
statement 9;  
statement 10;
```

Suppose there are 10 statements in your program and there occurs an exception at statement 5, the rest of the code will not be executed i.e. statement 6 to 10 will not be executed. If we perform exception handling, the rest of the statement will be executed.

Hierarchy of Java Exception classes

The `java.lang.Throwable` class is the root class of Java Exception hierarchy which is inherited by two subclasses: `Exception` and `Error`. A hierarchy of Java Exception classes are given below:



Types of Java Exceptions

There are mainly two types of exceptions: checked and unchecked. Here, an error is considered as the unchecked exception. According to Oracle, there are three types of exceptions:

1. Checked Exception
2. Unchecked Exception
3. Error

1) Checked Exception

The classes which directly inherit Throwable class except RuntimeException and Error are known as checked exceptions e.g. IOException, SQLException etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes which inherit RuntimeException are known as unchecked exceptions e.g. ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

3) Error

Error is irrecoverable e.g. OutOfMemoryError, VirtualMachineError, AssertionError etc.

Java Exception Keywords

There are 5 keywords which are used in handling exceptions in Java.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place exception code. The try block must be followed by either catch or finally. It means, we can't use try block alone.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the important code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It doesn't throw an exception. It specifies that there may occur an exception in the method. It is always used with method signature.

Example

Let's see an example of Java Exception Handling where we using a try-catch statement to handle the exception.

```
public class JavaExceptionExample
{
    public static void main(String args[])
    {
        try
        {
            //code that may raise exception
            int data = 100 / 0;
        }
        catch(ArithmeticException e)
        {
            System.out.println(e);
        }
        //rest code of the program
        System.out.println("rest of the code...");
    }
}
```

Output:

```
Exception in thread main java.lang.ArithmeticException:/ by zero
rest of the code...
```

Common Scenarios of Java Exceptions

There are given some scenarios where unchecked exceptions may occur. They are as follows:

1) A scenario where ArithmeticException occurs

If we divide any number by zero, there occurs an ArithmeticException.

```
int a=50/0;//ArithmeticException
```

2) A scenario where NullPointerException occurs

If we have a null value in any variable, performing any operation on the variable throws a NullPointerException.

```
String s=null;
System.out.println(s.length());//NullPointerException
```

3) A scenario where NumberFormatException occurs

The wrong formatting of any value may occur NumberFormatException. Suppose I have a string variable that has characters, converting this variable into digit will occur NumberFormatException.

```
String s="abc";  
int i=Integer.parseInt(s);//NumberFormatException
```

4) A scenario where `ArrayIndexOutOfBoundsException` occurs

If you are inserting any value in the wrong index, it would result in `ArrayIndexOutOfBoundsException` as shown below:

```
int a[]=new int[5];  
a[10]=50; //ArrayIndexOutOfBoundsException
```

Java try block

Java **try** block is used to enclose the code that might throw an exception. It must be used within the method.

If an exception occurs at the particular statement of try block, the rest of the block code will not execute. So, it is recommended not to keep the code in try block that will not throw an exception.

Java try block must be followed by either catch or finally block.

Syntax of Java try-catch

```
try  
{  
    //code that may throw an exception  
}  
catch(Exception_class_Name ref)  
{  
}
```

Syntax of try-finally block

```
try  
{  
    //code that may throw an exception  
}  
finally  
{  
}
```

Java catch block

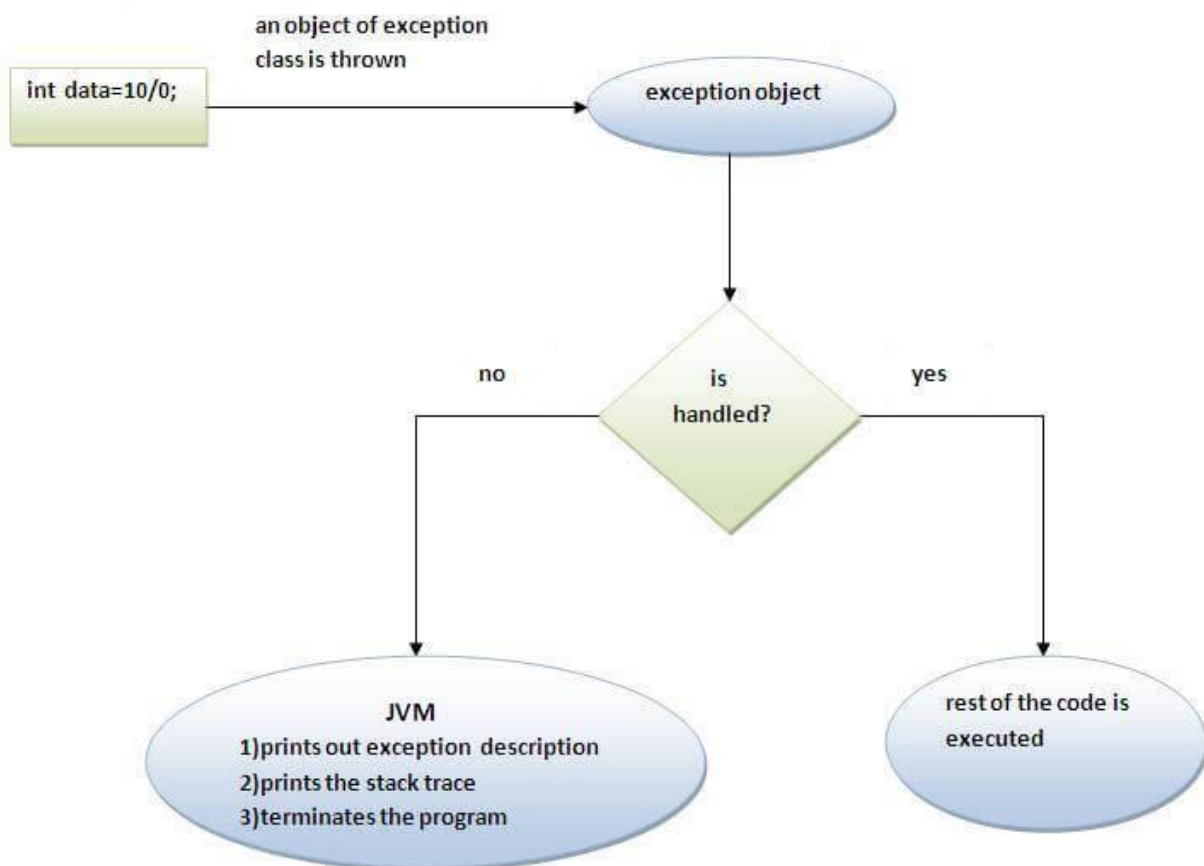
Java catch block is used to handle the Exception by declaring the type of exception within the parameter. The declared exception must be the parent class exception (i.e., `Exception`) or the

generated exception type. However, the good approach is to declare the generated type of exception.

The catch block must be used after the try block only. You can use multiple catch block with a single try block.

```
public class TryCatchExample
{
    public static void main(String[] args)
    {
        try
        {
            int arr[] = {1, 3, 5, 7};
            System.out.println(arr[10]); //may throw exception
        }
        // handling the array exception
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println(e);
        }
        System.out.println("rest of the code");
    }
}
```

Internal working of java try-catch block



Multi-catch block

A try block can be followed by one or more catch blocks. Each catch block must contain a different exception handler. So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.

Points to remember

- At a time only one exception occurs and at a time only one catch block is executed.
- All catch blocks must be ordered from most specific to most general, i.e. catch for `ArithmeticException` must come before catch for `Exception`.

```
public class MultipleCatchBlock
{
    public static void main(String[] args)
    {
        try
        {
            int a[] = new int[5];
            a[5] = 30 / 0;
        }
        catch(ArithmeticException e)
        {
            System.out.println("Arithmetic Exception occurs");
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println("ArrayIndexOutOfBoundsException occurs");
        }
        catch(Exception e)
        {
            System.out.println("Parent Exception occurs");
        }
        System.out.println("rest of the code");
    }
}
```

Output:

```
Arithmetic Exception occurs
rest of the code
```

Nested try block

The try block within a try block is known as nested try block in java.

Why use nested try block?

Sometimes a situation may arise where a part of a block may cause one error and the entire block itself may cause another error. In such cases, exception handlers have to be nested.

Syntax:

```
....  
try  
{  
    statement 1;  
    statement 2;  
    try  
    {  
        statement 1;  
        statement 2;  
    }  
    catch(Exception e)  
    {  
    }  
}  
catch(Exception e)  
{  
}  
....
```

Nested try example

```
class Excep  
{  
    public static void main(String args[])  
    {  
        try  
        {  
            try  
            {  
                System.out.println("going to divide");  
                int b = 39 / 0;  
            }  
            catch(ArithmeticException e)  
            {  
                System.out.println(e);  
            }  
            try  
            {  
                int a[] = new int[5];  
                a[5] = 4;  
            }  
            catch(ArrayIndexOutOfBoundsException e)  
            {  
                System.out.println(e);  
            }  
            System.out.println("other statement");  
        }  
        catch(Exception e)  
        {  
            System.out.println("handeled");  
        }  
        System.out.println("normal flow..");  
    }  
}
```


finally block

finally block is a block that is used *to execute important code* such as closing connection, stream etc.

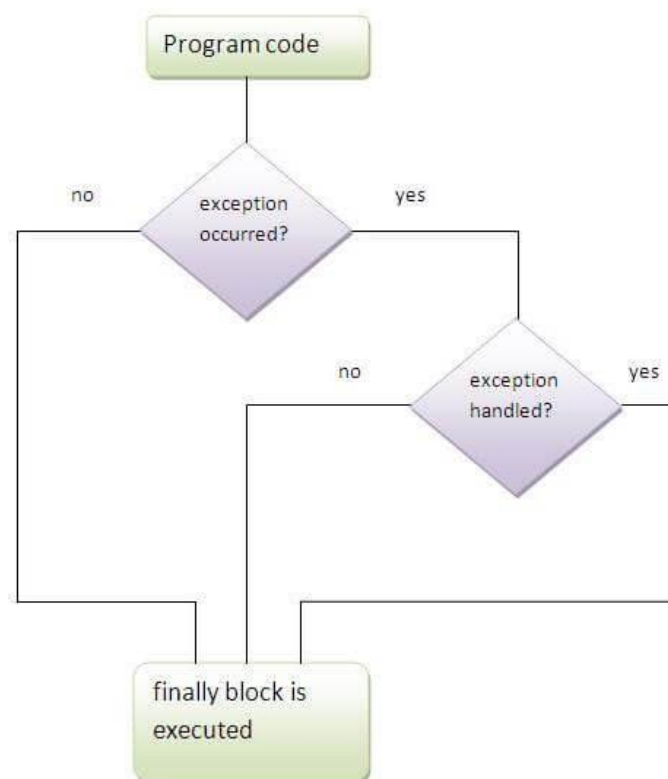
finally block is always executed whether exception is handled or not.

finally block follows try or catch block.

Note: If you don't handle exception, before terminating the program, JVM executes finally block(if any).

Why use java finally

- Finally block in java can be used to put "cleanup" code such as closing a file, closing connection etc.



Example where **exception occurs and handled:**

```
public class TestFinallyBlock
{
    public static void main(String args[])
    {
        try
        {
            int data = 25 / 0;
            System.out.println(data);
        }
    }
}
```

```
    }  
    catch(ArithmeticException e)  
    {  
        System.out.println(e);  
    }  
    finally  
    {  
        System.out.println("finally block is always executed");  
    }  
    System.out.println("rest of the code...");  
}  
}
```

Output:

```
Exception in thread main java.lang.ArithmeticException:/ by zero  
finally block is always executed  
rest of the code...
```

throw keyword

The Java throw keyword is used to explicitly throw an exception.

We can throw either checked or unchecked exception in java by throw keyword. The throw keyword is mainly used to throw custom exception. We will see custom exceptions later.

The syntax of java throw keyword is given below.

throw exception;

Let's see the example of throw IOException.

throw new IOException("sorry device error);

throw keyword example

In this example, we have created the validate method that takes integer value as a parameter. If the age is less than 18, we are throwing the ArithmeticException otherwise print a message welcome to vote.

```
public class TestThrow  
{  
    static void validate(int age)  
    {  
        if(age<18)  
            throw new ArithmeticException("not valid");  
        else  
            System.out.println("welcome to vote");  
    }  
    public static void main(String args[])  
    {  
        validate(13);  
        System.out.println("rest of the code...");  
    }  
}
```

Output:

```
Exception in thread main java.lang.ArithmeticException: not valid
```

Exception propagation

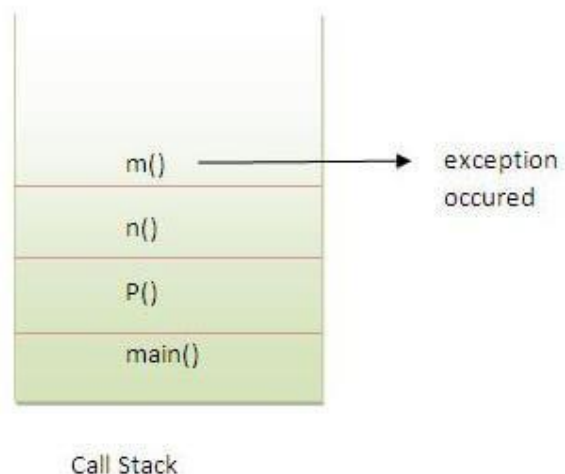
An exception is first thrown from the top of the stack and if it is not caught, it drops down the call stack to the previous method. If not caught there, the exception again drops down to the previous method, and so on until they are caught or until they reach the very bottom of the call stack. This is called exception propagation.

Rule: By default Unchecked Exceptions are forwarded in calling chain (propagated).

```
class TestExceptionPropagation
{
    void m()
    {
        int data = 50/0;
    }
    void n()
    {
        m();
    }
    void p()
    {
        try
        {
            n();
        }
        catch (Exception e)
        {
            System.out.println("exception handled");
        }
    }
    public static void main(String args[])
    {
        TestExceptionPropagation1 obj = new TestExceptionPropagation1();
        obj.p();
        System.out.println("normal flow...");
    }
}
```

Output:

```
exception handled
normal flow...
```



In the above example exception occurs in m() method where it is not handled,so it is propagated to previous n() method where it is not handled, again it is propagated to p() method where exception is handled.

Exception can be handled in any method in call stack either in main() method,p() method,n() method or m() method.

throws keyword

The **Java throws keyword** is used to declare an exception. It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code so that normal flow can be maintained.

Exception Handling is mainly used to handle the checked exceptions. If there occurs any unchecked exception such as NullPointerException, it is programmers fault that he is not performing check up before the code being used.

Syntax of java throws

```
return_type method_name() throws exception_class_name
{
    //method code
}
```

Which exception should be declared

Ans) checked exception only, because:

- **unchecked Exception:** under your control so correct your code.
- **error:** beyond your control e.g. you are unable to do anything if there occurs VirtualMachineError or StackOverflowError.

Advantage of Java throws keyword

Now Checked Exception can be propagated (forwarded in call stack).

It provides information to the caller of the method about the exception.

throws example

```
import java.io.IOException;
class Testthrows
{
    void m()throws IOException
    {
        throw new IOException("device error");//checked exception
    }
    void n()throws IOException
    {
        m();
    }
}
```

```

        void p()
        {
            try
            {
                n();
            }
            catch(Exception e)
            {
                System.out.println("exception handled");
            }
        }
        public static void main(String args[])
        {
            Testthrows obj = new Testthrows();
            obj.p();
            System.out.println("normal flow...");
        }
    }

```

Output:

```

exception handled
normal flow...

```

Difference between throw and throws:

There are many differences between throw and throws keywords. A list of differences between throw and throws are given below:

No.	throw	throws
1)	Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
2)	Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
3)	Throw is followed by an instance.	Throws is followed by class.
4)	Throw is used within the method.	Throws is used with the method signature.
5)	You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method()throws IOException,SQLException.

Difference between final, finally and finalize

There are many differences between final, finally and finalize. A list of differences between final, finally and finalize are given below:

No.	final	finally	finalize
1)	Final is used to apply restrictions on class, method and variable. Final class can't be inherited, final method can't be overridden and final variable value can't be changed.	Finally is used to place important code, it will be executed whether exception is handled or not.	Finalize is used to perform clean up processing just before object is garbage collected.
2)	Final is a keyword.	Finally is a block.	Finalize is a method.

Custom Exception

If you are creating your own Exception that is known as custom exception or user-defined exception. Java custom exceptions are used to customize the exception according to user need.

By the help of custom exception, you can have your own exception and message.

Let's see a simple example of java custom exception.

```

class InvalidAgeException extends Exception
{
    InvalidAgeException(String s)
    {
        super(s);
    }
}

class TestCustomException
{
    static void validate(int age)throws InvalidAgeException
    {
        if(age<18)
            throw new InvalidAgeException("not valid");
        else
            System.out.println("welcome to vote");
    }

    public static void main(String args[])
    {
        try
        {
            validate(13);
        }
        catch(Exception m)
        {
            System.out.println("Exception occurred: "+m);
        }
        System.out.println("rest of the code...");
    }
}

```

Output:

```

Exception occurred: InvalidAgeException:not valid
rest of the code...

```

BuiltIn Exceptions

Checked Exception	Unchecked Exceptions
ClassNotFoundException	ArithmeticException
IllegalAccessException	ArrayIndexOutOfBoundsException
NoSuchMethodException	IllegalArgumentException
IOException	NegativeArraySizeException
InterruptedException	NullPointerException
	NumberFormatException
	StringIndexOutOfBoundsException

Re-throwing An Exception :

We all know that exceptions occurred in the try block are caught in catch block. Thus caught exceptions can be re-thrown using **throw** keyword. Re-thrown exception must be handled some where in the program, otherwise program will terminate abruptly. For example,

```
public class ExceptionHandling
{
    public static void main(String[] args)
    {
        try
        {
            methodWithThrow();
        }
        catch(NullPointerException ex)
        {
            System.out.println("NullPointerException Re-thrown in methodWithThrow() method
                               will be handled here");
        }
    }
    static void methodWithThrow()
    {
        try
        {
            String s = null;
            System.out.println(s.length()); //This statement throws NullPointerException
        }
        catch(NullPointerException ex)
        {
            System.out.println("NullPointerException is caught here");
            throw ex; //Re-throwing NullPointerException
        }
    }
}
```

Multithreading

Multithreading in java is a process of executing multiple threads simultaneously.

A thread is a lightweight sub-process, the smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking.

However, we use multithreading than multiprocessing because threads use a shared memory area. They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.

Java Multithreading is mostly used in games, animation, etc.

Advantages of Java Multithreading

- 1) It **doesn't block the user** because threads are independent and you can perform multiple operations at the same time.
- 2) You **can perform many operations together, so it saves time**.
- 3) Threads are **independent**, so it doesn't affect other threads if an exception occurs in a single thread.

Multitasking

Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU. Multitasking can be achieved in two ways:

- Process-based Multitasking (Multiprocessing)
- Thread-based Multitasking (Multithreading)

1) Process-based Multitasking (Multiprocessing)

- Each process has an address in memory. In other words, each process allocates a separate memory area.
- A process is heavyweight.
- Cost of communication between the process is high.
- Switching from one process to another requires some time for saving and loading registers, memory maps, updating lists, etc.

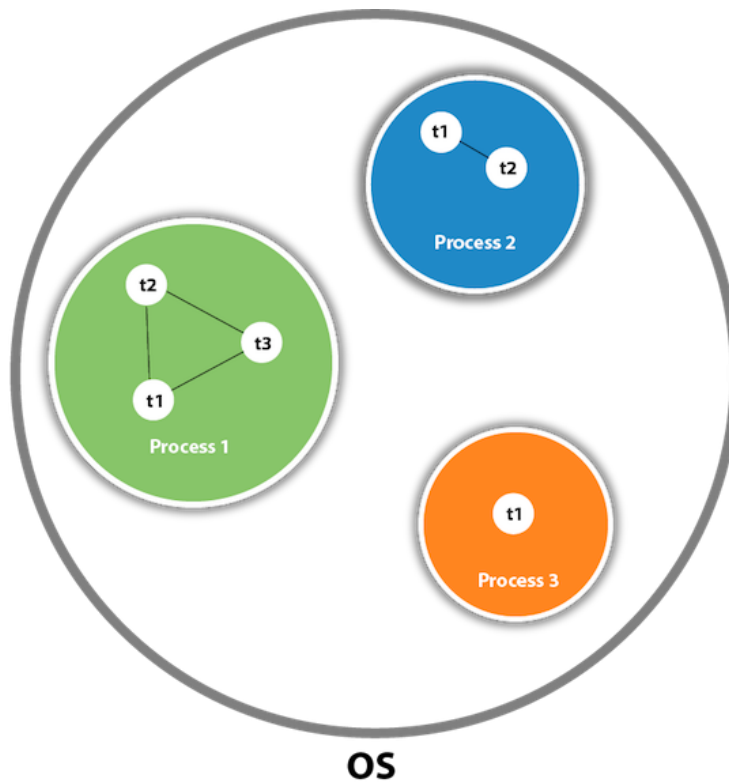
2) Thread-based Multitasking (Multithreading)

- Threads share the same address space.
- A thread is lightweight.
- Cost of communication between the thread is low.

What is Thread in java

A thread is a lightweight subprocess, the smallest unit of processing. It is a separate path of execution.

Threads are independent. If there occurs exception in one thread, it doesn't affect other threads. It uses a shared memory area.



As shown in the above figure, a thread is executed inside the process. There is context-switching between the threads. There can be multiple processes inside the OS, and one process can have multiple threads.

Java Thread class

Java provides **Thread class** to achieve thread programming. Thread class provides constructors and methods to create and perform operations on a thread. Thread class extends Object class and implements Runnable interface.

Thread Methods

S.N.	Modifier and Type	Method	Description
1	void	start()	It is used to start the execution of the thread.
2	void	run()	It is used to do an action for a thread.
3	static void	sleep()	It sleeps a thread for the specified amount of time.

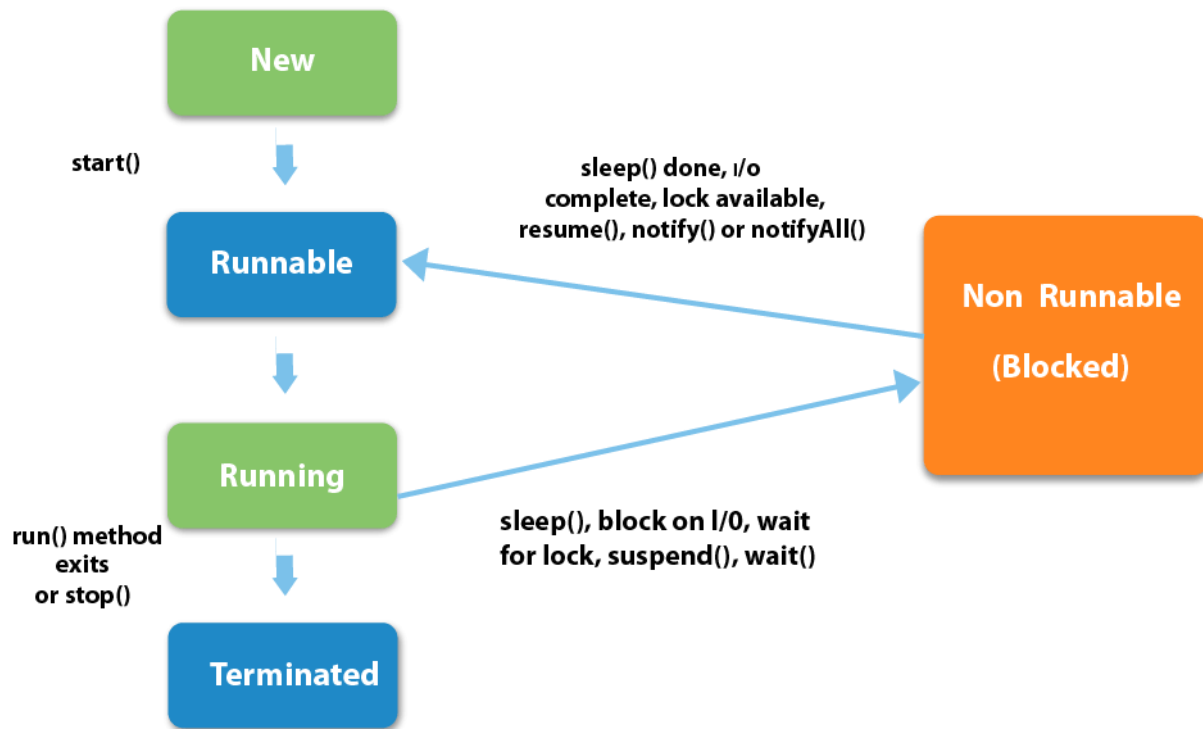
4	static Thread	<u>currentThread()</u>	It returns a reference to the currently executing thread object.
5	void	<u>join()</u>	It waits for a thread to die.
6	int	<u>getPriority()</u>	It returns the priority of the thread.
7	void	<u>setPriority()</u>	It changes the priority of the thread.
8	String	<u>getName()</u>	It returns the name of the thread.
9	void	<u>setName()</u>	It changes the name of the thread.
10	long	<u>getId()</u>	It returns the id of the thread.
11	boolean	<u>isAlive()</u>	It tests if the thread is alive.
12	void	<u>suspend()</u>	It is used to suspend the thread.
13	void	<u>resume()</u>	It is used to resume the suspended thread.
14	void	<u>stop()</u>	It is used to stop the thread.
15	void	<u>interrupt()</u>	It interrupts the thread.
16	boolean	<u>isinterrupted()</u>	It tests whether the thread has been interrupted.
17	static boolean	<u>interrupted()</u>	It tests whether the current thread has been interrupted.
18	String	<u>toString()</u>	It is used to return a string representation of this thread, including the thread's name, priority, and thread group.
19	void	<u>notify()</u>	It is used to give the notification for only one thread which is waiting for a particular object.
20	void	<u>notifyAll()</u>	It is used to give the notification to all waiting threads of a particular object.

Life cycle of a Thread (Thread States)

A thread can be in one of the five states. According to sun, there is only 4 states in **thread life cycle in java** new, runnable, non-runnable and terminated. There is no running state.

But for better understanding the threads, we are explaining it in the 5 states.

The life cycle of the thread in java is controlled by JVM. The java thread states are as follows:



1) New

The thread is in new state if you create an instance of Thread class but before the invocation of `start()` method.

2) Runnable

The thread is in runnable state after invocation of `start()` method, but the thread scheduler has not selected it to be the running thread.

3) Running

The thread is in running state if the thread scheduler has selected it.

4) Non-Runnable (Blocked)

This is the state when the thread is still alive, but is currently not eligible to run.

5) Terminated

A thread is in terminated or dead state when its `run()` method exits.

Creating thread

There are two ways to create a thread:

1. By extending Thread class
2. By implementing Runnable interface.

Thread class provide constructors and methods to create and perform operations on a thread. Thread class extends Object class and implements Runnable interface.

Commonly used Constructors of Thread class:

- Thread()
- Thread(String name)
- Thread(Runnable r)
- Thread(Runnable r, String name)

Runnable interface:

The Runnable interface should be implemented by any class whose instances are intended to be executed by a thread. Runnable interface have only one method named run().

public void run(): is used to perform action for a thread.

Starting a thread:

start() method of Thread class is used to start a newly created thread. It performs following tasks:

- A new thread starts(with new callstack).
- The thread moves from New state to the Runnable state.
- When the thread gets a chance to execute, its target run() method will run.

Thread Example by extending Thread class

```
class Multi extends Thread
{
    public void run()
    {
        System.out.println("thread is running...");
    }
    public static void main(String args[])
    {
        Multi t1 = new Multi();
        t1.start();
    }
}
```

Output: thread is running...

Thread Example by implementing Runnable interface

```
class Multi implements Runnable
{
    public void run()
    {
        System.out.println("thread is running...");
    }
}
```

```
public static void main(String args[])
{
    Multi m1 = new Multi();
    Thread t1 = new Thread(m1);
    t1.start();
}
}
```

Output: thread is running...

If you are not extending the Thread class, your class object would not be treated as a thread object. So you need to explicitly create Thread class object. We are passing the object of your class that implements Runnable so that your class run() method may execute.

Sleep method

The sleep() method of Thread class is used to sleep a thread for the specified amount of time.

The Thread class provides two methods for sleeping a thread:

- public static void sleep(long milliseconds) throws InterruptedException
- public static void sleep(long milliseconds, int nanos) throws InterruptedException

Example:

```
class TestSleepMethod extends Thread
{
    public void run()
    {
        for(int i=1; i<5; i++)
        {
            try
            {
                Thread.sleep(500);
            }
            catch (InterruptedException e)
            {
                System.out.println(e);
            }
            System.out.println(i);
        }
    }
    public static void main(String args[])
    {
        TestSleepMethod t1 = new TestSleepMethod();
        TestSleepMethod t2 = new TestSleepMethod();
        t1.start();
        t2.start();
    }
}
```

Output: 1 1 2 2 3 3 4 4

Priority of a Thread (Thread Priority):

Each thread have a priority. Priorities are represented by a number between 1 and 10. In most cases, thread scheduler schedules the threads according to their priority (known as preemptive scheduling). But it is not guaranteed because it depends on JVM specification that which scheduling it chooses.

3 constants defined in Thread class:

1. public static int MIN_PRIORITY
2. public static int NORM_PRIORITY
3. public static int MAX_PRIORITY

Default priority of a thread is 5 (NORM_PRIORITY). The value of MIN_PRIORITY is 1 and the value of MAX_PRIORITY is 10.

Example of priority of a Thread:

```
class TestMultiPriority extends Thread
{
    public void run()
    {
        System.out.println("running thread name is:"+Thread.currentThread().getName());
        System.out.println("running thread priority is:"+Thread.currentThread().getPriority());
    }
    public static void main(String args[])
    {
        TestMultiPriority m1 = new TestMultiPriority ();
        TestMultiPriority m2 = new TestMultiPriority ();
        m1.setPriority(Thread.MIN_PRIORITY);
        m2.setPriority(Thread.MAX_PRIORITY);
        m1.start();
        m2.start();
    }
}
```

Output:

```
running thread name is:Thread-0
running thread priority is:10
running thread name is:Thread-1
running thread priority is:1
```

Synchronization

Synchronization in java is the capability *to control the access of multiple threads to any shared resource*.

Synchronization is better option where we want to allow only one thread to access the shared resource.

Why use Synchronization?

The synchronization is mainly used to

1. To prevent thread interference.
2. To prevent consistency problem.

Thread Synchronization

There are two types of thread synchronization mutual exclusive and inter-thread communication.

1. Mutual Exclusive
 1. Synchronized method.
 2. Synchronized block.
 3. static synchronization.
2. Cooperation (Inter-thread communication in java)

Java synchronized method

If you declare any method as synchronized, it is known as synchronized method.

Synchronized method is used to lock an object for any shared resource.

When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.

```
class Table
{
    synchronized void printTable(int n) //synchronized method
    {
        for(int i=1; i<=5; i++)
        {
            System.out.println(n*i);
            try
            {
                Thread.sleep(400);
            }
            catch(Exception e)
            {
                System.out.println(e);
            }
        }
    }
}
```

```
}
class MyThread1 extends Thread
{
    Table t;
    MyThread1(Table t)
    {
        this.t = t;
    }
    public void run()
    {
        t.printTable(5);
    }
}
class MyThread2 extends Thread
{
    Table t;
    MyThread2(Table t)
    {
        this.t = t;
    }
    public void run()
    {
        t.printTable(100);
    }
}

public class TestSynchronization2
{
    public static void main(String args[])
    {
        Table obj = new Table();//only one object
        MyThread1 t1 = new MyThread1(obj);
        MyThread2 t2 = new MyThread2(obj);
        t1.start();
        t2.start();
    }
}
```

Output: 5
10
15
20
25
100
200
300
400
500

Synchronized block

Synchronized block can be used to perform synchronization on any specific resource of the method.

Suppose you have 50 lines of code in your method, but you want to synchronize only 5 lines, you can use synchronized block.

If you put all the codes of the method in the synchronized block, it will work same as the synchronized method.

Points to remember for Synchronized block

- Synchronized block is used to lock an object for any shared resource.
- Scope of synchronized block is smaller than the method.

Syntax to use synchronized block

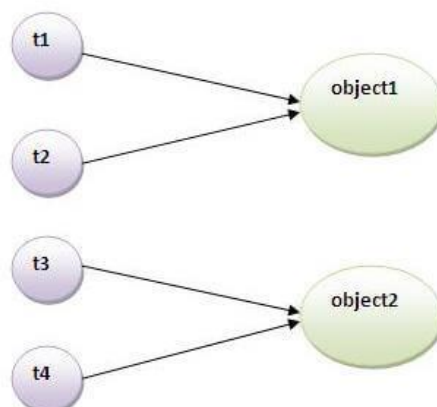
```
synchronized (object reference expression)
{
    //code block
}
```

Example:

```
class Table
{
    void printTable(int n)
    {
        synchronized(this) //synchronized block
        {
            for(int i=1; i<=5; i++)
            {
                System.out.println(n*i);
                try
                {
                    Thread.sleep(400);
                }
                catch(Exception e)
                {
                    System.out.println(e);
                }
            }
        }
    } //end of the method
}
```

Static synchronization

If you make any static method as synchronized, the lock will be on the class not on object.



Problem without static synchronization

Suppose there are two objects of a shared class(e.g. Table) named object1 and object2. In case of synchronized method and synchronized block there cannot be interference between t1 and t2 or t3 and t4 because t1 and t2 both refer to a common object that has a single lock. But there can be interference between t1 and t3 or t2 and t4 because t1 acquires another lock and t3 acquires another lock. I want no interference between t1 and t3 or t2 and t4. Static synchronization solves this problem.

Example of static synchronization

In this example we are applying the synchronized keyword on the static method to perform static synchronization.

```
class Table
{
    synchronized static void printTable(int n)
    {
        for(int i=1;i<=10;i++)
        {
            System.out.println(n*i);
            try
            {
                Thread.sleep(400);
            }
            catch(Exception e)
            {
            }
        }
    }
}

class MyThread1 extends Thread
{
    public void run()
    {
        Table.printTable(1);
    }
}

class MyThread2 extends Thread
{
    public void run()
    {
        Table.printTable(10);
    }
}

class MyThread3 extends Thread
{
    public void run()
    {
        Table.printTable(100);
    }
}

class MyThread4 extends Thread
{
    public void run()
```

```
        {
            Table.printTable(1000);
        }
    }
    public class TestSynchronization4
    {
        public static void main(String t[])
        {
            MyThread1 t1 = new MyThread1();
            MyThread2 t2 = new MyThread2();
            MyThread3 t3 = new MyThread3();
            MyThread4 t4 = new MyThread4();
            t1.start();
            t2.start();
            t3.start();
            t4.start();
        }
    }
}
```

Inter-thread communication

Inter-thread communication or **Co-operation** is all about allowing synchronized threads to communicate with each other.

Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed. It is implemented by following methods of **Object class**:

- wait()
- notify()
- notifyAll()

1) wait() method

Causes current thread to release the lock and wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed.

The current thread must own this object's monitor, so it must be called from the synchronized method only otherwise it will throw exception.

Method	Description
public final void wait()throws InterruptedException	waits until object is notified.
public final void wait(long timeout)throws InterruptedException	waits for the specified amount of time.

2) notify() method

Wakes up a single thread that is waiting on this object's monitor. If any threads are waiting on this object, one of them is chosen to be awakened. The choice is arbitrary and occurs at the discretion of the implementation.

Syntax: public final void notify()

3) notifyAll() method

Wakes up all threads that are waiting on this object's monitor.

Syntax: public final void notifyAll()

Why wait(), notify() and notifyAll() methods are defined in Object class not Thread class?

It is because they are related to lock and object has a lock.

Difference between wait and sleep

wait()	sleep()
wait() method releases the lock	sleep() method doesn't release the lock.
is the method of Object class	is the method of Thread class
is the non-static method	is the static method
is the non-static method	is the static method
should be notified by notify() or notifyAll() methods	after the specified amount of time, sleep is completed.

Example of inter thread communication in java

```
class Customer
{
    int amount=10000;
    synchronized void withdraw(int amount)
    {
        System.out.println("going to withdraw...");
        if(this.amount<amount)
        {
            System.out.println("Less balance; waiting for deposit...");
            try
            {
                wait();
            }
            catch(Exception e)
            {
            }
        }
        this.amount -= amount;
        System.out.println("withdraw completed...");
    }
    synchronized void deposit(int amount)
    {
        System.out.println("going to deposit...");
```

```
        this.amount += amount;
        System.out.println("deposit completed... ");
        notify();
    }
}
class Test
{
    public static void main(String args[])
    {
        final Customer c = new Customer();
        new Thread()
        {
            public void run()
            {
                c.withdraw(15000);
            }
        }.start();
        new Thread()
        {
            public void run()
            {
                c.deposit(10000);
            }
        }.start();
    }
}
```

Output: going to withdraw...
Less balance; waiting for deposit...
going to deposit...
deposit completed...
withdraw completed

Interrupting a Thread

If any thread is in sleeping or waiting state (i.e. `sleep()` or `wait()` is invoked), calling the `interrupt()` method on the thread, breaks out the sleeping or waiting state throwing `InterruptedException`. If the thread is not in the sleeping or waiting state, calling the `interrupt()` method performs normal behaviour and doesn't interrupt the thread but sets the interrupt flag to true. Let's first see the methods provided by the `Thread` class for thread interruption.

The 3 methods provided by the Thread class for interrupting a thread

- **public void interrupt()**
- **public static boolean interrupted()**
- **public boolean isInterrupted()**

Example of interrupting a thread that stops working

In this example, after interrupting the thread, we are propagating it, so it will stop working. If we don't want to stop the thread, we can handle it where `sleep()` or `wait()` method is invoked. Let's first see the example where we are propagating the exception.

```
class TestInterruptingThread extends Thread
{
    public void run()
    {
        try
        {
            Thread.sleep(1000);
            System.out.println("task");
        }
        catch (InterruptedException e)
        {
            throw new RuntimeException("Thread interrupted..." + e);
        }
    }
    public static void main(String args[])
    {
        TestInterruptingThread t1 = new TestInterruptingThread ();
        t1.start();
        try
        {
            t1.interrupt();
        }
        catch (Exception e)
        {
            System.out.println("Exception handled " + e);
        }
    }
}
```

Output:Exception in thread-0

```
java.lang.RuntimeException: Thread interrupted...
java.lang.InterruptedException: sleep interrupted
at A.run(A.java:7)
```

Example of interrupting a thread that doesn't stop working

In this example, after interrupting the thread, we handle the exception, so it will break out the sleeping but will not stop working.

```
class TestInterruptingThread extends Thread
{
    public void run()
    {
        try
        {
            Thread.sleep(1000);
            System.out.println("task");
        }
        catch (InterruptedException e)
        {
            System.out.println("Exception handled " + e);
        }
        System.out.println("thread is running...");
    }
}
```

```
    public static void main(String args[])
    {
        TestInterruptingThread t1=new TestInterruptingThread();
        t1.start();
        t1.interrupt();
    }
}
```

Output: Exception handled
 java.lang.InterruptedExceptio: sleep interrupted
 thread is running...

isInterrupted and interrupted method

The isInterrupted() method returns the interrupted flag either true or false. The static interrupted() method returns the interrupted flag afterthat it sets the flag to false if it is true.

```
public class TestInterruptingThread extends Thread
{
    public void run()
    {
        for(int i=1; i<=2; i++)
        {
            if(Thread.interrupted())
            {
                System.out.println("code for interrupted thread");
            }
            else
            {
                System.out.println("code for normal thread");
            }
        }
        }//end of for loop
    }
    public static void main(String args[])
    {
        TestInterruptingThread t1 = new TestInterruptingThread();
        TestInterruptingThread t2 = new TestInterruptingThread();
        t1.start();
        t1.interrupt();
        t2.start();
    }
}
```

Output:
Code for interrupted thread
code for normal thread
code for normal thread
code for normal thread

Collections in Java

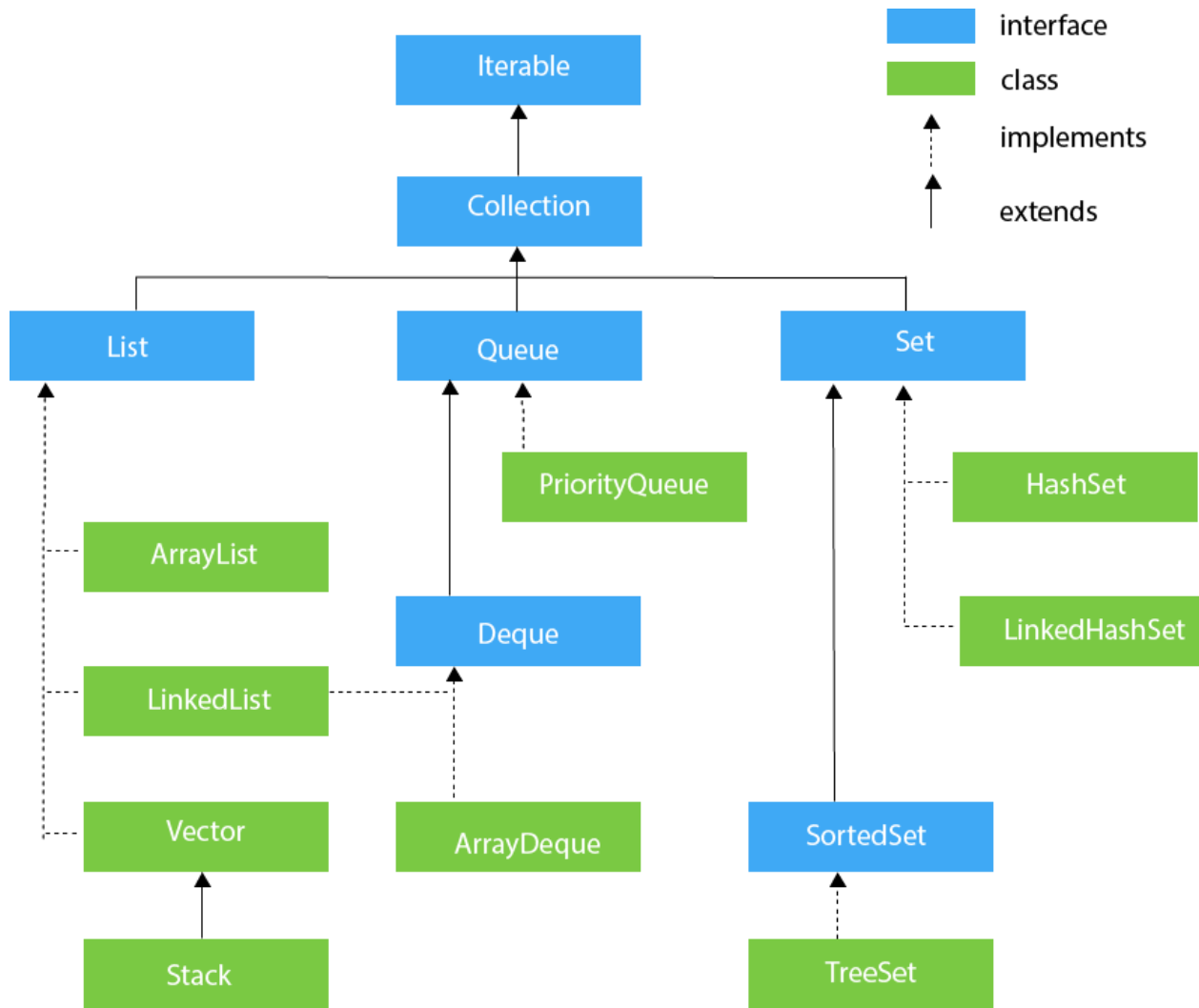
The **Collection in Java** is a framework that provides an architecture to store and manipulate the group of objects.

Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.

Java Collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).

Hierarchy of Collection Framework

Let us see the hierarchy of Collection framework. The **java.util** package contains all the classes and interfaces for the Collection framework.



What is Collection in Java

A Collection represents a single unit of objects, i.e., a group.

What is a framework in Java

- It provides readymade architecture.
- It represents a set of classes and interfaces.
- It is optional.

What is Collection framework

The Collection framework represents a unified architecture for storing and manipulating a group of objects. It has:

1. Interfaces and its implementations, i.e., classes
2. Algorithm

ArrayList class

ArrayList class uses a dynamic array for storing the elements. It inherits AbstractList class and implements List interface.

The important points about Java ArrayList class are:

- Java ArrayList class can contain duplicate elements.
- Java ArrayList class maintains insertion order.
- Java ArrayList class is non synchronized.
- Java ArrayList allows random access because array works at the index basis.
- In Java ArrayList class, manipulation is slow because a lot of shifting needs to occur if any element is removed from the array list.

Methods of Java ArrayList

Method	Description
void add(int index, E element)	It is used to insert the specified element at the specified position in a list.
boolean add(E e)	It is used to append the specified element at the end of a list.
boolean addAll(Collection<? extends E> c)	It is used to append all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's iterator.
boolean addAll(int index, Collection<? extends E> c)	It is used to append all the elements in the specified collection, starting at the specified position of the list.
void clear()	It is used to remove all of the elements from this list.
E get(int index)	It is used to fetch the element from the particular position of the list.
boolean isEmpty()	It returns true if the list is empty, otherwise false.
int lastIndexOf(Object o)	It is used to return the index in this list of the last occurrence of the specified element, or -1 if the list does not contain this element.
Object clone()	It is used to return a shallow copy of an ArrayList.
boolean contains(Object o)	It returns true if the list contains the specified element
int indexOf(Object o)	It is used to return the index in this list of the first occurrence of the specified element, or -1 if the List does not contain this element.

E remove(int index)	It is used to remove the element present at the specified position in the list.
boolean removeAll(Collection<?> c)	It is used to remove all the elements from the list.
void retainAll(Collection<?> c)	It is used to retain all the elements in the list that are present in the specified collection.
void sort(Comparator<? super E> c)	It is used to sort the elements of the list on the basis of specified comparator.
int size()	It is used to return the number of elements present in the list.

```

import java.util.*;
class ArrayListExample
{
    public static void main(String[] args)
    {
        ArrayList <Integer> arl = new ArrayList<Integer>();
        arl.add(10);
        arl.add(20);
        arl.add(30);
        arl.add(40);
        arl.add(50);
        arl.add(60);
        arl.remove(2);
        System.out.println(arl.size());

        //Traversing using Iterator
        Iterator it = arl.iterator();
        while(it.hasNext())
            System.out.println(it.next()); // A obj = new B();

        System.out.println(arl);
        for(Integer obj:arl) //Traversing using for each loop
            System.out.println(obj);

        // Traversing using ListIterator
        ListIterator<Integer> li = arl.listIterator();
        while(li.hasNext())
        {
            System.out.println(li.next());
        }

        // Traversing in reverse using ListIterator
        ListIterator<Integer> rev = arl.listIterator(arl.size());
        while(rev.hasPrevious())
        {
            Integer val = rev.previous();
            System.out.println(val);
        }
    }
}

```

LinkedList class

Java LinkedList class uses a doubly linked list to store the elements. It provides a linked-list data structure. It inherits the AbstractList class and implements List and Deque interfaces.

The important points about Java LinkedList are:

- Java LinkedList class can contain duplicate elements.
- Java LinkedList class maintains insertion order.
- Java LinkedList class is non synchronized.
- In Java LinkedList class, manipulation is fast because no shifting needs to occur.
- Java LinkedList class can be used as a list, stack or queue.

Methods of Java LinkedList

Method	Description
boolean add(E e)	It is used to append the specified element to the end of a list.
void add(int index, E element)	It is used to insert the specified element at the specified position index in a list.
void addFirst(E e)	It is used to insert the given element at the beginning of a list.
void addLast(E e)	It is used to append the given element to the end of a list.
void clear()	It is used to remove all the elements from a list.
E element()	It is used to retrieve the first element of a list.
E get(int index)	It is used to return the element at the specified position in a list.
E getFirst()	It is used to return the first element in a list.
E getLast()	It is used to return the last element in a list.
int indexOf(Object o)	It is used to return the index in a list of the first occurrence of the specified element, or -1 if the list does not contain any element.
int lastIndexOf(Object o)	It is used to return the index in a list of the last occurrence of the specified element, or -1 if the list does not contain any element.
ListIterator<E> listIterator(int index)	It is used to return a list-iterator of the elements in proper sequence, starting at the specified position in the list.
E remove()	It is used to retrieve and removes the first element of a list.
E remove(int index)	It is used to remove the element at the specified position in a list.

E removeFirst()	It removes and returns the first element from a list.
E removeLast()	It removes and returns the last element from a list.
E set(int index, E element)	It replaces the element at the specified position in a list with the specified element.
int size()	It is used to return the number of elements in a list.

LinkedList Example

```
import java.util.*;
public class LinkedListExample
{
    public static void main(String args[])
    {
        LinkedList<String> al = new LinkedList<String>();
        al.add("Ravi");
        al.add("Vijay");
        al.add("Ravi");
        al.add("Ajay");

        Iterator<String> itr = al.iterator();
        while(itr.hasNext())
        {
            System.out.println(itr.next());
        }
    }
}
```

Difference between ArrayList and LinkedList

ArrayList and LinkedList both implements List interface and maintains insertion order. Both are non synchronized classes.

However, there are many differences between ArrayList and LinkedList classes that are given below.

ArrayList	LinkedList
1) ArrayList internally uses a dynamic array to store the elements.	LinkedList internally uses a doubly linked list to store the elements.
2) Manipulation with ArrayList is slow because it internally uses an array. If any element is removed from the array, all the bits are shifted in memory.	Manipulation with LinkedList is faster than ArrayList because it uses a doubly linked list, so no bit shifting is required in memory.
3) An ArrayList class can act as a list only because it implements List only.	LinkedList class can act as a list and queue both because it implements List and Deque interfaces.
4) ArrayList is better for storing and accessing data.	LinkedList is better for manipulating data.

Vector Class

Java Vector class comes under the java.util package. The vector class implements a growable array of objects. Like an array, it contains the component that can be accessed using an integer index.

Vector is very useful if we don't know the size of an array in advance or we need one that can change the size over the lifetime of a program.

Vector implements a dynamic array that means it can grow or shrink as required. It is similar to the ArrayList, but with two differences-

- Vector is synchronized.
- The vector contains many legacy methods that are not the part of a collections framework

Vector Class Constructors

Vector class supports four types of constructors. These are:

SN	Constructor	Description
1)	vector()	It constructs an empty vector with the default size as 10.
2)	vector(int initialCapacity)	It constructs an empty vector with the specified initial capacity and with its capacity increment equal to zero.
3)	vector(int initialCapacity, int capacityIncrement)	It constructs an empty vector with the specified initial capacity and capacity increment.
4)	Vector(Collection<? extends E> c)	It constructs a vector that contains the elements of a collection c.

The following are the list of vector class methods:

SN	Method	Description
1	add()	It is used to append the specified element in the given vector.
2	addAll()	It is used to append all of the elements in the specified collection to the end of this Vector.
3	addElement()	It is used to append the specified component to the end of this vector. It increases the vector size by one.
4	capacity()	It is used to get the current capacity of this vector.
5	clear()	It is used to delete all of the elements from this vector.

6	<u>clone()</u>	It returns a clone of this vector.
7	<u>contains()</u>	It returns true if the vector contains the specified element.
8	<u>containsAll()</u>	It returns true if the vector contains all of the elements in the specified collection.
9	<u>elementAt()</u>	It is used to get the component at the specified index.
10	<u>elements()</u>	It returns an enumeration of the components of a vector.
11	<u>equals()</u>	It is used to compare the specified object with the vector for equality.
12	<u>firstElement()</u>	It is used to get the first component of the vector.
13	<u>get()</u>	It is used to get an element at the specified position in the vector.
14	<u>indexOf()</u>	It is used to get the index of the first occurrence of the specified element in the vector. It returns -1 if the vector does not contain the element.
15	<u>insertElementAt()</u>	It is used to insert the specified object as a component in the given vector at the specified index.
16	<u>isEmpty()</u>	It is used to check if this vector has no components.
17	<u>iterator()</u>	It is used to get an iterator over the elements in the list in proper sequence.
18	<u>lastElement()</u>	It is used to get the last component of the vector.
19	<u>lastIndexOf()</u>	It is used to get the index of the last occurrence of the specified element in the vector. It returns -1 if the vector does not contain the element.
20	<u>listIterator()</u>	It is used to get a list iterator over the elements in the list in proper sequence.
21	<u>remove()</u>	It is used to remove the specified element from the vector. If the vector does not contain the element, it is unchanged.
22	<u>size()</u>	It is used to get the number of components in the given vector.
23	<u>sort()</u>	It is used to sort the list according to the order induced by the specified Comparator.

Example

```
import java.util.*;
public class VectorExample
{
    public static void main(String args[])
```

```

{
    //Create an empty vector with initial capacity 4
    Vector<String> vec = new Vector<String>(4);
    //Adding elements to a vector
    vec.add("Tiger");
    vec.add("Lion");
    vec.add("Dog");
    vec.add("Elephant");
    //Check size and capacity
    System.out.println("Size is: "+vec.size());
    System.out.println("Default capacity is: "+vec.capacity());
    //Display Vector elements
    System.out.println("Vector element is: "+vec);
    vec.addElement("Rat");
    vec.addElement("Cat");
    vec.addElement("Deer");
    //Again check size and capacity after two insertions
    System.out.println("Size after addition: "+vec.size());
    System.out.println("Capacity after addition is: "+vec.capacity());
    //Display Vector elements again
    System.out.println("Elements are: "+vec);
    //Checking if Tiger is present or not in this vector
    if(vec.contains("Tiger"))
    {
        System.out.println("Tiger is present at the index " +vec.indexOf("Tiger"));
    }
    else
    {
        System.out.println("Tiger is not present in the list.");
    }
    //Get the first element
    System.out.println("The first animal of the vector is = "+vec.firstElement());
    //Get the last element
    System.out.println("The last animal of the vector is = "+vec.lastElement());
}
}

```

Difference between ArrayList and Vector

ArrayList and Vector both implements List interface and maintains insertion order.

However, there are many differences between ArrayList and Vector classes that are given below.

ArrayList	Vector
1) ArrayList is not synchronized .	Vector is synchronized .
2) ArrayList increments 50% of current array size if the number of elements exceeds from its capacity.	Vector increments 100% means doubles the array size if the total number of elements exceeds than its capacity.
3) ArrayList is not a legacy class. It is introduced in JDK 1.2.	Vector is a legacy class.

4) ArrayList is fast because it is non-synchronized.	Vector is slow because it is synchronized, i.e., in a multithreading environment, it holds the other threads in runnable or non-runnable state until current thread releases the lock of the object.
5) ArrayList uses the Iterator interface to traverse the elements.	A Vector can use the Iterator interface or Enumeration interface to traverse the elements.

Stack Class

Stack is a subclass of Vector that implements a standard last-in, first-out stack.

Stack only defines the default constructor, which creates an empty stack. Stack includes all the methods defined by Vector, and adds several of its own.

Apart from the methods inherited from its parent class Vector, Stack defines the following methods

Sr.No.	Method & Description
1	boolean empty() Tests if this stack is empty. Returns true if the stack is empty, and returns false if the stack contains elements.
2	Object peek() Returns the element on the top of the stack, but does not remove it.
3	Object pop() Returns the element on the top of the stack, removing it in the process.
4	Object push(Object element) Pushes the element onto the stack. Element is also returned.
5	int search(Object element) Searches for element in the stack. If found, its offset from the top of the stack is returned. Otherwise, -1 is returned.

Example

```
import java.util.*;
public class StackDemo
{
    static void showpush(Stack st, int a)
    {
        st.push(new Integer(a));
        System.out.println("push(" + a + ")");
        System.out.println("stack: " + st);
    }
    static void showpop(Stack st)
    {
        System.out.print("pop -> ");
        Integer a = (Integer) st.pop();
        System.out.println(a);
    }
}
```

```
        System.out.println("stack: " + st);
    }
    public static void main(String args[])
    {
        Stack st = new Stack();
        System.out.println("stack: " + st);
        showpush(st, 42);
        showpush(st, 66);
        showpush(st, 99);
        showpop(st);
        showpop(st);
        showpop(st);
        try
        {
            showpop(st);
        }
        catch (EmptyStackException e)
        {
            System.out.println("empty stack");
        }
    }
}
```

Hashtable class

Java Hashtable class implements a hashtable, which maps keys to values. It inherits Dictionary class and implements the Map interface.

Points to remember

- A Hashtable is an array of a list. Each list is known as a bucket. The position of the bucket is identified by calling the hashCode() method. A Hashtable contains values based on the key.
- Java Hashtable class contains unique elements.
- Java Hashtable class doesn't allow null key or value.
- Java Hashtable class is synchronized.
- The initial default capacity of Hashtable class is 11 whereas loadFactor is 0.75.

Hashtable class Parameters

Let's see the Parameters for java.util.Hashtable class.

- **K**: It is the type of keys maintained by this map.
- **V**: It is the type of mapped values.

Constructors of Java Hashtable class

Constructor	Description
Hashtable()	It creates an empty hashtable having the initial default capacity and load factor.

Hashtable(int capacity)	It accepts an integer parameter and creates a hash table that contains a specified initial capacity.
Hashtable(int capacity, float loadFactor)	It is used to create a hash table having the specified initial capacity and loadFactor.
Hashtable(Map<? extends K,? extends V> t)	It creates a new hash table with the same mappings as the given Map.

Methods of Java Hashtable class

Method	Description
void clear()	It is used to reset the hash table.
Object clone()	It returns a shallow copy of the Hashtable.
Enumeration elements()	It returns an enumeration of the values in the hash table.
boolean equals(Object o)	It is used to compare the specified Object with the Map.
V put(K key, V value)	It inserts the specified value with the specified key in the hash table.
boolean remove(Object key, Object value)	It removes the specified values with the associated specified keys from the hashtable.
V replace(K key, V value)	It replaces the specified value for a specified key.
Collection values()	It returns a collection view of the values contained in the map.
boolean isEmpty()	This method returns true if the hash table is empty; returns false if it contains at least one key.
V get(Object key)	This method returns the object that contains the value associated with the key.
V remove(Object key)	It is used to remove the key and its value. This method returns the value associated with the key.
int size()	This method returns the number of entries in the hash table.

Hashtable Example

```
import java.util.*;
class HashtableExample
{
    public static void main(String args[])
    {
```

```
        Hashtable<Integer,String> hm = new Hashtable<Integer,String>();

        hm.put(100, "Amit");
        hm.put(102, "Ravi");
        hm.put(101, "Vijay");
        hm.put(103, "Rahul");
        for(Map.Entry m : hm.entrySet())
        {
            System.out.println(m.getKey()+" "+m.getValue());
        }
    }
```

Output:

```
103 Rahul
102 Ravi
101 Vijay
100 Amit
```

Comparable interface

Java Comparable interface is used to order the objects of the user-defined class. This interface is found in java.lang package and contains only one method named compareTo(Object). It provides a single sorting sequence only, i.e., you can sort the elements on the basis of single data member only. For example, it may be rollno, name, age or anything else.

compareTo(Object obj) method

public int compareTo(Object obj): It is used to compare the current object with the specified object. It returns

- positive integer, if the current object is greater than the specified object.
- negative integer, if the current object is less than the specified object.
- zero, if the current object is equal to the specified object.

We can sort the elements of:

1. String objects
2. Wrapper class objects
3. User-defined class objects

Comparator interface

Java Comparator interface is used to order the objects of a user-defined class.

This interface is found in `java.util` package and contains 2 methods `compare(Object obj1, Object obj2)` and `equals(Object element)`.

It provides multiple sorting sequences, i.e., you can sort the elements on the basis of any data member, for example, rollno, name, age or anything else.

Methods of Java Comparator Interface

Method	Description
<code>public int compare(Object obj1, Object obj2)</code>	It compares the first object with the second object.
<code>public boolean equals(Object obj)</code>	It is used to compare the current object with the specified object.
<code>public boolean equals(Object obj)</code>	It is used to compare the current object with the specified object.

Difference between Comparable and Comparator

Comparable and Comparator both are interfaces and can be used to sort collection elements.

However, there are many differences between Comparable and Comparator interfaces that are given below.

Comparable	Comparator
1) Comparable provides a single sorting sequence . In other words, we can sort the collection on the basis of a single element such as id, name, and price.	The Comparator provides multiple sorting sequences . In other words, we can sort the collection on the basis of multiple elements such as id, name, and price etc.
2) Comparable affects the original class , i.e., the actual class is modified.	Comparator doesn't affect the original class , i.e., the actual class is not modified.
3) Comparable provides compareTo() method to sort elements.	Comparator provides compare() method to sort elements.
4) Comparable is present in java.lang package.	A Comparator is present in the java.util package.
5) We can sort the list elements of Comparable type by Collections.sort(List) method.	We can sort the list elements of Comparator type by Collections.sort(List, Comparator) method.

Comparable Example

Let's see the example of a Comparable interface that sorts the list elements on the basis of age.

//Creating a class which implements Comparable Interface

```
import java.util.*;
import java.io.*;
class Student implements Comparable<Student>
{
    int rollNo;
    String name;
    int age;
    Student(int rollNo,String name,int age)
    {
        this.rollNo = rollNo;
        this.name = name;
        this.age = age;
    }
    public int compareTo(Student st)
    {
        if(age == st.age)
            return 0;
        else if(age > st.age)
            return 1;
        else
            return -1;
    }
}
//Creating a test class to sort the elements
public class TestSort
{
    public static void main(String args[])
    {
        ArrayList<Student> al = new ArrayList<Student>();
        al.add(new Student(101, "Vijay", 23));
        al.add(new Student(106, "Ajay", 27));
        al.add(new Student(105, "Jai", 21));

        Collections.sort(al);
        for(Student st:al)
        {
            System.out.println(st.rollNo+" "+st.name+" "+st.age);
        }
    }
}
```

Output:

```
105 Jai 21
101 Vijay 23
106 Ajay 27
```

Comparator Example (Generic)

Student.java

```
class Student
{
    int rollNo;
```

```
String name;  
int age;  
Student(int rollno, String name, int age)  
{  
    this.rollno = rollno;  
    this.name = name;  
    this.age = age;  
}  
}
```

AgeComparator.java

```
import java.util.*;  
class AgeComparator implements Comparator<Student>  
{  
    public int compare(Student s1, Student s2)  
    {  
        if(s1.age == s2.age)  
            return 0;  
        else if(s1.age > s2.age)  
            return 1;  
        else  
            return -1;  
    }  
}
```

NameComparator.java

This class provides comparison logic based on the name. In such case, we are using the compareTo() method of String class, which internally provides the comparison logic.

```
import java.util.*;  
class NameComparator implements Comparator<Student>  
{  
    public int compare(Student s1, Student s2)  
    {  
        return s1.name.compareTo(s2.name);  
    }  
}
```

Simple.java

In this class, we are printing the values of the object by sorting on the basis of name and age.

```
import java.util.*;  
import java.io.*;  
class Simple  
{  
    public static void main(String args[])  
    {  
        ArrayList<Student> al = new ArrayList<Student>();  
        al.add(new Student(101, "Vijay", 23));  
        al.add(new Student(106, "Ajay", 27));  
        al.add(new Student(105, "Jai", 21));  
  
        System.out.println("Sorting by Name");  
  
        Collections.sort(al,new NameComparator());  
    }  
}
```

```
        for(Student st: al)
        {
            System.out.println(st.rollno+" "+st.name+" "+st.age);
        }

        System.out.println("Sorting by age");

        Collections.sort(al,new AgeComparator());
        for(Student st: al)
        {
            System.out.println(st.rollno+" "+st.name+" "+st.age);
        }
    }
}
```

Output:

```
Sorting by Name
106 Ajay 27
105 Jai 21
101 Vijay 23

Sorting by age
105 Jai 21
101 Vijay 23
106 Ajay 27
```

Iterator in Java

'Iterator' is an interface which belongs to collection framework. It allows us to traverse the collection, access the data element and remove the data elements of the collection.

java.util package has **public interface Iterator** and contains three methods:

1. **boolean hasNext():** It returns true if Iterator has more element to iterate.
2. **Object next():** It returns the next element in the collection until the hasNext() method return true. This method throws 'NoSuchElementException' if there is no next element.
3. **void remove():** It removes the current element in the collection. This method throws 'IllegalStateException' if this function is called before next() is invoked.

```
import java.io.*;
import java.util.*;

class Test
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<String>();

        list.add("A");
        list.add("B");
        list.add("C");
        list.add("D");
        list.add("E");
```



```

        // Iterator to traverse the list
        Iterator iterator = list.iterator();
        System.out.println("List elements : ");
        while (iterator.hasNext())
            System.out.print(iterator.next() + " ");
        System.out.println();
    }
}

```

ListIterator

'ListIterator' in Java is an Iterator which allows users to traverse Collection in both direction. It contains the following methods:

1. **void add(Object object)**: It inserts object immediately before the element that is returned by the next() function.
2. **boolean hasNext()**: It returns true if the list has a next element.
3. **boolean hasPrevious()**: It returns true if the list has a previous element.
4. **Object next()**: It returns the next element of the list. It throws 'NoSuchElementException' if there is no next element in the list.
5. **Object previous()**: It returns the previous element of the list. It throws 'NoSuchElementException' if there is no previous element.
6. **void remove()**: It removes the current element from the list. It throws 'IllegalStateException' if this function is called before next() or previous() is invoked.

```

import java.io.*;
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<String>();

        list.add("A");
        list.add("B");
        list.add("C");
        list.add("D");
        list.add("E");
        // ListIterator to traverse the list
        ListIterator iterator = list.listIterator();
        // Traversing the list in forward direction
        System.out.println("Displaying list elements in forward direction : ");
        while (iterator.hasNext())
            System.out.print(iterator.next() + " ");
        System.out.println();
        // Traversing the list in backward direction
        System.out.println("Displaying list elements in backward direction : ");
        while (iterator.hasPrevious())
            System.out.print(iterator.previous() + " ");
        System.out.println();
    }
}

```

Iterator vs ListIterator:

BASIS FOR COMPARISON	ITERATOR	LISTITERATOR
Basic	Iterator can traverse the elements in a collection only in forward direction.	ListIterator can traverse the elements in a collection in forward as well as the backwards direction.
Add	Iterator is unable to add elements to a collection.	ListIteror can add elements to a collection.
Modify	Iterator can not modify the elements in a collection.	ListIterator can modify the elements in a collection using set().
Traverse	Iterator can traverse Map, List and Set.	ListIterator can traverse List objects only.
Index	Iterator has no method to obtain an index of the element in a collection.	Using ListIterator, you can obtain an index of the element in a collection.

Legacy Classes

Early version of java did not include the Collections framework. It only defined several classes and interfaces that provide methods for storing objects. When Collections framework were added in J2SE 1.2, the original classes were reengineered to support the collection interface. These classes are also known as Legacy classes. All legacy classes and interface were redesign by JDK 5 to support Generics. In general, the legacy classes are supported because there is still some code that uses them.

The following are the legacy classes defined by **java.util** package

- 1. Dictionary**
- 2. HashTable**
- 3. Properties**
- 4. Stack**
- 5. Vector**

There is only one legacy interface called **Enumeration**

```
import java.util.*;
class VectorExample{
    public static void main(String args[]){
        Vector<String> v = new Vector<String>();//creating vector
        v.add("ravi");//method of Collection
        v.addElement("lakshmi");//method of Vector
        v.addElement("ram");
        //traversing elements using Enumeration
        Enumeration e = v.elements();
        while(e.hasMoreElements()) {
            System.out.println(e.nextElement());
        }
    }
}
```

Java I/O

Java I/O (Input and Output) is used *to process the input and produce the output*.

Java uses the concept of a stream to make I/O operation fast. The java.io package contains all the classes required for input and output operations.

We can perform **file handling in Java** by Java I/O API.

Stream

A stream is a sequence of data. In Java, a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow.

In Java, 3 streams are created for us automatically. All these streams are attached with the console.

1) System.out: standard output stream

2) System.in: standard input stream

3) System.err: standard error stream

Let's see the code to print **output and an error** message to the console.

```
System.out.println("simple message");  
System.err.println("error message");
```

Let's see the code to get **input** from console.

```
int i=System.in.read();//returns ASCII code of 1st character  
System.out.println((char)i);//will print the character
```

OutputStream vs InputStream

The explanation of OutputStream and InputStream classes are given below:

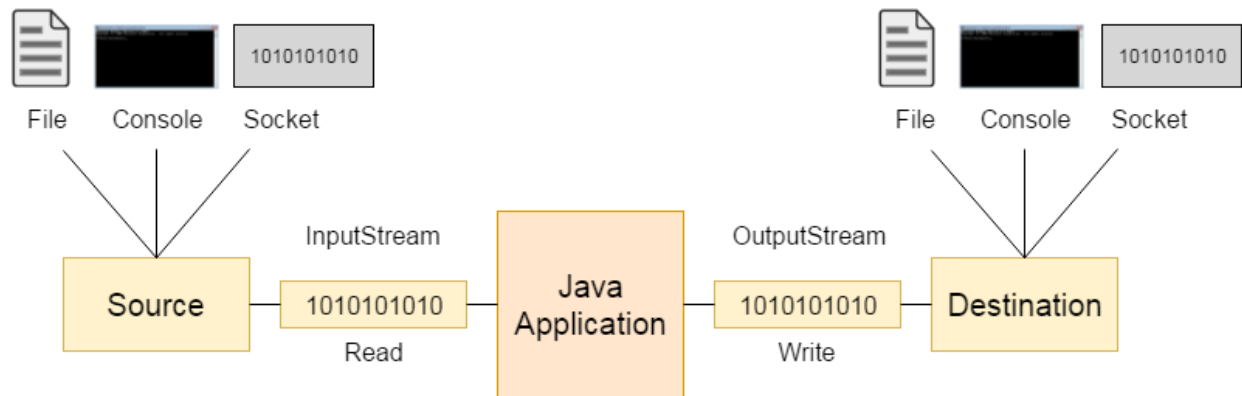
OutputStream

Java application uses an output stream to write data to a destination; it may be a file, an array, peripheral device or socket.

InputStream

Java application uses an input stream to read data from a source; it may be a file, an array, peripheral device or socket.

Let's understand the working of Java OutputStream and InputStream by the figure given below.



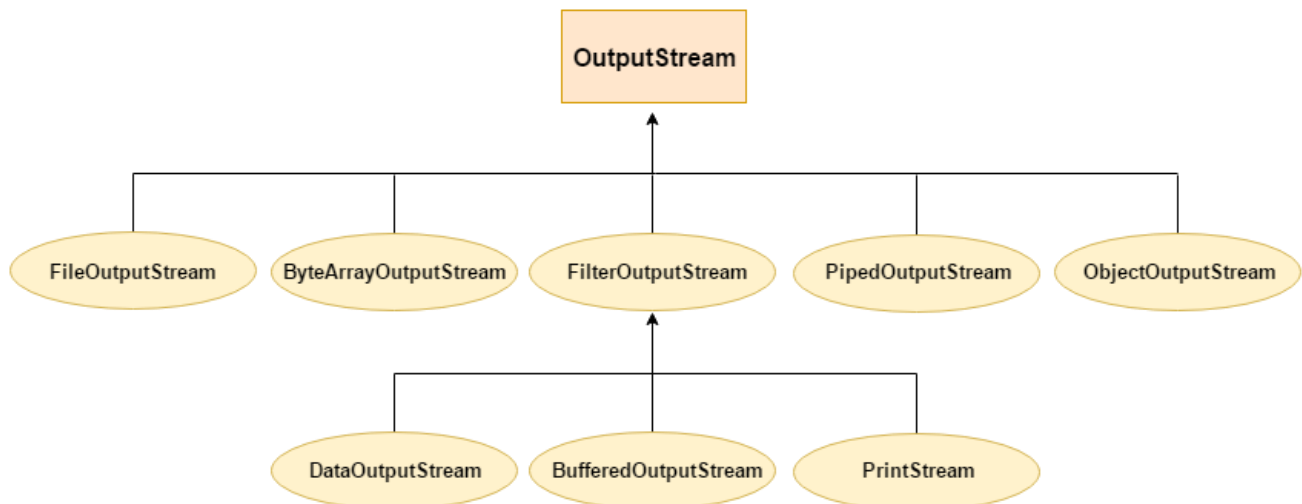
OutputStream class

OutputStream class is an abstract class. It is the superclass of all classes representing an output stream of bytes. An output stream accepts output bytes and sends them to some sink.

Useful methods of OutputStream

Method	Description
1) public void write(int)throws IOException	is used to write a byte to the current output stream.
2) public void write(byte[])throws IOException	is used to write an array of byte to the current output stream.
3) public void flush()throws IOException	flushes the current output stream.
4) public void close()throws IOException	is used to close the current output stream.

OutputStream Hierarchy



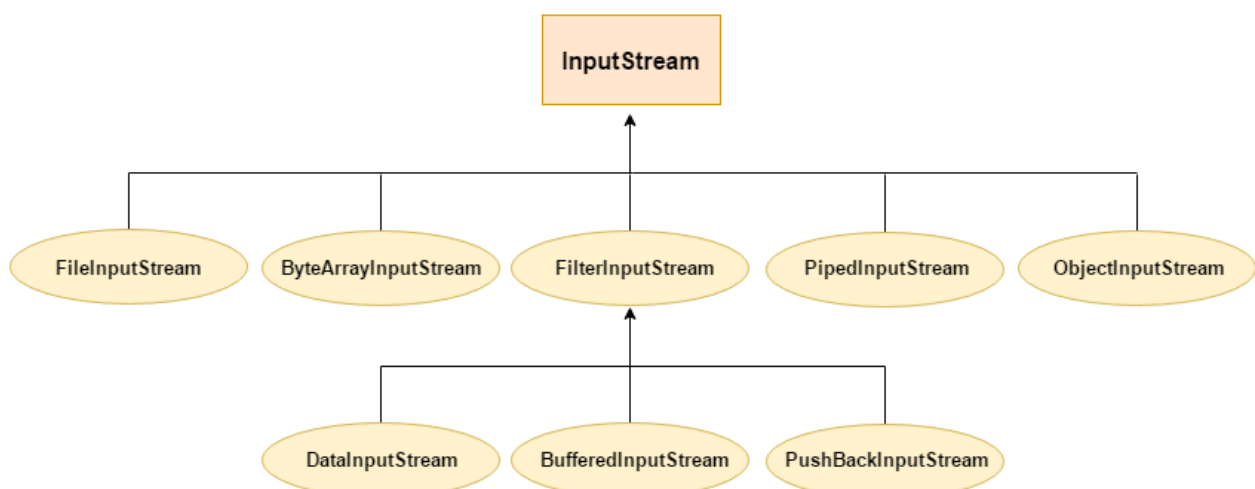
InputStream class

InputStream class is an abstract class. It is the superclass of all classes representing an input stream of bytes.

Useful methods of InputStream

Method	Description
1) public abstract int read()throws IOException	reads the next byte of data from the input stream. It returns -1 at the end of the file.
2) public int available()throws IOException	returns an estimate of the number of bytes that can be read from the current input stream.
3) public void close()throws IOException	is used to close the current input stream.

InputStream Hierarchy



FileOutputStream Class

FileOutputStream is an output stream used for writing data to a [file](#).

If you have to write primitive values into a file, use FileOutputStream class. You can write byte-oriented as well as character-oriented data through FileOutputStream class. But, for character-oriented data, it is preferred to use [FileWriter](#) than FileOutputStream.

```
public class FileOutputStream extends OutputStream
```

FileOutputStream class methods

Method	Description
protected void finalize()	It is used to clean up the connection with the file output stream.
void write(byte[] ary)	It is used to write ary.length bytes from the byte array to the file output stream.
void write(byte[] ary, int off, int len)	It is used to write len bytes from the byte array starting at offset off to the file output stream.
void write(int b)	It is used to write the specified byte to the file output stream.
FileChannel getChannel()	It is used to return the file channel object associated with the file output stream.
FileDescriptor getFD()	It is used to return the file descriptor associated with the stream.
void close()	It is used to closes the file output stream.

Example

```
import java.io.FileOutputStream;
public class FileOutputStreamExample
{
    public static void main(String args[])
    {
        try
        {
            FileOutputStream fout=new FileOutputStream("D:\\testout.txt");
            String s = "Welcome to java files.";
            byte b[] = s.getBytes();//converting string into byte array
            fout.write(b);
        }
    }
}
```

```

        fout.close();
        System.out.println("success...");
    }
    catch(Exception e)
    {
        System.out.println(e);
    }
}
}

```

Output:

```
Success...
```

The content of a text file **testout.txt** is set with the data **Welcome to javaTpoint.**

testout.txt

```
Welcome to java files.
```

FileInputStream Class

FileInputStream class obtains input bytes from a [file](#). It is used for reading byte-oriented data (streams of raw bytes) such as image data, audio, video etc. You can also read character-stream data. But, for reading streams of characters, it is recommended to use [FileReader](#) class.

```
public class FileInputStream extends InputStream
```

FileInputStream class methods

Method	Description
int available()	It is used to return the estimated number of bytes that can be read from the input stream.
int read()	It is used to read the byte of data from the input stream.
int read(byte[] b)	It is used to read up to b.length bytes of data from the input stream.
int read(byte[] b, int off, int len)	It is used to read up to len bytes of data from the input stream.
long skip(long x)	It is used to skip over and discards x bytes of data from the input stream.
FileChannel getChannel()	It is used to return the unique FileChannel object associated with the file input stream.
FileDescriptor getFD()	It is used to return the FileDescriptor object.
protected void finalize()	It is used to ensure that the close method is call when there is no more reference to the file input stream.
void close()	It is used to closes the stream .

Example

```
import java.io.FileInputStream;
```

```
public class DataStreamExample
{
    public static void main(String args[])
    {
        try
        {
            FileInputStream fin = new FileInputStream("D:\\testout.txt");
            int i = 0;
            while((i = fin.read()) != -1)
            {
                System.out.print((char)i);
            }
            fin.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Output:

```
Welcome to java files.
```

Byte Streams

Java byte streams are used to perform input and output of 8-bit bytes. Though there are many classes related to byte streams but the most frequently used classes are, **FileInputStream** and **FileOutputStream**. Following is an example which makes use of these two classes to copy an input file into an output file –

Example

```
import java.io.*;
public class CopyFile
{
    public static void main(String args[]) throws IOException
    {
        FileInputStream in = null;
        FileOutputStream out = null;
        try
        {
            in = new FileInputStream("input.txt");
            out = new FileOutputStream("output.txt");
            int c;
            while ((c = in.read()) != -1)
            {
                out.write(c);
            }
        }
        finally
        {
            if (in != null)
            {
                in.close();
            }
        }
    }
}
```



```
        }
        if (out != null)
        {
            out.close();
        }
    }
}
```

Now let's have a file **input.txt** with the following content –

```
This is a test for copy file.
```

As a next step, compile the above program and execute it, which will result in creating an output.txt file with the same content as we have in input.txt. So let's put the above code in CopyFile.java file and do the following –

```
$javac CopyFile.java
```

```
$java CopyFile
```

Character Streams

Java **Byte** streams are used to perform input and output of 8-bit bytes, whereas Java **Character** streams are used to perform input and output for 16-bit Unicode. Though there are many classes related to character streams but the most frequently used classes are, **FileReader** and **FileWriter**. Though internally FileReader uses FileInputStream and FileWriter uses FileOutputStream but here the major difference is that FileReader reads two bytes at a time and FileWriter writes two bytes at a time.

We can re-write the above example, which makes the use of these two classes to copy an input file (having Unicode characters) into an output file –

Example

```
import java.io.*;
public class CopyFile
{
    public static void main(String args[]) throws IOException
    {
        FileReader in = null;
        FileWriter out = null;
        try
        {
            in = new FileReader("input.txt");
            out = new FileWriter("output.txt");
            int c;
            while ((c = in.read()) != -1)
            {
                out.write(c);
            }
        }
    }
}
```

```
        finally
        {
            if (in != null)
            {
                in.close();
            }
            if (out != null)
            {
                out.close();
            }
        }
    }
}
```

Now let's have a file **input.txt** with the following content –

This is test for copy file.

As a next step, compile the above program and execute it, which will result in creating an output.txt file with the same content as we have in input.txt. So let's put the above code in CopyFile.java file and do the following –

```
$javac CopyFile.java
```

```
$java CopyFile
```

File Class

The File class is an abstract representation of file and directory pathname. A pathname can be either absolute or relative.

The File class have several methods for working with directories and files such as creating new directories or files, deleting and renaming directories or files, listing the contents of a directory etc.

Constructor	Description
File(File parent, String child)	It creates a new File instance from a parent abstract pathname and a child pathname string.
File(String pathname)	It creates a new File instance by converting the given pathname string into an abstract pathname.
File(String parent, String child)	It creates a new File instance from a parent pathname string and a child pathname string.
File(URI uri)	It creates a new File instance by converting the given file: URI into an abstract pathname.

Methods

Modifier and Type	Method	Description
boolean	createNewFile()	It atomically creates a new, empty file named by this abstract pathname if and only if a file with this name does not yet exist.
boolean	canWrite()	It tests whether the application can modify the file denoted by this abstract pathname. String[]
boolean	canExecute()	It tests whether the application can execute the file denoted by this abstract pathname.
boolean	canRead()	It tests whether the application can read the file denoted by this abstract pathname.
boolean	isDirectory()	It tests whether the file denoted by this abstract pathname is a directory.
boolean	isFile()	It tests whether the file denoted by this abstract pathname is a normal file.
String	getName()	It returns the name of the file or directory denoted by this abstract pathname.
String	getParent()	It returns the pathname string of this abstract pathname's parent, or null if this pathname does not name a parent directory.
File[]	listFiles()	It returns an array of abstract pathnames denoting the files in the directory denoted by this abstract pathname
String[]	list(FilenameFilter filter)	It returns an array of strings naming the files and directories in the directory denoted by this abstract pathname that satisfy the specified filter.
boolean	mkdir()	It creates the directory named by this abstract pathname.

Example

```
import java.io.*;
public class FileDemo
{
    public static void main(String[] args)
    {
        try
        {
            File file = new File("javaFile123.txt");
            if (file.createNewFile())
            {
                System.out.println("New File is created!");
            }
        }
    }
}
```

```

        else
        {
            System.out.println("File already exists.");
        }
    }
    catch (IOException e)
    {
        e.printStackTrace();
    }
}

```

Output:

New File is created!

RandomAccessFile

This [class](#) is used for reading and writing to random access file. A random access file behaves like a large [array](#) of bytes. There is a cursor implied to the array called file [pointer](#), by moving the cursor we do the read write operations. If end-of-file is reached before the desired number of byte has been read than EOFException is [thrown](#). It is a type of IOException.

<u>Constructor</u>	Description
RandomAccessFile(File file, String mode)	Creates a random access file stream to read from, and optionally to write to, the file specified by the File argument.
RandomAccessFile(String name, String mode)	Creates a random access file stream to read from, and optionally to write to, a file with the specified name.

Methods

Modifier and Type	Method	Method
void	close()	It closes this random access file stream and releases any system resources associated with the stream.
int	readInt()	It reads a signed 32-bit integer from this file.
void	seek(long pos)	It sets the file-pointer offset, measured from the beginning of this file, at which the next read or write occurs.
void	writeDouble(double v)	It converts the double argument to a long using the doubleToLongBits method in class Double, and then writes that long value to the file as an eight-byte quantity, high byte first.

void	writeFloat(float v)	It converts the float argument to an int using the floatToIntBits method in class Float, and then writes that int value to the file as a four-byte quantity, high byte first.
void	write(int b)	It writes the specified byte to this file.
int	read()	It reads a byte of data from this file.
long	length()	It returns the length of this file.
void	seek(long pos)	It sets the file-pointer offset, measured from the beginning of this file, at which the next read or write occurs.

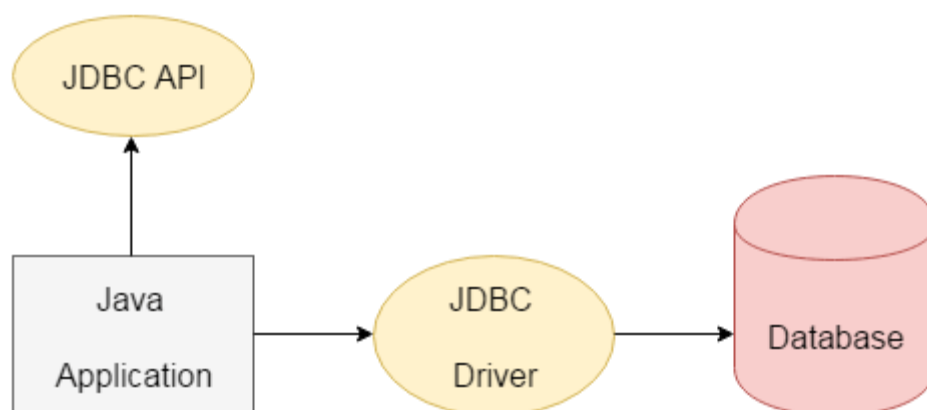
Java JDBC

JDBC stands for Java Database Connectivity. JDBC is a Java API to connect and execute the query with the database. It is a part of JavaSE (Java Standard Edition). JDBC API uses JDBC drivers to connect with the database. There are four types of JDBC drivers:

- JDBC-ODBC Bridge Driver,
- Native Driver,
- Network Protocol Driver, and
- Thin Driver

We have discussed the above four drivers in the next chapter.

We can use JDBC API to access tabular data stored in any relational database. By the help of JDBC API, we can save, update, delete and fetch data from the database. It is like Open Database Connectivity (ODBC) provided by Microsoft.



The current version of JDBC is 4.3. It is the stable release since 21st September, 2017. It is based on the X/Open SQL Call Level Interface. The **java.sql** package contains classes and interfaces for JDBC API. A list of popular *interfaces* of JDBC API are given below:

- Driver interface
- Connection interface
- Statement interface
- PreparedStatement interface
- CallableStatement interface
- ResultSet interface
- ResultSetMetaData interface
- DatabaseMetaData interface
- RowSet interface

A list of popular *classes* of JDBC API are given below:

- DriverManager class
- Blob class
- Clob class
- Types class

Why Should We Use JDBC?

Before JDBC, ODBC API was the database API to connect and execute the query with the database. But, ODBC API uses ODBC driver which is written in C language (i.e. platform dependent and unsecured). That is why Java has defined its own API (JDBC API) that uses JDBC drivers (written in Java language).

We can use JDBC API to handle database using Java program and can perform the following activities:

1. Connect to the database
2. Execute queries and update statements to the database
3. Retrieve the result received from the database.

What is API?

API (Application programming interface) is a document that contains a description of all the features of a product or software. It represents classes and interfaces that software programs can follow to communicate with each other. An API can be created for applications, libraries, operating systems, etc.

JDBC Driver

JDBC Driver is a software component that enables java application to interact with the database. There are 4 types of JDBC drivers:

1. JDBC-ODBC bridge driver
2. Native-API driver (partially java driver)
3. Network Protocol driver (fully java driver)
4. Thin driver (fully java driver)

JDBC-ODBC bridge driver

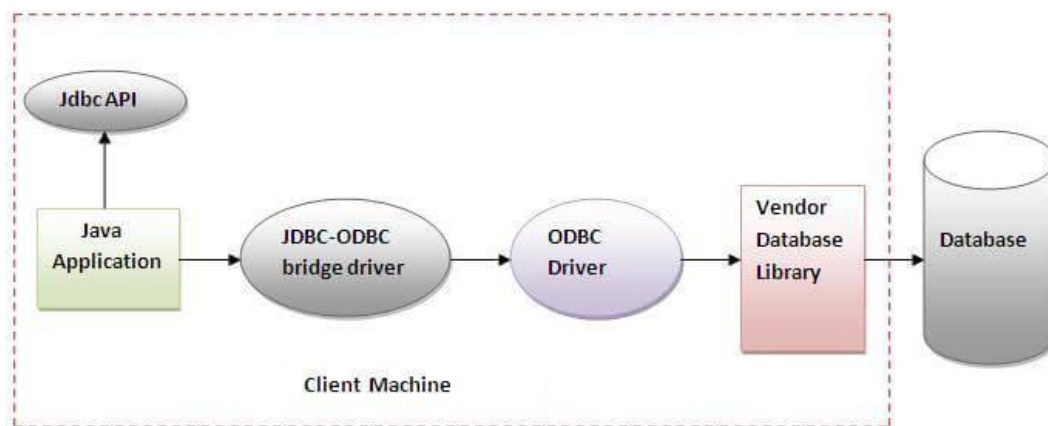


Figure- JDBC-ODBC Bridge Driver

The JDBC-ODBC bridge driver uses ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls. This is now discouraged because of thin driver.

Oracle does not support the JDBC-ODBC Bridge from Java 8. Oracle recommends that you use JDBC drivers provided by the vendor of your database instead of the JDBC-ODBC Bridge.

Advantages:

- easy to use.
- can be easily connected to any database.

Disadvantages:

- Performance degraded because JDBC method call is converted into the ODBC function calls.
- The ODBC driver needs to be installed on the client machine.

Native-API driver

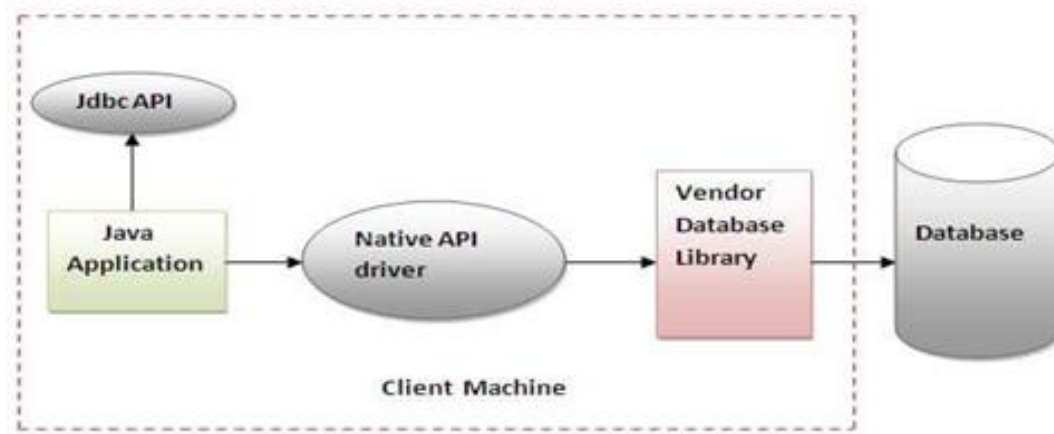


Figure- Native API Driver

The Native API driver uses the client-side libraries of the database. The driver converts JDBC method calls into native calls of the database API. It is not written entirely in java.

Advantage:

- performance upgraded than JDBC-ODBC bridge driver.

Disadvantage:

- The Native driver needs to be installed on each client machine.
- The Vendor client library needs to be installed on client machine.

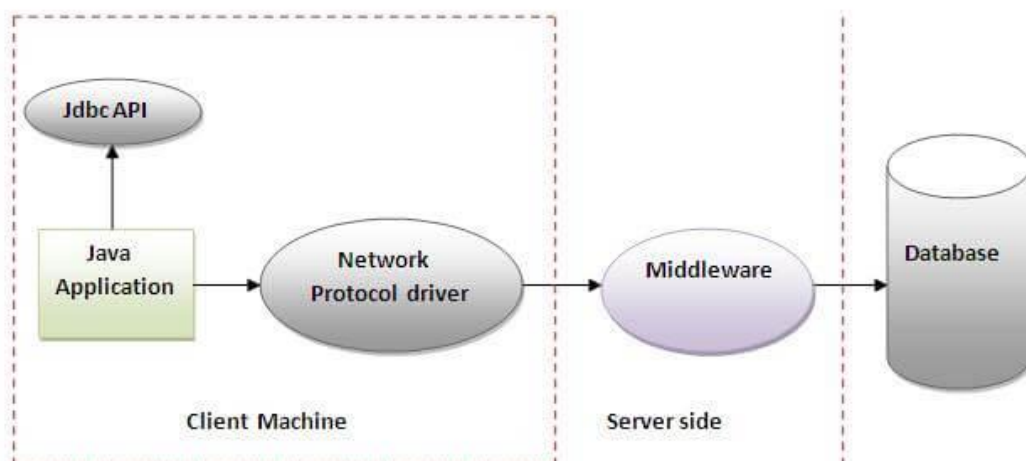
Network Protocol driver

Figure- Network Protocol Driver

The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. It is fully written in java.

Advantage:

- No client side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.

Disadvantages:

- Network support is required on client machine.
- Requires database-specific coding to be done in the middle tier.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

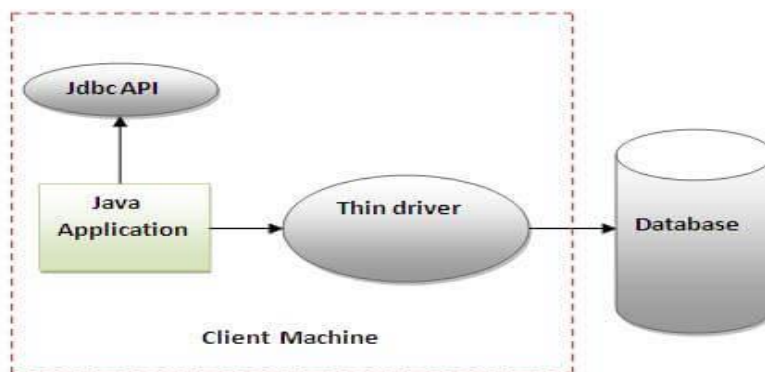
Thin driver

Figure- Thin Driver

The thin driver converts JDBC calls directly into the vendor-specific database protocol. That is why it is known as thin driver. It is fully written in Java language.

Advantage:

- Better performance than all other drivers.
- No software is required at client side or server side.

Disadvantage:

- Drivers depend on the Database.

Java Database Connectivity with 5 Steps

1. Register the Driver class
2. Create connection
3. Create statement
4. Execute queries and handle the results
5. Close connection

1) Register the driver class

The **forName()** method of Class class is used to register the driver class. This method is used to dynamically load the driver class.

Syntax:

```
public static void forName(String className)throws ClassNotFoundException
```

Programming Statement:

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

2) Create the connection object

The **getConnection()** method of DriverManager class is used to establish connection with the database.

Syntax:

- 1) **public static** Connection getConnection(String url)**throws** SQLException
- 2) **public static** Connection getConnection(String url,String name,String password)
throws SQLException

Programming Statement:

```
Connection con = DriverManager.getConnection( "jdbc:oracle:thin:@localhost:1521:xe",  
"system", "password");
```

3) Create the Statement object

The **createStatement()** method of Connection interface is used to create statement. The object of statement is responsible to execute queries with the database.

Syntax:

```
public Statement createStatement()throws SQLException
```

Programming Statement:

```
Statement stmt = con.createStatement();
```

4) Execute the query and Handle the results

The **executeQuery()** and **executeUpdate()** methods of Statement interface is used to execute queries to the database.

ResultSet executeQuery(String SQL): Returns a ResultSet object. Use this method when you expect to get a result set, as you would with a SELECT statement.

int executeUpdate(String SQL): Returns the number of rows affected by the execution of the SQL statement. Use this method to execute SQL statements, for which you expect to get a number of rows affected - for example, an INSERT, UPDATE, or DELETE statement.

Example

```
ResultSet rs=stmt.executeQuery("select * from emp");

while(rs.next())
{
    System.out.println(rs.getInt(1)+" "+rs.getString(2));
}
```

5) Close the connection object

By closing connection object statement and ResultSet will be closed automatically. The close() method of Connection interface is used to close the connection.

Syntax:

```
public void close()throws SQLException
```

Programming Statement:

```
con.close();
```

Java Database Connectivity with Oracle

To connect java application with the oracle database, we need to follow 5 following steps. In this example, we are using Oracle 10g as the database. So we need to know following information for the oracle database:

1. **Driver class:** The driver class for the oracle database is

```
oracle.jdbc.driver.OracleDriver.
```

2. **Connection URL:** The connection URL for the oracle10G database is:

```
jdbc:oracle:thin:@localhost:1521:xe
```

where jdbc is the API, oracle is the database, thin is the driver, localhost is the server name on which oracle is running, we may also use IP address, 1521 is the port number and XE is the Oracle service name. You may get all these information from the tnsnames.ora file.

3. **Username:** The default username for the oracle database is **system**.
4. **Password:** It is the password given by the user at the time of installing the oracle database.

Create a Table

Before establishing connection, let's first create a table in oracle database. Following is the SQL query to create a table.

```
create table emp(id number(10),name varchar2(40),age number(3));
```

JDBC program to insert the values in to Database.

```
import java.sql.*;
class JDBCInsert
{
    public static void main(String args[])
    {
        try
        {
            //step1 load the driver class
            Class.forName("oracle.jdbc.driver.OracleDriver");

            //step2 create the connection object
            Connection con = DriverManager.getConnection
                ("jdbc:oracle:thin:@localhost:1521:xe", "system", "oracle");

            //step3 create the statement object
            Statement stmt = con.createStatement();

            //step4 execute query
            int status = stmt.executeUpdate ("insert into emp values (1001, 'Ram',
                50000)");
            if(status > 0)
                System.out.println("Values inserted successfully");

            //step5 close the connection object
            con.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

JDBC program to retrieve the values from the Database.

```
import java.sql.*;
class JDBCRetrieve
{
    public static void main(String args[])
    {
        try
        {
            //step1 load the driver class
            Class.forName("oracle.jdbc.driver.OracleDriver");

            //step2 create the connection object
            Connection con = DriverManager.getConnection
                ("jdbc:oracle:thin:@localhost:1521:xe", "system", "oracle");

            //step3 create the statement object
            Statement stmt = con.createStatement();

            //step4 execute query
            ResultSet rs = stmt.executeQuery("select * from emp");
```

```
        while(rs.next())
            System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3));

        //step5 close the connection object
        con.close();
    }
    catch(Exception e)
    {
        System.out.println(e);
    }
}
```

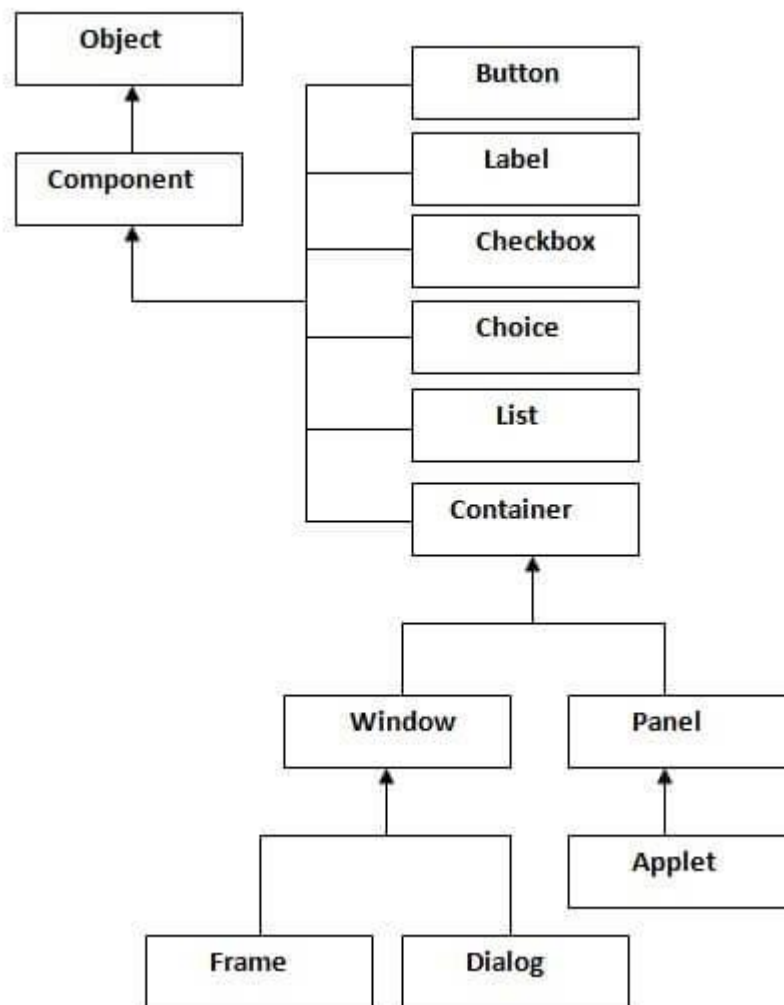
Java AWT

Java AWT (Abstract Window Toolkit) is an API to develop GUI or window-based applications in java.

Java AWT components are platform-dependent i.e. components are displayed according to the view of operating system. AWT is heavyweight i.e. its components are using the resources of OS.

The java.awt package provides classes for AWT api such as TextField, Label, TextArea, Choice, RadioButton, CheckBox, List etc.

AWT Hierarchy



Container

The Container is a component in AWT that can contain other components like [buttons](#), textfields, labels etc. The classes that extend Container class are known as container such as Frame, Dialog and Panel.

Window

The window is the container that has no borders and menu bars. You must use frame, dialog or another window for creating a window.

Panel

The Panel is the container that doesn't contain title bar and menu bars. It can have other components like button, textfield etc.

Frame

The Frame is the container that contain title bar and can have menu bars. It can have other components like button, textfield etc.

Useful Methods of Component class

Method	Description
public void add(Component c)	inserts a component on this component.
public void setSize(int width,int height)	sets the size (width and height) of the component.
public void setLayout(LayoutManager m)	defines the layout manager for the component.
public void setVisible(boolean status)	changes the visibility of the component, by default false.

AWT Example

To create simple awt example, you need a frame. There are two ways to create a frame in AWT.

- By extending Frame class (inheritance)
- By creating the object of Frame class (association)

AWT Example by Inheritance

```
import java.awt.*;
class First extends Frame
{
    First()
    {
        Button b = new Button("click me");
        b.setBounds(30, 100, 80, 30); // setting button position
        add(b); // adding button into frame
        setSize(300, 300); // frame size 300 width and 300 height
        setLayout(null); // no layout manager
        setVisible(true); // now frame will be visible, by default not visible
    }
    public static void main(String args[])
    {
        First f = new First();
    }
}
```

The `setBounds(int xaxis, int yaxis, int width, int height)` method is used in the above example that sets the position of the awt button.



AWT Example by Association

```
import java.awt.*;
class First
{
    First()
    {
        Frame f = new Frame();
        Button b = new Button("click me");
        b.setBounds(30, 50, 80, 30);
        f.add(b);
        f.setSize(300, 300);
        f.setLayout(null);
        f.setVisible(true);
    }
    public static void main(String args[])
    {
        First f = new First();
    }
}
```



Event Handling

Changing the state of an object is known as an event. For example, click on button, dragging mouse etc. The java.awt.event package provides many event classes and Listener interfaces for event handling.

Java Event classes and Listener interfaces

Event Classes	Listener Interfaces
ActionEvent	ActionListener
MouseEvent	MouseListener and MouseMotionListener
MouseWheelEvent	MouseWheelListener
KeyEvent	KeyListener
ItemEvent	ItemListener
TextEvent	TextListener
AdjustmentEvent	AdjustmentListener
WindowEvent	WindowListener
ComponentEvent	ComponentListener
ContainerEvent	ContainerListener
FocusEvent	FocusListener

Registration Methods

For registering the component with the Listener, many classes provide the registration methods. For example:

- **Button**
 - `public void addActionListener(ActionListener a){}`
- **MenuItem**
 - `public void addActionListener(ActionListener a){}`
- **TextField**
 - `public void addActionListener(ActionListener a){}`
 - `public void addTextListener(TextListener a){}`
- **TextArea**

- public void addTextListener(TextListener a){}
- **Checkbox**
 - public void addItemListener(ItemListener a){}
- **Choice**
 - public void addItemListener(ItemListener a){}
- **List**
 - public void addActionListener(ActionListener a){}
 - public void addItemListener(ItemListener a){}

Event Handling Code

We can put the event handling code into one of the following places:

1. Within class
2. Other class
3. Anonymous class

Java event handling by implementing ActionListener

```
import java.awt.*;
import java.awt.event.*;
class AEvent extends Frame implements ActionListener
{
    TextField tf;
    AEvent()
    {
        //create components
        tf = new TextField();
        tf.setBounds(60, 50, 170, 20);
        Button b = new Button("click me");
        b.setBounds(100, 120, 80, 30);

        //register listener
        b.addActionListener(this); //passing current instance

        //add components and set size, layout and visibility
        add(b); add(tf);
        setSize(300, 300);
        setLayout(null);
        setVisible(true);
    }
    public void actionPerformed(ActionEvent e)
    {
        tf.setText("Welcome");
    }
    public static void main(String args[])
    {
        new AEvent();
    }
}
```

Output:



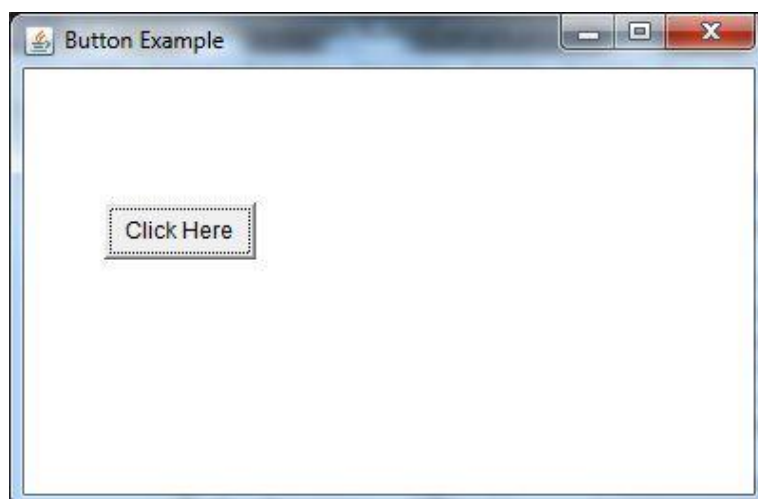
AWT Button

The button class is used to create a labeled button that has platform independent implementation. The application result in some action when the button is pushed.

public class Button **extends** Component **implements** Accessible

Example

```
import java.awt.*;  
public class ButtonExample  
{  
    public static void main(String[] args)  
    {  
        Frame f = new Frame("Button Example");  
        Button b = new Button("Click Here");  
        b.setBounds(50, 100, 80, 30);  
        f.add(b);  
        f.setSize(400, 400);  
        f.setLayout(null);  
        f.setVisible(true);  
    }  
}
```



MouseListener Interface

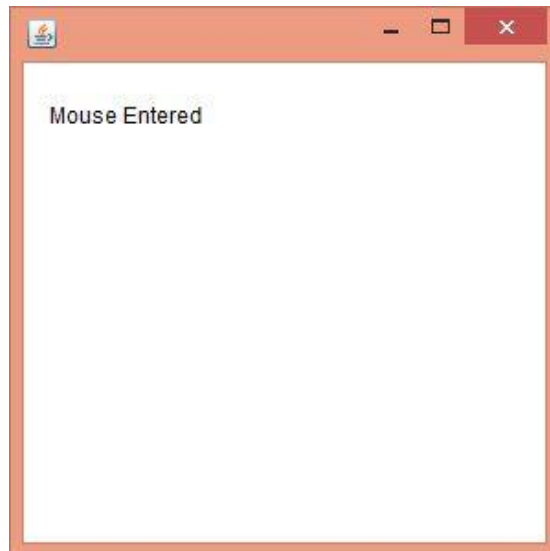
The Java MouseListener is notified whenever you change the state of mouse. It is notified against MouseEvent. The MouseListener interface is found in java.awt.event package. It has five methods.

The signature of 5 methods found in MouseListener interface are given below:

```
public abstract void mouseClicked(MouseEvent e);  
public abstract void mouseEntered(MouseEvent e);  
public abstract void mouseExited(MouseEvent e);  
public abstract void mousePressed(MouseEvent e);  
public abstract void mouseReleased(MouseEvent e);
```

MouseListener Example

```
import java.awt.*;  
import java.awt.event.*;  
public class MouseListenerExample extends Frame implements MouseListener  
{  
    Label l;  
    MouseListenerExample()  
    {  
        addMouseListener(this);  
        l = new Label();  
        l.setBounds(20, 50, 100, 20);  
        add(l);  
        setSize(300, 300);  
        setLayout(null);  
        setVisible(true);  
    }  
    public void mouseClicked(MouseEvent e)  
    {  
        l.setText("Mouse Clicked");  
    }  
    public void mouseEntered(MouseEvent e)  
    {  
        l.setText("Mouse Entered");  
    }  
    public void mouseExited(MouseEvent e)  
    {  
        l.setText("Mouse Exited");  
    }  
    public void mousePressed(MouseEvent e)  
    {  
        l.setText("Mouse Pressed");  
    }  
    public void mouseReleased(MouseEvent e)  
    {  
        l.setText("Mouse Released");  
    }  
    public static void main(String[] args)  
    {  
        new MouseListenerExample();  
    }  
}
```

Output:

MouseMotionListener Interface

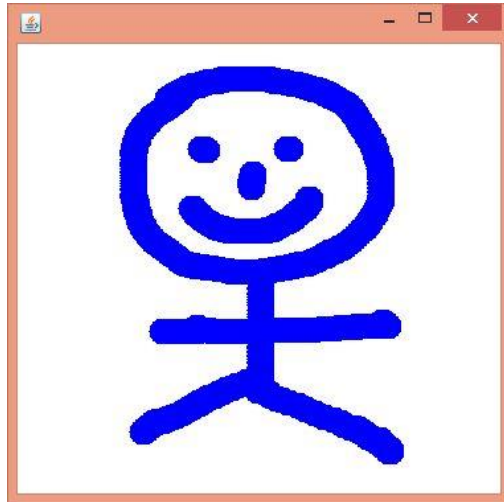
The Java MouseMotionListener is notified whenever you move or drag mouse. It is notified against MouseEvent. The MouseMotionListener interface is found in java.awt.event package. It has two methods.

```
public abstract void mouseDragged(MouseEvent e);
```

```
public abstract void mouseMoved(MouseEvent e);
```

MouseMotionListener Example

```
import java.awt.*;
import java.awt.event.*;
public class MouseMotionListenerExample extends Frame implements MouseMotionListener
{
    MouseMotionListenerExample()
    {
        addMouseMotionListener(this);
        setSize(300, 300);
        setLayout(null);
        setVisible(true);
    }
    public void mouseDragged(MouseEvent e)
    {
        Graphics g = getGraphics();
        g.setColor(Color.BLUE);
        g.fillOval(e.getX(), e.getY(), 20, 20);
    }
    public void mouseMoved(MouseEvent e)
    {}
    public static void main(String[] args)
    {
        new MouseMotionListenerExample();
    }
}
```

Output:

Adapter Classes

Java adapter classes provide the default implementation of listener [interfaces](#). If you inherit the adapter class, you will not be forced to provide the implementation of all the methods of listener interfaces. So it saves code.

The adapter classes are found in `java.awt.event`, `java.awt.dnd` and `javax.swing.event` [packages](#). The Adapter classes with their corresponding listener interfaces are given below.

java.awt.event Adapter classes

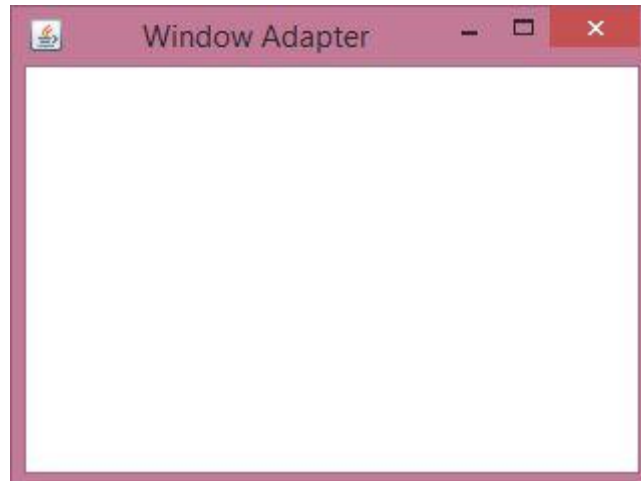
Adapter class	Listener interface
WindowAdapter	WindowListener
KeyAdapter	KeyListener
MouseAdapter	MouseListener
MouseMotionAdapter	MouseMotionListener
FocusAdapter	FocusListener
ComponentAdapter	ComponentListener
ContainerAdapter	ContainerListener
HierarchyBoundsAdapter	HierarchyBoundsListener

WindowAdapter Example

```
import java.awt.*;
import java.awt.event.*;
public class AdapterExample
{
    Frame f;
    AdapterExample()
    {
        f = new Frame("Window Adapter");
        f.addWindowListener(new WindowAdapter()
        {
            public void windowClosing(WindowEvent e)
            {
                f.dispose();
            }
        });

        f.setSize(400, 400);
        f.setLayout(null);
        f.setVisible(true);
    }
    public static void main(String[] args)
    {
        new AdapterExample();
    }
}
```

Output:

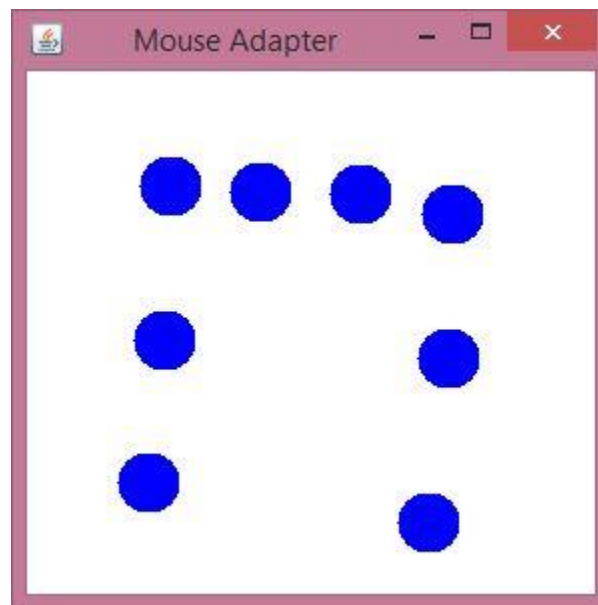


MouseAdapter Example

```
import java.awt.*;
import java.awt.event.*;
public class MouseAdapterExample extends MouseAdapter
{
    Frame f;
    MouseAdapterExample()
    {
        f = new Frame("Mouse Adapter");
        f.addMouseListener(this);
        f.setSize(300, 300);
    }
}
```

```
        f.setLayout(null);  
        f.setVisible(true);  
    }  
public void mouseClicked(MouseEvent e)  
{  
    Graphics g = f.getGraphics();  
    g.setColor(Color.BLUE);  
    g.fillOval(e.getX(), e.getY(), 30, 30);  
}  
public static void main(String[] args)  
{  
    new MouseAdapterExample();  
}  
}
```

Output:



Java Swing

Java Swing is a part of Java Foundation Classes (JFC) that is *used to create window-based applications*. It is built on the top of AWT (Abstract Windowing Toolkit) API and entirely written in java.

Unlike AWT, Java Swing provides platform-independent and lightweight components.

The javax.swing package provides classes for java swing API such as JButton, JTextField, JTextArea, JRadioButton, JCheckbox, JMenu, JColorChooser etc.

Difference between AWT and Swing

No.	Java AWT	Java Swing
1)	AWT components are platform-dependent .	Java swing components are platform-independent .
2)	AWT components are heavyweight .	Swing components are lightweight .
3)	AWT doesn't support pluggable look and feel .	Swing supports pluggable look and feel .
4)	AWT provides less components than Swing.	Swing provides more powerful componentss uch as tables, lists, scrollpanes, colorchooser, tabbedpane etc.
5)	AWT doesn't follows MVC (Model View Controller) where model represents data, view represents presentation and controller acts as an interface between model and view.	Swing follows MVC .

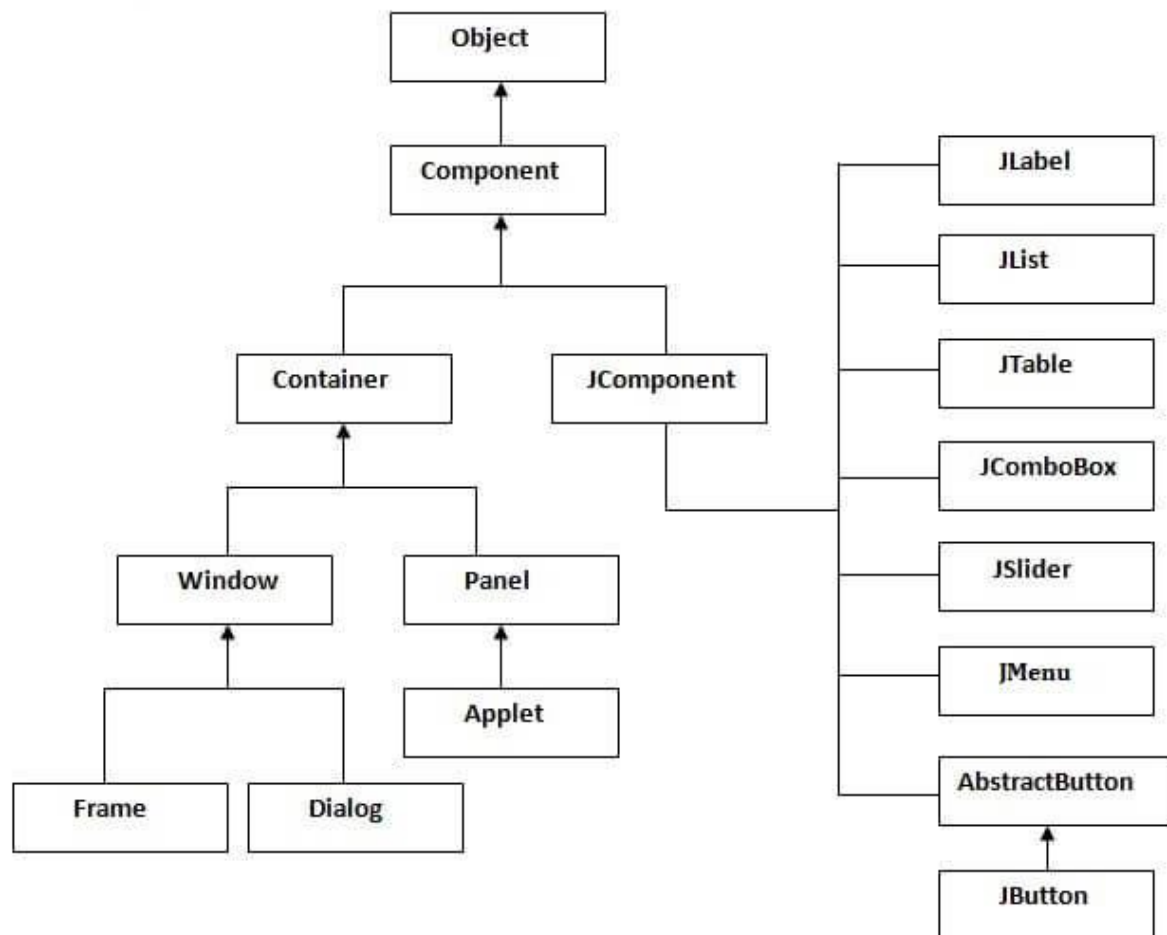
What is JFC?

The Java Foundation Classes (JFC) are a set of GUI components which simplify the development of desktop applications.

Commonly used Methods of Component class in Java Swing

Method	Description
public void add(Component c)	add a component on another component.
public void setSize(int width,int height)	sets size of the component.
public void setLayout(LayoutManager m)	sets the layout manager for the component.
public void setVisible(boolean b)	sets the visibility of the component. It is by default false.

Hierarchy of Java Swing classes



Example

```
import javax.swing.*;
public class Simple
{
    JFrame f;
    Simple()
    {
        f = new JFrame();//creating instance of JFrame
        JButton b = new JButton("click");//creating instance of JButton
        b.setBounds(130, 100, 100, 40);
        f.add(b);//adding button in JFrame
        f.setSize(400, 500);//400 width and 500 height
        f.setLayout(null);//using no layout managers
        f.setVisible(true);//making the frame visible
    }
    public static void main(String[] args)
    {
        new Simple();
    }
}
```

JTextArea

The object of a JTextArea class is a multi line region that displays text. It allows the editing of multiple line text. It inherits JTextComponent class

public class JTextArea **extends** JTextComponent

Commonly used Constructors:

Constructor	Description
JTextArea()	Creates a text area that displays no text initially.
JTextArea(String s)	Creates a text area that displays specified text initially.
JTextArea(int row, int column)	Creates a text area with the specified number of rows and columns that displays no text initially.
JTextArea(String s, int row, int column)	Creates a text area with the specified number of rows and columns that displays specified text.

Commonly used Methods:

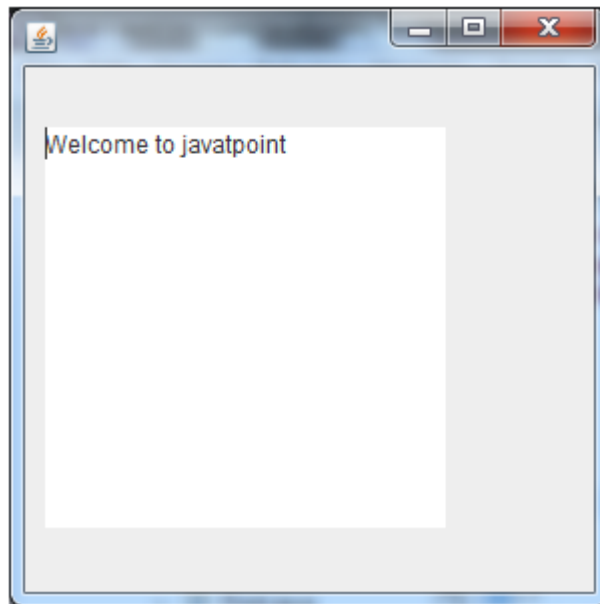
Methods	Description
void setRows(int rows)	It is used to set specified number of rows.
void setColumns(int cols)	It is used to set specified number of columns.
void setFont(Font f)	It is used to set the specified font.
void insert(String s, int position)	It is used to insert the specified text on the specified position.
void append(String s)	It is used to append the given text to the end of the document.

Example

```
import javax.swing.*;
public class TextAreaExample
{
    TextAreaExample()
    {
        JFrame f = new JFrame();
        JTextArea area = new JTextArea("Welcome to javatpoint");
        area.setBounds(10, 30, 200, 200);
        f.add(area);
        f.setSize(300, 300);
        f.setLayout(null);
        f.setVisible(true);
    }
}
```

```
public static void main(String args[])  
{  
    new TextAreaExample();  
}
```

Output:



JDialog

The JDialog control represents a top level window with a border and a title used to take some form of input from the user. It inherits the Dialog class.

Unlike JFrame, it doesn't have maximize and minimize buttons.

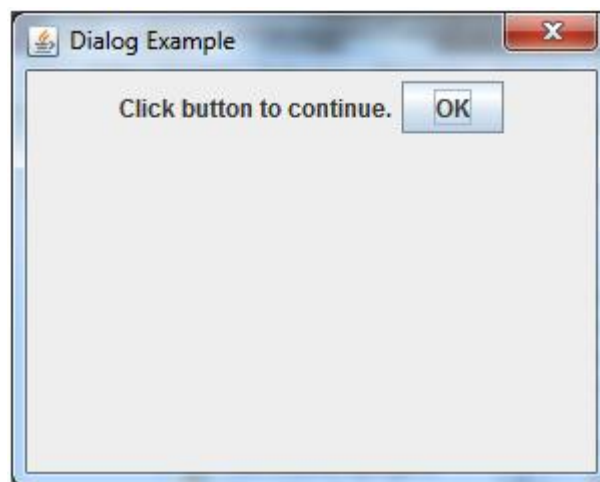
public class JDialog **extends** Dialog **implements** WindowConstants, Accessible, RootPaneContainer

Commonly used Constructors:

Constructor	Description
JDialog()	It is used to create a modeless dialog without a title and without a specified Frame owner.
JDialog(Frame owner)	It is used to create a modeless dialog with specified Frame as its owner and an empty title.
JDialog(Frame owner, String title, boolean modal)	It is used to create a dialog with the specified title, owner Frame and modality.

Example

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class DialogExample
{
    private static JDialog d;
    DialogExample()
    {
        JFrame f = new JFrame();
        d = new JDialog(f, "Dialog Example", true);
        d.setLayout( new FlowLayout() );
        JButton b = new JButton ("OK");
        b.addActionListener ( new ActionListener()
        {
            public void actionPerformed((ActionEvent e) )
            {
                DialogExample.d.setVisible(false);
            }
        });
        d.add( new JLabel ("Click button to continue."));
        d.add(b);
        d.setSize(300, 300);
        d.setVisible(true);
    }
    public static void main(String args[])
    {
        new DialogExample();
    }
}
```

Output:

LayoutManagers

The LayoutManagers are used to arrange components in a particular manner. LayoutManager is an interface that is implemented by all the classes of layout managers. There are following classes that represents the layout managers:

1. java.awt.BorderLayout
2. java.awt.FlowLayout
3. java.awt.GridLayout
4. java.awt.CardLayout
5. java.awt.GridBagLayout
6. javax.swing.BoxLayout
7. javax.swing.GroupLayout
8. javax.swing.ScrollPaneLayout
9. javax.swing.SpringLayout etc.

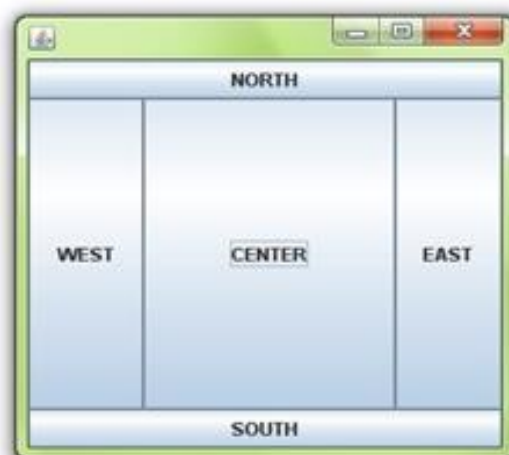
BorderLayout

The BorderLayout is used to arrange the components in five regions: north, south, east, west and center. Each region (area) may contain one component only. It is the default layout of frame or window. The BorderLayout provides five constants for each region:

1. **public static final int NORTH**
2. **public static final int SOUTH**
3. **public static final int EAST**
4. **public static final int WEST**
5. **public static final int CENTER**

Constructors of BorderLayout class:

- **BorderLayout():** creates a border layout but with no gaps between the components.
- **JBorderLayout(int hgap, int vgap):** creates a border layout with the given horizontal and vertical gaps between the components.



Example:

```
import java.awt.*;
import javax.swing.*;

public class Border
{
    JFrame f;
    Border()
    {
        f=new JFrame();

        JButton b1 = new JButton("NORTH");
        JButton b2 = new JButton("SOUTH");
        JButton b3 = new JButton("EAST");
        JButton b4 = new JButton("WEST");
        JButton b5 = new JButton("CENTER");

        f.add(b1, BorderLayout.NORTH);
        f.add(b2, BorderLayout.SOUTH);
        f.add(b3, BorderLayout.EAST);
        f.add(b4, BorderLayout.WEST);
        f.add(b5, BorderLayout.CENTER);

        f.setSize(300, 300);
        f.setVisible(true);
    }
    public static void main(String[] args)
    {
        new Border();
    }
}
```

GridLayout

The GridLayout is used to arrange the components in rectangular grid. One component is displayed in each rectangle.

Constructors of GridLayout class

1. **GridLayout():** creates a grid layout with one column per component in a row.
2. **GridLayout(int rows, int columns):** creates a grid layout with the given rows and columns but no gaps between the components.
3. **GridLayout(int rows, int columns, int hgap, int vgap):** creates a grid layout with the given rows and columns alongwith given horizontal and vertical gaps.



Example:

```
import java.awt.*;
import javax.swing.*;

public class MyGridLayout
{
    JFrame f;
    MyGridLayout()
    {
        f = new JFrame();

        JButton b1 = new JButton("1");
        JButton b2 = new JButton("2");
        JButton b3 = new JButton("3");
        JButton b4 = new JButton("4");
        JButton b5 = new JButton("5");
        JButton b6 = new JButton("6");
        JButton b7 = new JButton("7");
        JButton b8 = new JButton("8");
        JButton b9 = new JButton("9");

        f.add(b1); f.add(b2); f.add(b3); f.add(b4); f.add(b5);
        f.add(b6); f.add(b7); f.add(b8); f.add(b9);

        f.setLayout(new GridLayout(3, 3));
        //setting grid layout of 3 rows and 3 columns

        f.setSize(300, 300);
        f.setVisible(true);
    }
    public static void main(String[] args)
    {
        new MyGridLayout();
    }
}
```

FlowLayout

The FlowLayout is used to arrange the components in a line, one after another (in a flow). It is the default layout of applet or panel.

Fields of FlowLayout class

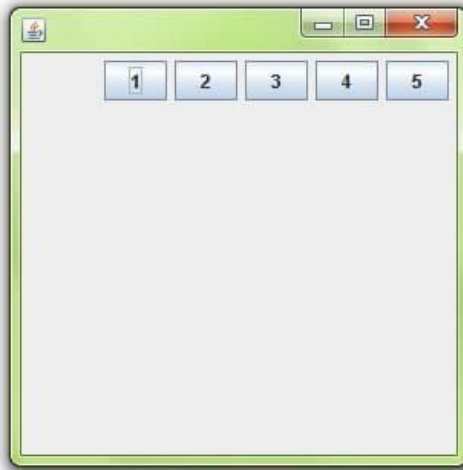
1. **public static final int LEFT**
2. **public static final int RIGHT**
3. **public static final int CENTER**
4. **public static final int LEADING**
5. **public static final int TRAILING**

Constructors of FlowLayout class

1. **FlowLayout():** creates a flow layout with centered alignment and a default 5 unit horizontal and vertical gap.
2. **FlowLayout(int align):** creates a flow layout with the given alignment and a default 5 unit horizontal and vertical gap.

3. **FlowLayout(int align, int hgap, int vgap):** creates a flow layout with the given alignment and the given horizontal and vertical gap.

Example:



```
import java.awt.*;
import javax.swing.*;

public class MyFlowLayout
{
    JFrame f;
    MyFlowLayout()
    {
        f = new JFrame();

        JButton b1 = new JButton("1");
        JButton b2 = new JButton("2");
        JButton b3 = new JButton("3");
        JButton b4 = new JButton("4");
        JButton b5 = new JButton("5");

        f.add(b1); f.add(b2); f.add(b3); f.add(b4); f.add(b5);

        f.setLayout(new FlowLayout(FlowLayout.RIGHT));
        //setting flow layout of right alignment

        f.setSize(300, 300);
        f.setVisible(true);
    }
    public static void main(String[] args)
    {
        new MyFlowLayout();
    }
}
```

BoxLayout

The BoxLayout is used to arrange the components either vertically or horizontally. For this purpose, BoxLayout provides four constants. They are as follows:

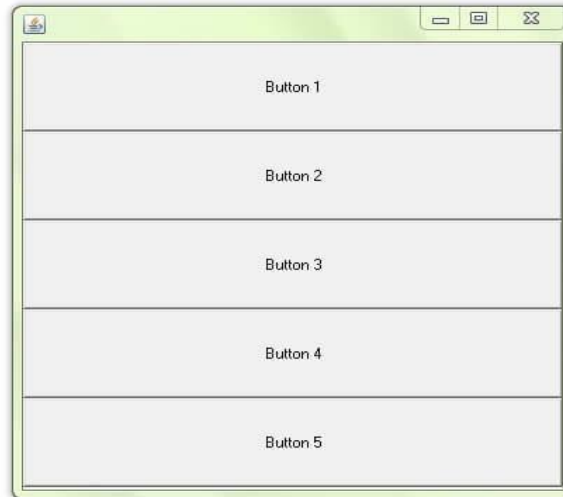
Fields of BorderLayout class

1. **public static final int X_AXIS**
2. **public static final int Y_AXIS**
3. **public static final int LINE_AXIS**
4. **public static final int PAGE_AXIS**

Constructor of BorderLayout class

1. **BoxLayout(Container c, int axis):** creates a box layout that arranges the components with the given axis.

Example:



```
import java.awt.*;
import javax.swing.*;

public class BoxLayoutExample extends Frame
{
    Button buttons[];

    public BoxLayoutExample1 ()
    {
        buttons = new Button [5];

        for (int i = 0; i<5; i++)
        {
            buttons[i] = new Button ("Button " + (i + 1));
            add (buttons[i]);
        }

        setLayout (new BoxLayout (this, BoxLayout.Y_AXIS));
        setSize(400, 400);
        setVisible(true);
    }

    public static void main(String args[])
    {
        BoxLayoutExample b = new BoxLayoutExample ();
    }
}
```

CardLayout

The CardLayout class manages the components in such a manner that only one component is visible at a time. It treats each component as a card that is why it is known as CardLayout.

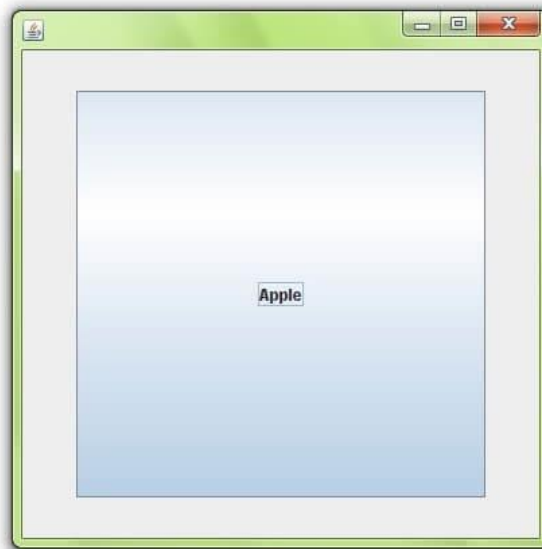
Constructors of CardLayout class

1. **CardLayout():** creates a card layout with zero horizontal and vertical gap.
2. **CardLayout(int hgap, int vgap):** creates a card layout with the given horizontal and vertical gap.

Commonly used methods of CardLayout class

- **public void next(Container parent):** is used to flip to the next card of the given container.
- **public void previous(Container parent):** is used to flip to the previous card of the given container.
- **public void first(Container parent):** is used to flip to the first card of the given container.
- **public void last(Container parent):** is used to flip to the last card of the given container.
- **public void show(Container parent, String name):** is used to flip to the specified card with the given name.

Example:



```
import java.awt.*;
import java.awt.event.*;

import javax.swing.*;

public class CardLayoutExample extends JFrame implements ActionListener
{
    CardLayout card;
    JButton b1,b2,b3;
    Container c;
    CardLayoutExample()
    {
```

```

        c = getContentPane();
        card = new CardLayout(40, 30);
        //create CardLayout object with 40 hor space and 30 ver space
        c.setLayout(card);

        b1 = new JButton("Apple");
        b2 = new JButton("Boy");
        b3 = new JButton("Cat");
        b1.addActionListener(this);
        b2.addActionListener(this);
        b3.addActionListener(this);

        c.add("a", b1); c.add("b", b2); c.add("c", b3);

    }
    public void actionPerformed(ActionEvent e)
    {
        card.next(c);
    }

    public static void main(String[] args)
    {
        CardLayoutExample cl = new CardLayoutExample();
        cl.setSize(400, 400);
        cl.setVisible(true);
        cl.setDefaultCloseOperation(EXIT_ON_CLOSE);
    }
}

```

GridBagLayout

The Java GridBagLayout class is used to align components vertically, horizontally or along their baseline.

The components may not be of same size. Each GridBagLayout object maintains a dynamic, rectangular grid of cells. Each component occupies one or more cells known as its display area. Each component associates an instance of GridBagConstraints. With the help of constraints object we arrange component's display area on the grid. The GridBagLayout manages each component's minimum and preferred sizes in order to determine component's size.

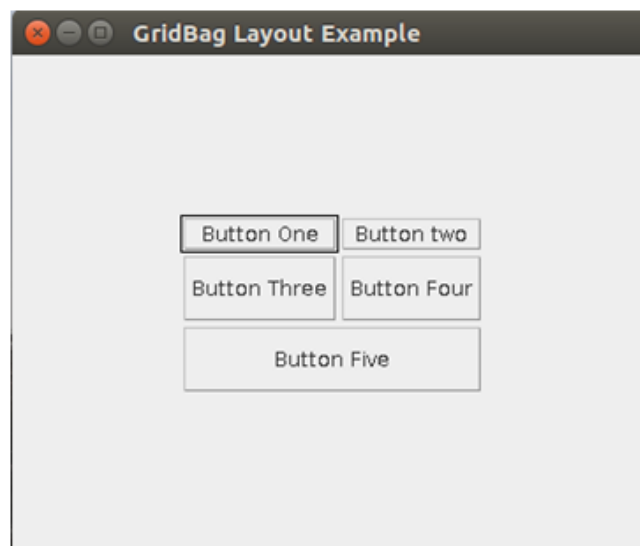
```

import java.awt.Button;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;

import javax.swing.*;
public class GridBagLayoutExample extends JFrame
{
    public static void main(String[] args)
    {
        GridBagLayoutExample a = new GridBagLayoutExample();
    }
    public GridBagLayoutExample()
    {
        GridBagLayoutgrid = new GridBagLayout();
        GridBagConstraints gbc = new GridBagConstraints();
        setLayout(grid);
        setTitle("GridBag Layout Example");
        GridBagLayout layout = new GridBagLayout();
    }
}

```

```
        this.setLayout(layout);
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.gridx = 0;
        gbc.gridy = 0;
        this.add(new Button("Button One"), gbc);
        gbc.gridx = 1;
        gbc.gridy = 0;
        this.add(new Button("Button two"), gbc);
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.ipady = 20;
        gbc.gridx = 0;
        gbc.gridy = 1;
        this.add(new Button("Button Three"), gbc);
        gbc.gridx = 1;
        gbc.gridy = 1;
        this.add(new Button("Button Four"), gbc);
        gbc.gridx = 0;
        gbc.gridy = 2;
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.gridwidth = 2;
        this.add(new Button("Button Five"), gbc);
        setSize(300, 300);
        setPreferredSize(getSize());
        setVisible(true);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }
}
```

Output:

Java Applet

Applet is a special type of program that is embedded in the webpage to generate the dynamic content. It runs inside the browser and works at client side.

Advantage of Applet

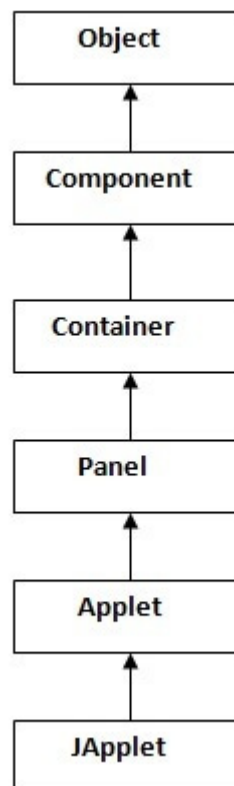
There are many advantages of applet. They are as follows:

- It works at client side so less response time.
- Secured
- It can be executed by browsers running under many platforms, including Linux, Windows, Mac Os etc.

Drawback of Applet

- Plugin is required at client browser to execute applet.

Hierarchy of Applet



As displayed in the above diagram, Applet class extends Panel. Panel class extends Container which is the subclass of Component.

Lifecycle of Java Applet

1. Applet is initialized.
2. Applet is started.
3. Applet is painted.
4. Applet is stopped.
5. Applet is destroyed.



Lifecycle methods for Applet:

The `java.applet.Applet` class provides 4 life cycle methods and the `java.awt.Component` class provides 1 life cycle method for an applet.

`java.applet.Applet` class

For creating any applet, the `java.applet.Applet` class must be inherited. It provides 4 life cycle methods for an applet.

1. **`public void init():`** is used to initialize the Applet. It is invoked only once.
2. **`public void start():`** is invoked after the `init()` method or the browser is maximized. It is used to start the Applet.
3. **`public void stop():`** is used to stop the Applet. It is invoked when the Applet is stopped or the browser is minimized.
4. **`public void destroy():`** is used to destroy the Applet. It is invoked only once.

`java.awt.Component` class

The `Component` class provides 1 life cycle method for an applet.

1. **`public void paint(Graphics g):`** is used to paint the Applet. It provides a `Graphics` class object that can be used for drawing oval, rectangle, arc, etc.

How to run an Applet?

There are two ways to run an applet:

1. By HTML file.
2. By `appletviewer` tool (for testing purpose).

Simple example of Applet by html file:

To execute the applet by html file, create an applet and compile it. After that create an html file and place the applet code in html file. Now click the html file.

```
//First.java
import java.applet.Applet;
import java.awt.Graphics;
public class First extends Applet
{
    public void paint(Graphics g)
    {
        g.drawString("welcome",150,150);
    }
}
```

myapplet.html

```
<html>
  <body>
    <applet code="First.class" width="300" height="300">
    </applet>
  </body>
</html>
```

Note: class must be public because its object is created by Java Plugin software that resides on the browser.

Simple example of Applet by appletviewer tool:

To execute the applet by appletviewer tool, create an applet that contains applet tag in comment and compile it. After that run it by: appletviewer First.java. Now Html file is not required but it is for testing purpose only.

```
//First.java
import java.applet.Applet;
import java.awt.Graphics;
public class First extends Applet
{
    public void paint(Graphics g)
    {
        g.drawString("welcome to applet", 150, 150);
    }
}
/*
<applet code="First.class" width="300" height="300">
</applet>
*/
```

To execute the applet by appletviewer tool, write in command prompt:

```
c:\>javac First.java
```

```
c:\>appletviewer First.java
```


Applications Vs. Applets

FEATURE	APPLICATION	APPLET
main() method	Present	Not present
Execution	Requires JRE	Requires a browser like Chrome
Nature	Called as stand-alone application as application can be executed from command prompt	Requires some third party tool help like a browser to execute
Restrictions	Can access any data or software available on the system	cannot access any thing on the system except browser's services
Security	Does not require any security	Requires highest security for the system as they are untrusted

Passing Parameter to Applet

We can get any information from the HTML file as a parameter. For this purpose, Applet class provides a method named `getParameter()`.

Syntax:

```
public String getParameter(String parameterName)
```

Example of using parameter in Applet:

```
import java.applet.Applet;
import java.awt.Graphics;

public class UseParam extends Applet
{
    public void paint(Graphics g)
    {
        String str = getParameter("msg");
        g.drawString(str, 50, 50);
    }
}
```

myapplet.html

```
<html>
  <body>
    <applet code="UseParam.class" width="300" height="300">
      <param name="msg" value="Welcome to applet">
    </applet>
  </body>
</html>
```

```
        </applet>  
    </body>  
</html>
```

Displaying Graphics in Applet

java.awt.Graphics class provides many methods for graphics programming.

Commonly used methods of Graphics class:

1. **public abstract void drawString(String str, int x, int y):** is used to draw the specified string.
2. **public void drawRect(int x, int y, int width, int height):** draws a rectangle with the specified width and height.
3. **public abstract void fillRect(int x, int y, int width, int height):** is used to fill rectangle with the default color and specified width and height.
4. **public abstract void drawOval(int x, int y, int width, int height):** is used to draw oval with the specified width and height.
5. **public abstract void fillOval(int x, int y, int width, int height):** is used to fill oval with the default color and specified width and height.
6. **public abstract void drawLine(int x1, int y1, int x2, int y2):** is used to draw line between the points(x1, y1) and (x2, y2).
7. **public abstract boolean drawImage(Image img, int x, int y, ImageObserver observer):** is used draw the specified image.
8. **public abstract void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used draw a circular or elliptical arc.
9. **public abstract void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used to fill a circular or elliptical arc.
10. **public abstract void setColor(Color c):** is used to set the graphics current color to the specified color.
11. **public abstract void setFont(Font font):** is used to set the graphics current font to the specified font.

Example

```
import java.applet.Applet;  
import java.awt.*;  
  
public class GraphicsDemo extends Applet  
{  
    public void paint(Graphics g)  
    {  
        g.setColor(Color.red);  
    }  
}
```

```

        g.drawString("Welcome", 50, 50);
        g.drawLine(20, 30, 20, 300);
        g.drawRect(70, 100, 30, 30);
        g.fillRect(170, 100, 30, 30);
        g.drawOval(70, 200, 30, 30);

        g.setColor(Color.pink);
        g.fillOval(170, 200, 30, 30);
        g.drawArc(90, 150, 30, 30, 30, 270);
        g.fillArc(270, 150, 30, 30, 0, 180);
    }
}

```

myapplet.html

```

<html>
    <body>
        <applet code="GraphicsDemo.class" width="300" height="300">
        </applet>
    </body>
</html>

```

JApplet

As we prefer Swing to AWT. Now we can use JApplet that can have all the controls of swing. The JApplet class extends the Applet class.

Example

```

import java.applet.*;
import javax.swing.*;
import java.awt.event.*;
public class EventJApplet extends JApplet implements ActionListener
{
    JButton b;
    JTextField tf;
    public void init()
    {
        tf = new JTextField();
        tf.setBounds(30, 40, 150, 20);
        b = new JButton("Click");
        b.setBounds(80, 150, 70, 40);
        add(b);
        add(tf);
        b.addActionListener(this);
        setLayout(null);
    }
    public void actionPerformed(ActionEvent e)
    {
        tf.setText("Welcome");
    }
}

```

myapplet.html

```

<html>

```

```
<body>
    <applet code="EventJApplet.class" width="300" height="300">
    </applet>
</body>
</html>
```

Security Issues with the Applet

Java applet is run inside a web browser. But an applet is restricted in some areas, until it has been deemed trustworthy by the end user. The security restriction is provided for protecting the user by malicious code, like copy important information from the hard disk or deleting the files. Generally, applets are loaded from the Internet and they are prevented from: the writing and reading the files on client side. Some security issues to applet are following :

- Applets are loaded over the internet and they are prevented to make open network connection to any computer, except for the host, which provided the .class file. Because the html page come from the host or the host specified codebase parameter in the applet tag, with codebase taking precedence.
- They are also prevented from starting other programs on the client. That means any applet, which you visited, cannot start any rogue process on your computer. In UNIX, applets cannot start any exec or fork processes. Applets are not allowed to invoke any program to list the contents of your file system that means it can't invoke **System.exit()** function to terminate your web browser. And they are not allowed to manipulate the threads outside the applets own thread group.
- Applets are loaded over the net. A web browser uses only one class loader that's established at start up. Then the system class loader can not be overloaded, overridden, extended, replaced. Applet is not allowed to create the reference of their own class loader.
- They can't load the libraries or define the native method calls. But if it can define native method calls then that would give the applet direct access to underlying computer.